

药材烘干过程的传热传质建模与数值仿真

摘 要

针对圆柱形中药材的热风烘干, 本文建立圆柱坐标系下一维径向的耦合抛物型偏微分方程组(含变物性的能量方程与 Fick 型水分扩散方程), 用节点中心有限体积法离散、后向 Euler 全隐式推进、Picard 迭代处理非线性, 四个问题共用同一套求解内核, 依次引入变物性、两阶段工况与干燥判据、移动边界与收缩, 保证结论之间自洽可比。本文的着力点不在求解本身, 而在题面未给定的闭合关系与结果可信度的分层检验: 输入与闭合是否可信、数值解是否可信、模型假设是否可信。

数据层面, 附件 1 的残差为高频交替型抖动、无异常点, 故不作平滑, 并在线性插值、PCHIP 与三次样条中保留不产生过冲的分段线性插值; 附件 2 的半径序列则用保单调三次 Hermite 插值, 获得不过冲、 C^1 连续的收缩几何。同时指出: 仅凭空气侧激励无法反演换热系数 h 与传质系数 h_m , 故用灵敏度分析替代参数标定, 并据测量噪声确定温度结果只有 3 位有效数字。尺度分析给出物理主线: $\alpha = 1.689 \times 10^{-7} \text{ m}^2/\text{s}$ 是 $D(C_0) = 4.938 \times 10^{-9} \text{ m}^2/\text{s}$ 的 34 倍, 1800 s 内热扩散长度 17.4 mm 已接近半径、而水分扩散长度仅 2.98 mm。

数值层面, 本文区分报告网格与求解网格: 把题面 0.1 cm 的输出网格直接当作求解网格, 会因网格误差把烘干时长低估 1.6%; 空间离散一阶收敛、时间离散误差仅 0.2%。更重要的是, 水分方程口径带来的 +6.3% 差异在 $N = 20 \sim 400$ 、 $\Delta t = 2 \sim 120 \text{ s}$ 共 20 组参数下保持不变, 据此判定它属模型层面而非离散层面, 不能靠加密网格消除, 也不应被误读为数值误差。

模型结构层面, 由恒等式 $\rho = \rho_d(1 + C)$ 证明“斜率等于截距”等价于“体积不变”, 把题面经验式的 $b < a$ 从“数据矛盾”翻译为收缩的定量标志, 并给出以斜率截距比 $r = b/a$ 为单参数的先验判据: 5% 半径容差下预热窗口阈值 $r^* = 0.164$, 附录 3 通过(线收缩 4.57%)、附录 4 已越界(5.68%); 全程窗口阈值升到 0.838, 两套物性都必须在问题四计入收缩。同一层面还判定两处题面未言明的闭合关系: 按题给传质关系构造蒸发潜热通量会把表面温度压至 6.1°C , 低于由空气状态算得的湿球温度 34.7°C , 出现非物理解, 说明题面参数体系对应忽略汽化潜热的简化; 而密度经验式与实测收缩半径隐含的干物质质量并不守恒(问题三 +116%、问题四 -10.9%), 本文如实报告并把结果定位为“题面给定数据体系下的自洽解”。

四个问题的结果: 问题一中 D 只依赖 C 、其余物性为常数, 热方程与水分方程完全解耦, 1800 s 时中心温度 33.5765°C 、表面 36.7863°C , 中心含水率始终 2.5500 kg/kg 、表面降至 1.5103 kg/kg , 水分变化限于最外侧 $3 \sim 5 \text{ mm}$; 若退化为均温均湿的集总参数模型(单个常微分方程), 脱水只需约 11.2 h, 仅为分布参数结果的 19.5%, 可见径向梯度是决定烘干时长的本体。问题二中物性随 C 、 T 变化, 两方程转为强耦合, 3 h 内基本达到烘房温度(径向温差 0.12°C); 问题三中“各处含水率低于 0.15 kg/kg ”等价于中心达标, 得烘干时长 **57.42 h (2.392 天)**, 其中最后 21 h 只换来 0.036 kg/kg 的下降, 末段由扩散系数只有芯部 $1/256$ 的干壳控制; 问题四采用实测收缩半径与贴体坐标 $\xi = r/R(t)$, 因 ξ 同时是材料坐标, 骨架对流项与坐标变换项精确抵消, 水分方程与问题二、三同口径, 得烘干时长 **51.08 h (2.128 天)**; 同物性、同口径下固定半径需 129.81 h, 即忽略收缩会把烘干时长高估为 **2.54 倍 (154%)**。

全部结论经过网格、时间步、界面取法、水分口径、参数灵敏度与干物质守恒六类自检, 误差来源可排序、结论可复核: D 的不确定度 ($\pm 20\%$ 对应 $48.9 \sim 70.0 \text{ h}$) $>$ 水分方程口径 (+6.3%/ -15.6%) $>$ 界面取法 ($N = 200$ 上 2.7%) $>$ 题面输出网格 $N = 20$ (1.6%) $>$ 端面效应 (不超过 1.1%) $>$ 时间步 (0.2%) $>$ 表面换热系数 (0.03%); 本文采用的 $N = 200$ 网格误差仅 0.07%。

关键词: 抛物型偏微分方程组; 常微分方程; 有限体积法; 数值模拟; 移动边界与收缩; 参数灵敏度分析

1 问题提出

中药材烘干以热风烘干为主 [2]，分预热平衡与恒温干燥两个阶段；工艺参数选取不当会造成效率低、能耗高、品质不稳等问题，而传统试验优化成本高、周期长，可以借助数理分析与数值仿真更快的获得干燥规律。

某圆柱形中药材长 25 cm、半径 2 cm，初始温度 28 °C、初始干基含水率 2.55 kg/kg。已知烘房空气的温度与水分浓度、烘干过程中药材半径的变化，给定问题一的物性参数，给出问题二、三、四的经验公式。需要解决：

1. 建立预热平衡阶段药材温度与水分浓度变化的数学模型，给出 1800 s 内按指定时空格点采样的结果；
2. 建立整个烘干过程（预热平衡 + 恒温干燥）的数学模型，给出 3 h 内的结果；
3. 按各处含水率低于 0.15 kg/kg 的要求确定烘干所需时间；
4. 考虑药材因失水发生的尺寸和 ρ 变化，重新确定烘干时长。

2 问题分析

四个问题本质是求解同一个物理过程的不同层面：

表 1 四个问题的逻辑关系

问题	物性来源	时间跨度	核心要求	解法
一	附录 2（恒定）	30 min	预热平衡	热、质方程解耦
二	附录 3（随 C 、 T 变化）	0-3 h	全流程耦合	强耦合，需联立迭代
三	同问题 2	2-3 天	烘干时长	反求达标时刻
四	附录 4（随 C 、 T 变化）	2-3 天	ρ 和尺寸变化	动边界，需坐标变换

问题的物理内核是含湿多孔介质内的耦合传热传质：热量由表面向内传导、水分由内向外迁移并在表面被空气带走，两者通过随含水率与温度变化的物性相互影响。三个量级分析决定了建模方向：长径比与端面占比说明可退化为一维径向问题；热扩散率与水分扩散系数相差数十倍，说明两场时间尺度不同、预热阶段以达热平衡为主；两个 Biot 数均处于中值区间，说明不能作集总参数近似。

本文的技术路线是同一个模型的四次升级：共同基础是圆柱坐标系下的一维径向耦合传热传质方程、节点中心有限体积、后向 Euler 全隐式推进与追赶法求解；在此基础上依次引入变物性（问题二）→ 两阶段工况与干燥判据（问题三）→ 移动边界与变化的 ρ （问题四）。四个问题具有一套共同的基础，保证了结论之间的自洽与可比。

3 模型假设

(A1) 介质与局部热平衡：药材视为均质、各向同性的含湿多孔介质；局部热平衡，不区分两相温度；固相体积分数允许随干燥收缩而变化，但固相自身不可压缩、干物质质量守恒。

(A2) 降维：忽略轴向输运，问题简化为一维径向轴对称。依据是长度半径比 $L/R = 12.5$ （长径比 $L/(2R) = 6.25$ ）、两端面总面积仅占侧面积的 $R/L = 8\%$ （误差分析见第 13.4 节）。

- (A3) 热源与辐射：只考虑对流换热与对流传质 [1]；烘房空气温度、湿度与流速在药材表面均匀分布， h 与 h_m 沿周向和轴向不变。
- (A4) 几何：问题一至三取固定半径 $R = 2\text{ cm}$ ，问题四按附件 2 的实测 $R(t)$ 收缩。误差分析见第 11 节。
- (A5) 物性：按各问对应的附录取值；经验式只在实验区间 $C \in [0, 2.55]$ 内使用，不作外推。
- (A6) 界面系数： h 与 h_m 题面未给出，统一沿用附录 2 的 $h = 25\text{ W}/(\text{m}^2\cdot\text{K})$ 、 $h_m = 8 \times 10^{-7}\text{ m/s}$ 。其可信度由灵敏度分析回答（第 7.6.3 节）。
- (A7) 汽化潜热：忽略蒸发吸热对表面能量平衡的贡献。依据与能量收支核算见第 5.6 节。
- (A8) 水分输运：问题一至三用 Fick 型扩散描述 [5]，表面为第三类边界条件 $-D \partial_r C|_{r=R} = h_m(C_s - C_\infty)$ ，且干基密度在方程两侧约去（第 5.6 节）；问题四考虑收缩，几何改用贴体坐标下的动区域，但水分方程仍是经典 Fick 口径——在材料坐标下对流项与坐标变换项精确抵消（式 (33)），故 (35) 与问题二的 (18) 形式相同，只多出度量因子 $R(t)^2$ 。
- (A9) 边界数据：附件 1 的烘房温湿度实测序列 不作平滑、不作差分、不剔除观测点，依据与量化影响见第 5 节。
- (A10) 收缩速率：附件 2 的半径序列改用保单调三次 Hermite 插值 (PCHIP)，理由见第 5 节。

4 符号说明

主要符号

符号	含义	单位
r, t	径向坐标、时间	m, s
T, C	药材温度、干基含水率	$^{\circ}\text{C}$, kg/kg
R, L	药材半径、长度	m
ξ	贴体坐标 $\xi = r/R(t)$	—
T_∞, C_∞	烘房空气温度、水分浓度	$^{\circ}\text{C}$, kg/kg
ρ, c_p, k	密度、比热容、热传导系数	kg/m^3 , $\text{J}/(\text{kg}\cdot\text{K})$, $\text{W}/(\text{m}\cdot\text{K})$
D	水分扩散系数	m^2/s
h, h_m	对流换热、传质系数	$\text{W}/(\text{m}^2\cdot\text{K})$, m/s
α	热扩散率	m^2/s
Bi, Bi_m	传热、传质 Biot 数	—
$r = b/a$	密度经验式的斜率截距比	—
$\beta, \varepsilon_r, \varepsilon_w$	端点收缩系数、径向线应变、水体积分数	—
λ, μ	径向、轴向特征值	1/m
w_i	有限体积控制体权重	m^2
t_{dry}	烘干时长	s

5 数据预处理与物性数据的可容许性检验

本节先作数据质量诊断，再说明附件 1 不作平滑、不作高阶插值、不作差分，而附件 2 作保单调插值的原因，并量化这些选择的影响。

5.1 附件 1 的数据质量诊断

附件 1 共 241 点、间隔 60 s、覆盖 0 ~ 14400 s。用无相位滞后的局部二次多项式分离趋势后，残差统计见表 2：滞后 1 自相关为负，说明它是传感器噪声与控温往复调节造成的高频交替型抖动，而非真实物理起伏；最大残差仅为标准差的 2.3 倍，不存在需要判别与剔除的异常点；序列无缺失、无负值，温度全局最低点即 $t = 0$ 的 28.0000 °C。因此附件 1 不需要通常意义上的清洗。

表 2 附件 1 实测序列的噪声诊断

量	趋势范围	残差标准差	最大残差	滞后 1 自相关	占全变幅
烘房温度 T_{∞}	28.00 → 50.00 °C	0.116 °C	0.27 °C	-0.31	0.52%
水分浓度 C_{∞}	0.0196 → 0.0500	1.1×10^{-4}	2.8×10^{-4}	-0.40	0.38%

5.2 附件 1 的预处理：不作平滑、保留线性插值、不作差分

第一条结论是不作平滑。表 3 给出四种处理方式对问题一结果的影响（均取 $N = 400$ 、 $\Delta t = 1$ s）其中 3 点滑动平均与 5 点 Savitzky–Golay 的偏差与边界噪声不可消除的涨落同量级、物理上不可区分，而 15 点窗口的 0.055 °C 明显超出该量级，恰印证上述判断。平滑既不改变结论、又要额外引入一个可调参数，故本文不作平滑。

表 3 不同平滑方式对问题一结果的影响（相对原始线性插值；偏差取全过程、全输出网格上的最大绝对偏差，末列为四舍五入到四位小数后发生变化的格子占比）

处理方式	含水率偏差	温度偏差	四位小数变化
原始 60 s 线性插值（本文采用）	—	—	—
3 点滑动平均（180 s）	7.1×10^{-6}	0.022 °C	温度 97%，含水率 0.3%
5 点 Savitzky–Golay（300 s）	5.9×10^{-6}	0.014 °C	温度 87%
15 点 Savitzky–Golay（900 s）	2.1×10^{-5}	0.055 °C	温度 97%

第二条结论是保留 **60 s** 分段线性插值。各种插值都严格通过观测点，差别只在两点之间（表 4），并且用现有数据无法判定哪种插值更准；而自然三次样条会把交替型噪声插入观测中并不存在的极值，产生 0.080 °C 的过冲，把烘房单调升温插成局部降温，属非物理结果。该差别虽在物理上可忽略，却大于四位小数的舍入阈值 5×10^{-5} ，会使温度表中约 87% 的格子第 4 位小数随插值方式改变，故必须锁定一种口径。本文锁定分段线性插值：无参数、无过冲、保单调，且对观测点不作改动。

表 4 点间插值方式的影响（相对分段线性插值；边界偏差为插值序列相对分段线性的最大偏离，过冲为插值曲线超出相邻样本区间的最大量）

插值方式	边界偏差	过冲	温度偏差	含水率偏差
分段线性（本文采用）	—	无	—	—
PCHIP 保单调三次	0.049 °C	无	0.0040 °C	1.9×10^{-6}
自然三次样条	0.097 °C	超调 0.080 °C	0.0054 °C	2.9×10^{-6}

第三条结论是不用差分估计变化率。温度的相邻差分标准差为 0.245 °C，除以 60 s 得 0.0041 °C/s 的伪梯度，而问题一所在的 0 ~ 1800 s 真实升温速率（线性拟合）为 0.0075 °C/s，信噪比仅约 2 : 1；差分把噪声放大 $1/\Delta t$ 倍，在这份数据上不可用。需要变化率时必须像附件 2 那样，用保形插值的解析导数给出。

5.3 恒温干燥段的外推依据

附件 1 只覆盖 $0 \sim 14400 \text{ s}$ (4 h), 而整个烘干过程长达约 57 h, 故 $t > 14400 \text{ s}$ 的边界条件必须外推。本文取常值 $T_\infty = 50^\circ\text{C}$ 、 $C_\infty = 0.05 \text{ kg/kg}$, 依据来自数据本身: 末 1 h 温度的线性拟合斜率仅 $+0.003^\circ\text{C/h}$, 末 10 个采样点在 $49.76 \sim 50.23^\circ\text{C}$ 之间围绕 50°C 抖动, 水分浓度稳定在 $0.0498 \sim 0.0502 \text{ kg/kg}$, 末 1 h 平均值分别为 49.9989°C 与 0.04999 kg/kg 。可见烘房已进入恒温恒湿状态。

5.4 附件 2 的预处理: 保单调三次 Hermite 插值

附件 2 共 145 点、间隔 1800 s , 半径由 2.000 cm 单调收缩至 1.198 cm 。半径以两种方式进入模型: 一是收缩域中的度量因子 $1/R^2$ 与表面边界条件中的 R , 二是第 11.5 节反演收缩速率、判断“收缩何时完成”所需的 $\dot{R}$ 。若直接对实测序列作差分, 相邻差分的信噪比只有 $2:1$, 噪声会被放大 $1/\Delta t$ 倍; 若用分段线性插值, R 的斜率在节点处跳变, 会使进入扩散算子与边界条件的几何量出现阶梯。因此采用 Fritsch–Carlson[6] 构造的保单调三次 Hermite 插值 (PCHIP), 它既保证 C^1 连续, 又保持数据单调、不产生过冲; 数值检验表明插值后 $R(t)$ 单调递减、 $\dot{R} \leq 0$ 恒成立 (三次样条在该序列上会出现过冲, 故不采用)。

5.5 参数不可反演与温度结果的有效位

附件 1 是实测数据, 但是无法用它反求表面换热系数 h 与对流传质系数 h_m 。这在本题中做不到。附件 1 记录的是烘房空气状态, 在模型中是边界激励; 而 h 、 h_m 出现在表面通量条件 $-k \partial_r T|_{r=R} = h(T_s - T_\infty)$ 中, 要定出它必须同时知道通量与驱动力, 而附件 1 只给出驱动力的空气侧, 药材侧的表面温度、表面含水率与通量本身全部未知, 属一个方程含两个未知量的不定方程问题。换言之, 把 h 改为任意值, 附件 1 的 241 个数据一个都不变, 参数完全不可辨识。附件 2 的 $R(t)$ 虽是唯一的药材侧实测响应, 但收缩并不等于失水 (还包含结构塌陷), 且 h_m 与扩散系数 D 高度相关、无法分离, 故单独用 $R(t)$ 反演 h_m 同样不可靠。

本题的正问题参数由附录 2 直接给定: $h = 25 \text{ W/(m}^2 \cdot \text{K)}$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$, 无需反演; 既然二者是“给定值”而非“标定值”, 正确的补救手段是灵敏度分析——四个问题的结论对二者均不敏感 (第 7.6.3 节)。由此也确定了温度结果的有效位: 边界噪声给输出带来的不可消除不确定度约为 0.02°C , 故表 5、表 7 中温度值的第 4 位小数不具物理意义、有效位为 3 位, 本文仍按题目要求保留四位小数, 但末位仅用于满足格式规范; 相比之下水分浓度场的四位小数对边界数据处理方式完全不敏感 (表 3、4), 是可靠的有效数字。

5.6 物性关联式的自洽性检验与守恒口径

附录给出的物性经验式 (ρ 、 c_p 、 k) 与 $D(C, T)$ 是后续模型的数据基础。其中体积密度 $\rho = a + bC$ 容易引起误解: 附录 3 中 $a = 650$, $b = 128$, 附录 4 中 $a = 760$, $b = 90$, 均有 $b < a$; 若把 $\rho = \rho_d(1 + C)$ 中的 ρ_d 视为常数, 似乎会得到“斜率应等于截距”, 进而怀疑题面经验式不自洽。实际上 ρ_d 是以当前湿物料总体积为基准的表现干物质密度, 只要干燥过程中体积发生变化, 它就不是常数。

本节只回答两个基础问题: 题面物性式是否具有物理可容许性, 以及 ρ 应如何进入守恒方程; 至于如何由 $\rho = a + bC$ 提取收缩信息、建立“何时可以忽略收缩”的先验判据并量化其对烘干时长的影响, 统一放在第 11 节讨论, 以避免重复推导并突出该判据的独立性。推导与数值旁证统一放在附录 B.1, 主文只保留三条结论。

1. 题面密度式并不自相矛盾, 它隐含了干燥收缩。由恒等式 $\rho = \rho_d(1 + C)$ (其中 $\rho_d = m_d/V$ 以当前总体积为基准) 可知, “斜率等于截距”只在总体积不变时才成立; 题面

两组经验式均满足 $b < a$ ，恰好说明随 C 降低、单位干物质所占总体积减小，与干燥收缩的物理图像一致。附录 B.1 另用三相体积分数与 c_p 、 k 的混合律极限作了旁证。

2. ρ 不直接写入问题二、三的水分方程。这两问采用固定几何的经典 Fick 口径（不显式描述骨架运动），干料密度在方程两侧约去；把题面 $\rho(C)$ 直接写回水分累积项会混合两套闭合假设并导致干物质守恒失配。作为口径敏感性检验，该写法使问题三烘干时长系统性延长 6.3%（表 11）。

3. ρ 仍按经验物性进入能量方程，与 $c_p(C)$ 共同构成体积热容：

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rk(C)\frac{\partial T}{\partial r}\right). \quad (1)$$

因此“ ρ 不进入水分扩散方程”并不意味着忽略题面密度数据，而是质量守恒与能量守恒中 ρ 的物理角色不同。

蒸发潜热属于同一类口径问题：完整表面能量平衡应含 $-L_v j_w$ 项，但按题给传质系数计入后表面温度会被压到湿球温度以下、与物理不相容 [1]，故本文按假设 A7 忽略汽化潜热；能量收支核算见附录 B.1。

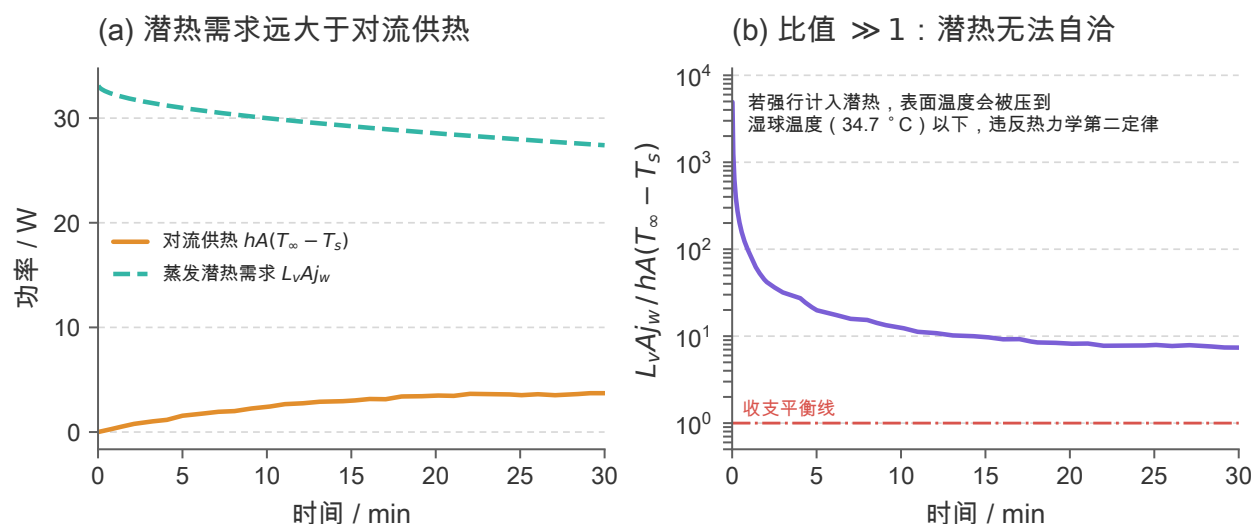


图1 附录 B.1 蒸发潜热否决的定量依据：按题给 h_m 估算的潜热需求远大于对流供热能力，比值全程大于 1，强行计入会把表面温度压到湿球温度以下。

6 问题一：预热平衡阶段药材温度与水分浓度模型

6.1 问题分析

热风烘干过程中，烘房空气通过对流把热量传给药材表面，热量再向内部传导；同时药材表层水分向空气迁移，形成由内向外的水分浓度梯度。预热平衡阶段的任务是刻画药材内部温度场 $T(r, t)$ 与水分浓度场 $C(r, t)$ 随时间和径向位置的演化规律。

题目给出的药材为圆柱体，长 $L = 25\text{ cm}$ 、半径 $R = 2\text{ cm}$ ，长径比为 6.25，两端面面积仅占侧面积的 8%，轴向输运可忽略，故可将问题降维为一维径向轴对称问题， $r \in [0, R]$ 。初始时刻药材温度 $T_0 = 28^\circ\text{C}$ 、干基含水率 $C_0 = 2.55\text{ kg/kg}$ ，烘房空气状态 $T_\infty(t)$ 、 $C_\infty(t)$ 由附件 1 给出，参数取自附录 2。

模型形态由三个量决定。由附录 2 的物性可得热扩散率

$$\alpha = \frac{k}{\rho c_p} = \frac{0.36}{820 \times 2600} = 1.689 \times 10^{-7} \text{ m}^2/\text{s},$$

而初始含水率下的水分扩散系数为

$$D(C_0) = 7 \times 10^{-9} e^{-0.89/2.55} = 4.938 \times 10^{-9} \text{ m}^2/\text{s},$$

两者相差 34 倍。在 1800 s 内，热扩散特征长度 $\sqrt{\alpha t} = 17.4 \text{ mm}$ 已接近半径 20 mm，而水分扩散特征长度 $\sqrt{Dt} = 2.98 \text{ mm}$ 还不足半径的 1/6。这说明：**30 分钟内热量已在整个截面上基本重新分布，而含水率仅在表层发生变化。这恰好是预热平衡阶段的意义所在。**

另外，换热与传质的 Biot 数分别为 $Bi = hR/k = 1.389$ 与 $Bi_m = h_m R/D(C_0) = 3.24$ ，意味着内部导热/扩散阻力与表面对流阻力都不能被忽略，不能采用集总参数近似，必须给出沿半径的分布。

6.2 模型假设与符号说明

本章符号沿用第 4 节符号表。

本章沿用第 3 节的全局假设 (A1)–(A10)，不再引入新的简化。需要强调的是，问题一的物性按附录 2 取常数，且水分扩散系数只依赖含水率 ($D = D(C)$ ，不含温度)，这一条是后文“热-质解耦”结论的物理前提；该结论在问题二起不再成立。

6.3 模型建立

6.3.1 控制方程

在圆柱坐标下，取控制体 $2\pi r \, dr \, dz$ 作能量与质量的守恒分析。热量守恒给出：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(k r \frac{\partial T}{\partial r} \right), \quad (2)$$

水分守恒给出：

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(D(C) r \frac{\partial C}{\partial r} \right). \quad (3)$$

说明：由于式 (3) 中 ρ 为常数，已在等式两侧约去。该建模选择在 §5.6 中专门说明。

6.3.2 定解条件

初始条件为均匀场

$$T(r, 0) = T_0 = 28^\circ\text{C}, \quad C(r, 0) = C_0 = 2.55 \text{ kg/kg}. \quad (4)$$

中心 $r = 0$ 处由对称性给出第二类齐次边界条件

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0. \quad (5)$$

表面 $r = R$ 处为第三类边界条件，即对流换热与对流传质

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h (T_s - T_\infty(t)), \quad (6)$$

$$-D \left. \frac{\partial C}{\partial r} \right|_{r=R} = h_m (C_s - C_\infty(t)), \quad (7)$$

其中 $T_s = T(R, t)$ 、 $C_s = C(R, t)$ 。附件 1 只覆盖 $0 \sim 14400$ s, 本问题求解区间为 $0 \sim 1800$ s, 全部落在附件范围内, $T_\infty(t)$ 与 $C_\infty(t)$ 由实测数据线性插值得到。

6.3.3 物性关系

由附录 2,

$$\rho = 820 \text{ kg/m}^3, \quad c_p = 2600 \text{ J/(kg} \cdot \text{K)}, \quad k = 0.36 \text{ W/(m} \cdot \text{K)}, \quad (8)$$

$$h = 25 \text{ W/(m}^2 \cdot \text{K)}, \quad h_m = 8 \times 10^{-7} \text{ m/s}, \quad (9)$$

$$D(C) = 7 \times 10^{-9} e^{-0.89/C} \text{ (m}^2/\text{s}). \quad (10)$$

式 (10) 的指数为负: 含水率越低扩散越慢。 C 由 2.55 降到 1.51 时, D 由 4.938×10^{-9} 降至 $3.883 \times 10^{-9} \text{ m}^2/\text{s}$ (降幅 21%); 若继续干燥到 $C = 0.15$, D 将降至 $1.855 \times 10^{-11} \text{ m}^2/\text{s}$, 跨越两个数量级。这使 (3) 成为非线性方程。

6.3.4 热-质解耦性质

在问题一中, ρ 、 c_p 、 k 均为常数, D 只依赖 C 而不依赖 T 。于是式 (2) 是常系数线性方程且不含 C , 式 (3) 是非线性方程但不含 T , 两者完全解耦, 可以分别独立求解。这是问题一区别于后续问题的关键结构性特征: 自问题二起 $D = D(C, T)$ 且 ρ, c_p, k 均为 C 的函数, 两个方程必须联立迭代求解。

6.4 无量纲分析与特征数

本题烘房温度 $T_\infty(t)$ 由附件 1 实测给出、在预热阶段随时间上升, 不存在恒定的远场温度, 故不采用以 T_∞ 为基准的温度归一化, 直接用 Fourier 数 $Fo = \alpha t / R^2$ ($\alpha = k / (\rho c_p)$) 刻画时间尺度, 用两个 Biot 数刻画表面对流阻力与内部扩散阻力之比:

$$Bi = \frac{hR}{k} = \frac{25 \times 0.02}{0.36} = 1.389. \quad (11)$$

水分方程同理, 以 Fourier 数 $Fo_m = Dt / R^2$ 与传质 Biot 数

$$Bi_m = \frac{h_m R}{D(C_0)} = \frac{8 \times 10^{-7} \times 0.02}{4.938 \times 10^{-9}} = 3.24 \quad (12)$$

表征。两个 Biot 数均为 $O(1)$, 说明表面对流阻力与内部输运阻力量级相当, 温度场和水分场在半径方向上都存在可观梯度, 必须给出分布而非平均值。

6.5 模型求解

式 (2)–(7) 为抛物型方程组, 含水率相关的非线性使解析求解不可行, 采用数值方法。

6.5.1 空间离散

采用节点中心有限体积法 [3, 4] 以保证离散守恒。将 $[0, R]$ 均分为 N 个控制体，节点 $r_i = ih_x$ ($h_x = R/N$)，内部界面位于 $r_{i\pm 1/2}$ 。控制体权重满足 $\int f dV = 2\pi L \sum_i w_i f_i$ ，其中

$$w_0 = \frac{h_x^2}{8}, \quad w_i = r_i h_x \quad (1 \leq i \leq N-1), \quad w_N = \frac{h_x(R - h_x/4)}{2}. \quad (13)$$

注意 $r_0 = 0$ ，若不单独处理中心与表面控制体权重，将会引入约 1% 的伪守恒误差。

界面上的扩散系数 $D_{i+1/2}$ 由两侧节点值构造。本文取算术平均，依据及与调和平均的定量对照见第 A.3 节。

6.5.2 时间离散与非线性处理

时间方向采用后向 Euler 格式 [3]。该格式无条件稳定，可避免 $D(C)$ 剧烈变化 (C 从 2.55 降至 0.15 时 D 跨越两个数量级) 导致的显式格式失稳。对每个时间步，将 $D(C)$ 的系数滞后处理，用 Picard 迭代求解：

$$(\mathbf{I} + \mathbf{A}(D(C^{(k)}))) \mathbf{C}^{(k+1)} = \mathbf{b}, \quad \|\mathbf{C}^{(k+1)} - \mathbf{C}^{(k)}\|_\infty < \varepsilon, \quad (14)$$

取 $\varepsilon = 10^{-13}$ ，通常 3 ~ 4 次迭代即收敛。每个迭代步化归为三对角线性方程组，用追赶法 [4] 在 $O(N)$ 时间内求解。

由于热方程与水分方程解耦，二者可在同一时间步内独立推进，总计算量约为问题二的三分之一。

6.5.3 计算参数

取细网格 $N = 400$ ($h_x = 0.005 \text{ cm}$)、时间步 $\Delta t = 1 \text{ s}$ 。题目要求的输出节点 $0, 0.1, \dots, 2.0 \text{ cm}$ 恰为细网格的子集 (对应 r_i , i 为 20 的倍数)，因此无需插值，避免引入额外误差。

6.6 结果与分析

6.6.1 温度场

表 5 给出 100 ~ 1800 s 内 5 个径向位置的温度。温度场呈现三个特征：其一，初期升温极慢，100 s 时中心仅升高 0.0001 °C，因为附件 1 中 $T_\infty(0) = 28.000 \text{ °C}$ 与药材初温完全相同，起始时刻没有温差驱动力；其二，随烘房温度上升，药材温度呈加速上升，600 s 后增速明显加快；其三，始终存在由内向外的正温度梯度，1800 s 时中心 33.58 °C、表面 36.79 °C，径向温差 3.21 °C，与 $Bi = 1.389$ 的判断一致。

6.6.2 水分浓度场

表 6 给出同一时刻的水分浓度分布。与温度场形成鲜明对比：1800 s 内中心含水率始终保持 2.5500 kg/kg (四位小数内完全未变)，0.5 cm 处仅下降 0.0003，而表面已由 2.5500 降至 1.5103，降幅 40.8%。水分变化被限制在最外侧约 3 ~ 5 mm 的薄层内，与 $\sqrt{Dt} = 2.98 \text{ mm}$ 的尺度估计吻合。

表 5 30 分钟内药材的温度 (单位: °C)

时间/s	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
100	28.0001	28.0004	28.0043	28.0332	28.1807
300	28.0415	28.0643	28.1523	28.3691	28.8498
600	28.4549	28.5376	28.8054	29.3171	30.1662
900	29.3262	29.4601	29.8771	30.6174	31.7313
1200	30.5445	30.7115	31.2239	32.1139	33.4286
1500	31.9974	32.1883	32.7674	33.7473	35.1209
1800	33.5765	33.7732	34.3653	35.3631	36.7863

表 6 30 分钟内药材的水分浓度 (单位: kg/kg)

时间/s	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
100	2.5500	2.5500	2.5500	2.5500	2.2475
300	2.5500	2.5500	2.5500	2.5492	2.0511
600	2.5500	2.5500	2.5500	2.5352	1.8771
900	2.5500	2.5500	2.5497	2.5045	1.7549
1200	2.5500	2.5500	2.5482	2.4645	1.6587
1500	2.5500	2.5499	2.5445	2.4206	1.5788
1800	2.5500	2.5497	2.5383	2.3755	1.5103

6.6.3 机理讨论

两个场的演化速率差异源于热扩散率与水分扩散系数相差 34 倍。热量在 1800 s 内的扩散长度已达半径的 87%，而水分的扩散长度仅为半径的 15%。因此“预热平衡”阶段的物理内容是药材整体趋于热平衡，水分脱除尚处于启动阶段，表面刚形成一层干燥壳层。这也解释了为何工程上需要先进行预热平衡再进入恒温干燥：若骤然大风量干燥，表层迅速失水收缩、结壳，反而阻碍内部水分的迁移通道。

6.7 本章小结

本章建立了预热平衡阶段药材内部耦合传热传质的一维径向模型，说明了问题一中热方程与水分方程解耦这一结构性质，给出无量纲特征数 $Bi = 1.389$ 、 $Bi_m = 3.24$ 与特征扩散长度，采用有限体积法配合后向 Euler 与 Picard 迭代完成求解，守恒残差达机器精度，网格与时间步均已收敛。计算结果表明：1800 s 内药材温度整体上升并在径向上形成 3.21 °C 的温差，而水分脱除仅发生在表面约 3 ~ 5 mm 的薄层内，中心含水率没有可观测的变化。完整结果保存在 result1.xlsx 中（时间 1 ~ 1800 s、径向 0 ~ 2.0 cm 每 0.1 cm，共 1800 × 21 个数据点，保留四位小数）。

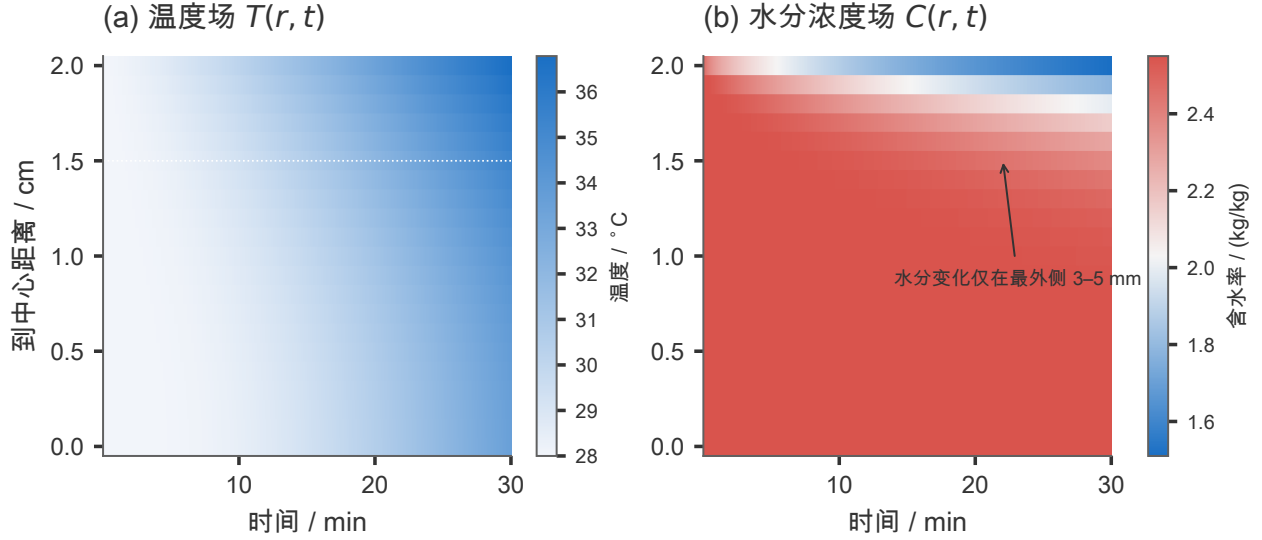


图2 问题一的温度场 $T(r, t)$ 与水分浓度场 $C(r, t)$ ($N = 400$ 、 $\Delta t = 1$ s)：30 min 内热量已接近穿透整个截面，而水分变化仅发生在最外侧 3 ~ 5 mm。

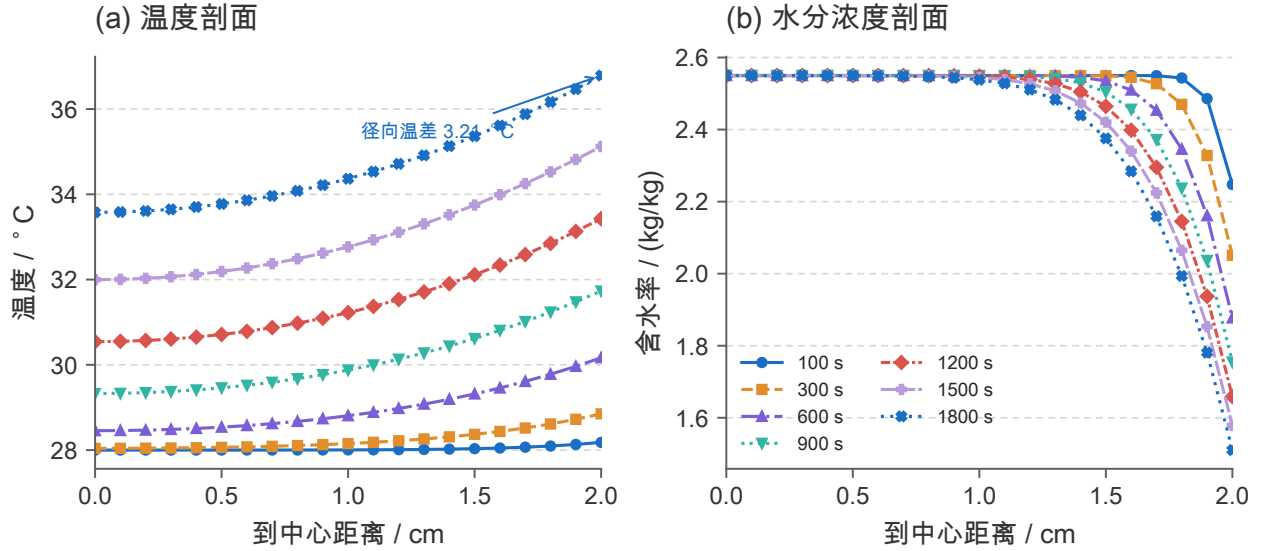


图3 问题一若干时刻的径向剖面族：(a) 温度剖面，1800 s 时径向温差 3.24 °C；(b) 水分浓度剖面，梯度集中在表面附近。

7 问题二：整个烘干过程的耦合传热传质模型

7.1 问题分析

问题二要求建立整个烘干过程（预热平衡 + 恒温干燥，持续 2 ~ 3 天）药材温度与水分浓度的变化规律。与问题一的区别在于物性不再恒定：附录 3 给出

$$\rho = 650 + 128C, \quad c_p = 1450 + 2736 \frac{C}{C+1}, \quad k = 0.21 + 0.38 \frac{C}{C+1}, \quad (15)$$

$$D = 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T}, \quad (16)$$

其中 D 同时依赖含水率 C 与温度 T (T 取绝对温标 K)， ρ 也随 C 变化。因此水分方程中的扩散系数随温度演化，而温度方程中的热容、导热系数随含水率演化，两个方程必须联立求解，问题一中“热-质解耦”的结构性质不再成立。

7.2 模型假设

在问题一假设的基础上补充与修改：

- (B1) 预热平衡阶段与恒温干燥阶段使用同一组物性关系式 (15)、(16)，两阶段的差别仅体现在烘房工况 T_∞ 、 C_∞ 上；
- (B2) 恒温干燥阶段烘房工况恒定，取 $T_\infty = 50^\circ\text{C}$ 、 $C_\infty = 0.05 \text{ kg/kg}$ （依据见 §7.3.3）；
- (B3) 问题二、三中不考虑药材尺寸变化（收缩由问题四处理），半径恒为 $R = 2 \text{ cm}$ ；
- (B4) 对流换热系数与对流传质系数题面未给出，沿用附录 2 的 $h = 25 \text{ W/(m}^2\cdot\text{K)}$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ ；
- (B5) 忽略汽化潜热，理由同问题一。

7.3 模型建立

7.3.1 控制方程

圆柱坐标下的一维径向守恒方程：

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(k(C)r\frac{\partial T}{\partial r}\right), \quad (17)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(Dr\frac{\partial C}{\partial r}\right). \quad (18)$$

关于式 (18) 的写法需要说明：未知量 C 是干基含水率 (kg 水 / kg 干料)，附录 3 给出的 D 是针对该干基含水率拟合的有效扩散系数 (m^2/s)，密度效应已被吸收进 D 。需要说明的是，这里的 $\rho_d = \rho/(1+C)$ 随 C 变化、在空间上并不均匀，并非“常数可约去”；本文采用经典 Fick 口径，把该密度效应整体计入有效扩散系数 D ，其代价由第 7.6.4 节的干物质守恒检验量化（问题三 +116%），干基密度守恒口径的对照见表 11（-15.6%）。体积密度 ρ 仍以 ρc_p 的形式进入热方程。具体误差分析见 §5.6。

式 (17) 采用体积热容 (**apparent heat capacity**) 口径：时间导数作用在温度上， ρc_p 由当前时刻的 C 取值，不把 $\rho c_p T$ 整体作为守恒量推进。问题一式 (2)、问题四式 (34) 与全部数值实现均采用同一形式，四个问题的能量口径一致。在附录给出的 $\rho(C)$ 下，若写成 $\partial_t(\rho c_p T)$ ，会额外引入 $T \partial_t(\rho c_p)$ 项，其量级与主项相当（见附录 B.1），本文不采用该口径。

7.3.2 定解条件

$$T(r, 0) = 28^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg}, \quad (19)$$

$$\left.\frac{\partial T}{\partial r}\right|_{r=0} = 0, \quad \left.\frac{\partial C}{\partial r}\right|_{r=0} = 0, \quad (20)$$

$$-k \left.\frac{\partial T}{\partial r}\right|_{r=R} = h(T_s - T_\infty(t)), \quad (21)$$

$$-D \left.\frac{\partial C}{\partial r}\right|_{r=R} = h_m(C_s - C_\infty(t)). \quad (22)$$

7.3.3 烘房工况的确定

附件 1 只覆盖 $0 \sim 14400$ s。对其末段 1 h ($10800 \sim 14400$ s) 取平均, 得 $T_\infty = 49.9989^\circ\text{C}$ 、 $C_\infty = 0.04999$ kg/kg ——与整数 50°C 、 0.05 kg/kg 几乎完全一致, 说明这正是题目设定的恒温干燥工况。因此边界条件取分段形式:

$$T_\infty(t) = \begin{cases} \text{附件 1 线性插值}, & 0 \leq t \leq 14400 \text{ s}, \\ 50^\circ\text{C}, & t > 14400 \text{ s}, \end{cases} \quad (23)$$

$C_\infty(t)$ 同理, $t > 14400$ s 后取 0.05 kg/kg。

7.4 模型求解

7.4.1 空间离散

仍采用节点中心有限体积法, 但此时每个控制体的“热容”随解变化 (ρc_p 依赖 C)。水分方程左端为 $\partial C / \partial t$; 界面上的扩散系数 D 与导热系数 k 由两侧节点值按算术平均构造, 与调和平均的对照见第 A.3 节。

7.4.2 时间离散与联立迭代

时间方向用后向 Euler, 非线性用 Picard 迭代:

$$\begin{cases} \left(I + A_C^{(k)} \right) C^{(k+1)} = b(C^n, T^{(k)}), \\ \left(I + A_T^{(k)} \right) T^{(k+1)} = b(T^n, C^{(k+1)}), \end{cases} \quad \left\| \cdot^{(k+1)} - \cdot^{(k)} \right\|_\infty < \varepsilon. \quad (24)$$

取 $\varepsilon = 10^{-11}$, 通常 $2 \sim 3$ 次迭代收敛。每一步中两个方程各化为一个三对角方程组, 用追赶法在 $O(N)$ 时间内求解。

需要强调的一点: 式 (24) 右端的时间层 n 必须是本时间步开始时的状态, 且在 Picard 迭代中保持不变; 若误用上一迭代步的结果, 则每迭代一次就相当于再推进一个 Δt , 数值解会比真实解快数倍。

7.4.3 计算参数

取 $N = 200$ (r 方向)、 $\Delta t = 5$ s, 求解至中心含水率低于 0.15 kg/kg。收敛性检验见 §7.6。

7.5 结果与分析

表 7 与表 8 分别给出 3 h 内每 0.5 h 的温度与水分浓度。

由结果可见三点。

第一, 升温呈“先慢后快”: 前期略慢于问题一, 随后迅速反超。问题一 (附录 2) 在 1800 s 时中心为 33.58°C , 问题二为 32.20°C , 反而略低 1.38°C 。这与两问物性一致: 附录 3 的初始体积热容 $\rho c_p = 3.33 \times 10^6$ J/(m³·K) 大于附录 2 的 2.13×10^6 , 初始热扩散率 1.45×10^{-7} m²/s 又小于 1.69×10^{-7} m²/s, 故前期升温更慢。但水分快速脱除使 ρc_p 显著下降 (C 由 2.55 降到 0.15 时由 3.33×10^6 降到 1.21×10^6 J/(m³·K)), 热扩散率随之提高; 同时式 (16) 中的 $e^{-3850/T}$ 使 D 随升温急剧增大, 两个效应互相强化。到 1.0 h 中心已达 40.38°C , 3 h 达 49.85°C , 后期升温明显快于问题一。

第二, 2.5 h 后药材基本达到烘房温度。3 h 时中心 49.85°C 、表面 49.97°C , 径向温差仅 0.12°C 。预热平衡阶段至此结束, 过程进入恒温干燥阶段。

表 7 3 小时内药材的温度 (单位: °C)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	32.1966	32.3893	32.9727	33.9667	35.4176
1.0	40.3817	40.5536	41.0604	41.8785	42.9983
1.5	45.8436	45.9317	46.1901	46.6024	47.1376
2.0	48.4476	48.4855	48.5961	48.7715	49.0016
2.5	49.4653	49.4775	49.5121	49.5648	49.6614
3.0	49.8492	49.8550	49.8742	49.9098	49.9663

表 8 3 小时内药材的水分浓度 (单位: kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
0.5	2.5499	2.5488	2.5253	2.3257	1.6491
1.0	2.5255	2.4945	2.3577	2.0231	1.4713
1.5	2.3858	2.3254	2.1344	1.8021	1.3477
2.0	2.1708	2.1084	1.9237	1.6260	1.2312
2.5	1.9567	1.9007	1.7361	1.4721	1.1167
3.0	1.7663	1.7166	1.5703	1.3333	1.0082

第三, 水分脱除呈现出明显的“由表及里”次序。0.5 h 时中心基本未变 (2.5499), 表面已降到 1.6491; 1 h 时中心才开始下降 (2.5255)。到 3 h, 表面降到 1.0082, 而中心仍有 1.7663, 径向梯度很大 (剖面族见图 3)。这说明过程始终由内部扩散控制。

7.6 模型检验

7.6.1 求解器交叉验证

把附录 3 的物性替换为附录 2 的常数物性后, 本模型应当退化为问题一的模型。实测: $t = 1800$ s 时温度场与独立的 `solve_p1.py` 结果最大偏差为 2.6×10^{-12} °C, 达到机器精度, 说明离散格式实现正确。

7.6.2 网格与时间步收敛性

问题三的烘干时长同时受空间网格数 N 与时间步长 Δt 的影响。本文不在此处罗列零散数据, 而是把它们各自单列一节做系统对照: 径向网格数的影响见第 A.1 节 (表 18), 时间步长的影响见第 A.2 节 (表 19)。两节同时给出本文 (经典 Fick) 口径与变密度口径的并排结果, 以便把“数值离散误差”与“模型口径差异”这两个来源分开。简要结论: 网格加密使烘干时长单调增加并近似一阶收敛; 在本文采用的算术平均格式内, $N = 200$ 已进入网格无关区 (与 $N = 400$ 相差 0.07%), 但格式间的差别收敛更慢——同为 $N = 200$ 时调和平均仍偏高 2.7% (第 A.3 节), 故“网格无关”应理解为格式内结论。时间步的影响比网格小一个数量级, Δt 由 5 s 放宽到 60 s 也仅改变 0.10%。

7.6.3 参数灵敏度分析

问题二至问题四的物性、换热与传质系数全部取自题面附录，其数值本身带有经验公式的不确定度；而题面并未提供可用于标定这些参数的独立观测。因此，“参数不准会不会改变结论”必须用灵敏度分析来回答。本文对三个关键系数 h 、 h_m 、 D 分别作 $\pm 20\%$ 扰动并重解问题三，结果见表 9。

表 9 问题三烘干时长对关键参数的灵敏度 ($N = 100$, $\Delta t = 5\text{ s}$)

扰动方式	烘干时长/s	烘干时长/h	相对基准
基准值 ($h = 25$, $h_m = 8 \times 10^{-7}$, D 按附录 3)	206365	57.32	—
h 减小 20%	206435	57.34	+0.03%
h 增大 20%	206320	57.31	-0.02%
h_m 减小 20%	212560	59.04	+3.0%
h_m 增大 20%	202555	56.27	-1.8%
D 减小 20%	252070	70.02	+22.2%
D 增大 20%	176175	48.94	-14.6%

表 9 给出三个结论。

第一，过程由内部水分扩散控制。 D 是唯一强敏感参数： D 变化 20%，烘干时长在 48.9 ~ 70.0 h 之间摆动，跨度接近一整天。

第二，表面换热系数 h 几乎不影响结果（变化仅 $\pm 0.03\%$ ）。原因是热扩散率比水分扩散系数大 34 倍，温度场在整个过程中始终接近准稳态，药材温度由空气温度直接决定， h 的大小只是把这一平衡推进或推迟几分钟。

第三，对流传质系数 h_m 只带来 2 ~ 3% 的影响，说明表面传质阻力不是限制环节，内部扩散才是。

这组结果的实际意义是：即使附录 2 给出的 h 、 h_m 存在成倍误差，问题三、问题四的结论也基本不变，这正面回应了“参数无标定则模型不可信”的质疑；同时也提示 D 的取值口径与单位必须严格无误——附录 3、4 中 D 的经验公式以绝对温标 T (K) 为自变量，若误用摄氏度将造成数量级错误。

7.6.4 基于附件 2 实测数据的守恒性检验

前面的守恒检验、收敛性检验与交叉验证都属于模型内部的自治性检验。可用的实测数据只有附件 1（烘房边界）与附件 2（药材半径），二者在模型中分别充当激励与几何输入，并非独立的观测量。但其中仍有一个可检验的物理量：干物质质量守恒。药材在烘干过程中只失去水分，固体骨架不流失，故单位长度内的干料质量

$$m_d(t) = \int_0^{R(t)} \frac{\rho(C)}{1+C} 2\pi r \, dr \quad (25)$$

应恒等于其初值。本文用已求得的 $C(r, t)$ 与附件 2 的实测 $R(t)$ 计算式 (25)，结果见表 10。

表 10 干物质质量的守恒性检验（相对初值）

时刻	问题三（固定半径，附录 3）	问题四（含收缩，附录 4）
12 h	+89%	-23.1%
30 h	+110%	-13.8%
结束时刻	+116%	-10.9%

结果表明：两组给定数据在干物质守恒意义下存在固有张力。根源在于附录 3、4 的密度经验式本身已内含收缩效应——干基密度 $\rho/(1+C)$ 随含水率下降由初值 275 kg/m^3 升至

约 600 kg/m^3 ($C = 0.15$ 的芯部点值为 582, 体积加权平均约 600; $\rho_d \equiv \rho/(1+C)$ 的定义见第 5.6 节); 而问题二、三又要求固定半径, 二者叠加便隐含了干物质质量的增长。这是题面两组给定数据之间的固有张力, 并非本文模型引入, 也不是“题目有错”——按第 5.6 节的分析, 这两式各自都是合理的实验拟合式, 只是隐含了不同的体积假设; 问题四因为直接采用了附件 2 的实际收缩, 把偏差压缩到 -11% 以内 (12 h 时一度到 -23.1% , 随后随半径停止收缩而回升)。

该结果说明两点。其一, 本文给出的水分浓度场与烘干时长应理解为“题面给定数据体系下的自洽解”, 而非绝对预测值; 其二, 若要求严格的质量守恒, 应改用干基参考框架重写水分方程并放弃 $\rho(C)$ 的经验式, 这已超出题面所给信息的范围。本文如实报告该偏差, 而不将其掩盖。

7.6.5 水分方程口径的敏感性

问题二起水分方程采用经典 Fick 形式 $\partial C/\partial t = \nabla \cdot (D \nabla C)$, 其中 C 为干基含水率、 D 为针对该干基含水率拟合的有效扩散系数, 体积密度 ρ 只以 ρc_p 的形式进入热方程、不进入水分方程。题面未明确规定 ρ 在水分方程中的作用, 为评估该口径对结论的影响, 本文另按两种“把密度写回水分方程”的常见口径重解问题三, 结果见表 11: 其一为直接代入附录的体积密度 $\partial(\rho C)/\partial t = \nabla \cdot (\rho D \nabla C)$, 其等效扩散系数为 $\rho D/(\rho + C\rho')$; 其二为按干基密度 $\rho_d = \rho/(1+C)$ 写守恒形式, 其等效扩散系数为 $\rho_d D/(\rho_d + C\rho'_d)$, 在 C_0 处约 $1.62D$ 、至判据 $C = 0.15$ 处回落到 $1.11D$ 。

表 11 水分方程口径对问题三烘干时长的影响 ($N = 100$ 、 $\Delta t = 5 \text{ s}$; $N = 200$ 时本文口径为 57.42 h、变密度口径为 61.02 h (+6.3%)、干基密度守恒口径为 48.47 h (-15.6%); 三种口径的相对差在两套网格上完全一致)

水分方程口径	烘干时长/h	相对基准
经典 Fick 形式 $\partial C/\partial t = \nabla \cdot (D \nabla C)$ (本文)	57.32	—
$\partial(\rho C)/\partial t = \nabla \cdot (\rho D \nabla C)$	60.93	+6.3%
干基密度守恒形式 $\partial(\rho_d C)/\partial t = \nabla \cdot (\rho_d D \nabla C)$	48.39	-15.6%

三种口径给出的烘干时长均在题面所述 2 ~ 3 天区间内, 四个问题的定性结论不变; 其中把体积密度 ρ 写回水分方程会延长 6.3%, 改按干基密度 ρ_d 的守恒形式则缩短 15.6%。后者与 D 的 $\pm 20\%$ 灵敏度同量级 (D 的扰动给出 48.9 ~ 70.0 h), 但大于网格与时间步引入的数值误差 ($\leq 1.6\%$ 与 $\leq 0.2\%$), 因此本文在此明确声明所采用的口径, 以便复核。进一步的计算表明, 该口径差并不随数值参数漂移: 在 $N = 20 \sim 400$ 、 $\Delta t = 2 \sim 120 \text{ s}$ 的全部组合中, 把体积密度 ρ 写回水分方程的口径相对本文稳定在 +6.3% (见第 A.1、A.2 节)。

8 烘干时间的确定（问题三）

8.1 问题分析

烘干要求是“药材各处的水分浓度应低于 0.15 kg/kg ”。由于水分由表面向内部逐步脱离, 整个过程中心处的含水率始终最高, 因此判据等价于

$$t_{\text{dry}} = \min \left\{ t \mid \max_{0 \leq r \leq R} C(r, t) < 0.15 \right\} = \min \{ t \mid C(0, t) < 0.15 \}. \quad (26)$$

数值上取求解过程中首次满足 $C(0, t) < 0.15$ 的时刻。

8.2 结果

模型给出的烘干时长为

$$t_{\text{dry}} = 206700 \text{ s} = 57.42 \text{ h} = 2.392 \text{ 天}.$$

(27)

该结果与题面”烘干过程一般持续 2 ~ 3 天”的描述一致，是对模型合理性的一个独立佐证。表 12 给出每隔 6 h 的水分浓度分布。

表 12 药材烘干过程的水分浓度（单位：kg/kg）

时间/h	到药材中心的距离/cm				
	0	0.5	1.0	1.5	2.0
6	1.0171	0.9883	0.9016	0.7551	0.5329
12	0.4562	0.4432	0.4031	0.3298	0.1642
18	0.2989	0.2916	0.2687	0.2253	0.0846
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1891	0.1640	0.0599
36	0.1858	0.1825	0.1717	0.1503	0.0567
42	0.1720	0.1691	0.1596	0.1406	0.0549
48	0.1618	0.1591	0.1506	0.1333	0.0537
54	0.1538	0.1514	0.1436	0.1276	0.0530
烘干结束	0.1500	0.1477	0.1402	0.1248	0.0527

8.3 结果分析

从表 12 可以看出烘干过程的三个典型阶段。

快速干燥段（0 ~ 12 h）：含水率大幅下降，中心由 2.55 降到 0.4562。这一段虽失水最快，却并非表面控制下的恒速干燥段：表面水分始终充足、限制环节在内部扩散，故自始至终处于降速状态——中心含水率在 0 ~ 6 h 下降 1.53，6 ~ 12 h 只下降 0.56，速率已接近腰斩。

降速干燥段（12 ~ 36 h）：速率明显放缓，中心由 0.4562 降到 0.1858。表面含水率已接近空气平衡值（0.05 附近），驱动力大幅衰减；同时式 (16) 中 $e^{-0.45/C}$ 使 D 随含水率降低而急剧减小，内部水分越来越难迁移。

极慢干燥段（36 ~ 57.4 h）：中心含水率由 0.1858 缓缓逼近 0.15，耗时约 21 h，占全程的 37%。这一段在工程上是最”不划算”的：大量时间只换来很小的含水率下降。若对产品均匀性要求可以放宽，适当延长或改用变温工艺会显著节能。

需要指出，判据由中心决定，而中心恰是最难干燥的位置。57.42 h 时有 $C(0) = 0.1500$ 、 $C(0.5 \text{ cm}) = 0.1477$ 、 $C(2 \text{ cm}) = 0.0527$ ，表面已干燥到接近平衡而中心刚刚达标，内外含水率相差近三倍，干燥全过程的曲线见图 4。

9 考虑尺寸变化的烘干模型（问题四）

9.1 问题分析

实际烘干中水分流失会引起药材收缩。附件 2 给出了半径随时间的实测变化： R 由 2.000 cm 单调收缩至 1.198 cm，收缩比 0.599，对应截面积减为 0.359 倍。

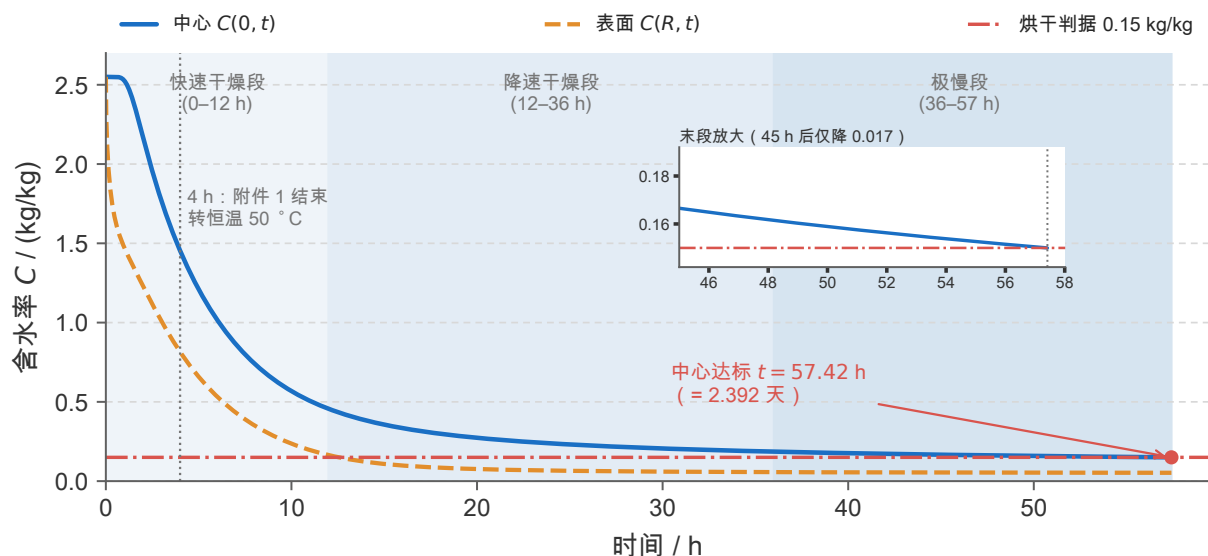


图 4 问题三干燥曲线：中心与表面水分浓度随时间的变化（ $N = 200$ 、 $\Delta t = 5$ s，判据为各点 $C < 0.15$ kg/kg），右上角插图为 45 h 之后的放大。

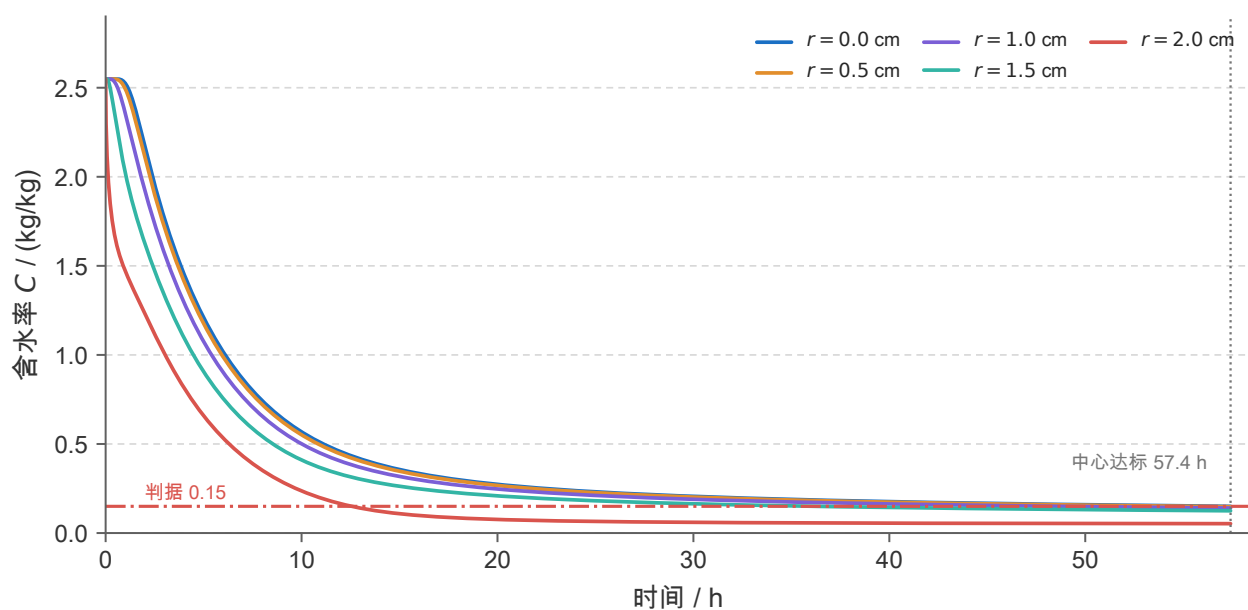


图 5 问题三 5 个径向位置的含水率历时曲线（ $N = 200$ 、 $\Delta t = 5$ s）：中心最后达标（57.4 h），表面在 12.6 h 即低于判据，此后由于干壳控制。

收缩带来了三个变化：其一，扩散路径变短，有利于干燥；其二，表面几何面积减小，但对单位体积而言换热传质面积反而增大；其三，计算区域随时间缩小，属于动边界问题，需要专门的坐标处理。

附件 2 的一个关键特征是收缩高度前置：完成 50% 的收缩只需 2.5 h，90% 只需 10 h，99% 在 22 h 内完成，之后到 72 h 半径几乎不再变化。也就是说，收缩集中在干燥前期，而含水率的缓慢下降一直持续到 50 h 之后。因此半径与含水率之间不存在简单的线性关系，不宜用 $R = R_0 C / C_0$ 一类的经验式，本文直接采用附件 2 的实测 $R(t)$ 。

9.2 模型假设

在问题二假设的基础上修改：

(C1) 药材各点按比例均匀收缩，半径由附件 2 的 $R(t)$ 给定，即任意时刻的物理半径为 $r = \xi R(t)$ ， $\xi \in [0, 1]$ 为贴体坐标；

(C2) 物性改用附录 4 的经验公式

$$\rho = 760 + 90C, \quad c_p = 1850 + 2150 \frac{C}{C+1}, \quad k = 0.12 + 0.20 \frac{C}{C+1}, \quad (28)$$

$$D = 4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T}, \quad (29)$$

(C3) 初始半径 $R(0) = 2.000 \text{ cm}$ ，与问题二一致，故初始条件仍为 $T = 28^\circ\text{C}$ 、 $C = 2.55 \text{ kg/kg}$ 。

9.3 模型建立

9.3.1 贴体坐标变换：对流项精确抵消

令

$$\xi = \frac{r}{R(t)} \in [0, 1], \quad (30)$$

把移动区域 $[0, R(t)]$ 映射为固定区域 $[0, 1]$ 。由于假设 (C1) 规定各点按同一比例收缩， ξ 同时是材料坐标：同一物质点在过程中保持 ξ 不变，其物理速度为 $u = \xi \dot{R}$ 。

这里有一个容易被漏掉、但决定方程形式的关键点。在物理坐标下，温度与水分满足带骨架对流的守恒关系，时间导数应写成物质导数

$$\frac{D}{Dt} = \frac{\partial}{\partial t} \Big|_r + u \frac{\partial}{\partial r}, \quad (31)$$

而坐标变换本身给出

$$\frac{\partial}{\partial t} \Big|_r = \frac{\partial}{\partial t} \Big|_\xi - \frac{\xi \dot{R}}{R} \frac{\partial}{\partial \xi}, \quad \frac{\partial}{\partial r} = \frac{1}{R} \frac{\partial}{\partial \xi}, \quad (32)$$

其中 $\dot{R} = dR/dt < 0$ 。两式相加，右端第二项与 $u \partial_r = \frac{\xi \dot{R}}{R} \partial_\xi$ 恰好抵消：

$$\frac{D}{Dt} = \frac{\partial}{\partial t} \Big|_\xi. \quad (33)$$

也就是说，材料坐标下的时间导数本身就是物质导数，方程中不应再出现任何额外的“挤压”对流项；算子的空间部分只须把 $r = \xi R$ 代入散度。代入式 (17)、(18) 并整理，得

$$\rho c_p \xi R^2 \frac{\partial T}{\partial t} = \frac{\partial}{\partial \xi} \left(k \xi \frac{\partial T}{\partial \xi} \right), \quad (34)$$

$$\xi R^2 \frac{\partial C}{\partial t} = \frac{\partial}{\partial \xi} \left(D \xi \frac{\partial C}{\partial \xi} \right). \quad (35)$$

与问题二、三的方程组相比，唯一的差别是度量因子由 R_0^2 变成 $R(t)^2$ 、表面边界条件多一个 R 。水分方程仍不含体积密度 ρ ，但在问题四里这一步是严格推出的而非沿用：由假设 (C1) 各点按同一比例收缩，任意物质微元的体积以同一因子缩放，干物质质量守恒使 $\rho_d = \rho/(1+C)$ 只随时间变化、在空间上均匀。于是材料坐标下 $\rho_d \partial_t C|_\xi = \frac{1}{r} \partial_r (r \rho_d D \partial_r C) = \rho_d \frac{1}{r} \partial_r (r D \partial_r C)$ ， $\rho_d(t)$ 可从散度中提出并在两侧约去，得到式 (35)；这与 ρ_d 是否随时间变化无关。这样四个问题共用同一个物质输运口径，问题四与问题三的差异只有几何一个因素，二者的比较才是严格单因素的。若令 $\dot{R} \equiv 0$ ，式 (34)、(35) 严格退化为问题二的方程组。

9.3.2 定解条件

初始条件与中心对称条件同式 (19)、(20)。表面 $\xi = 1$ 处边界条件形式不变：

$$-k \left. \frac{\partial T}{\partial \xi} \right|_{\xi=1} = hR(T_s - T_\infty), \quad -D \left. \frac{\partial C}{\partial \xi} \right|_{\xi=1} = h_m R(C_s - C_\infty). \quad (36)$$

9.3.3 $R(t)$ 的处理

附件 2 为每 1800 s 一个点的离散数据。 $R(t)$ 直接进入度量因子 $1/R^2$ 与表面边界条件，且第 11.5 节要用它的导数 $\dot{R}$ 反演收缩速率。若直接线性插值， R 的斜率在节点处跳变，会让进入方程系数的几何量出现阶梯；若用差分估计 $\dot{R}$ ，噪声会被放大 $1/\Delta t$ 倍。因此本文采用单调三次 **Hermite** 插值（**PCHIP**, **Fritsch–Carlson** 构造）[6]，它既保证 C^1 连续，又保持数据的单调性、不产生过冲。收缩终止的时刻由数据本身决定，无需外推。

9.4 数值实现

空间离散仍用节点中心有限体积，但控制体权重变为 $w_i = \int \xi \, d\xi$ ，即

$$w_0 = \frac{h^2}{8}, \quad w_i = \xi_i h \quad (1 \leq i \leq N-1), \quad w_N = \frac{h(1-h/4)}{2}. \quad (37)$$

收缩只通过每个时间步的 $R(t_n)$ 进入度量因子与表面边界条件，离散格式与问题二逐项相同；时间方向与非线性处理同问题二，取 $N = 200$ 、 $\Delta t = 5 \, \text{s}$ 。

9.5 结果

模型给出的烘干时长为

$$t_{\text{dry}} = 183885 \, \text{s} = 51.08 \, \text{h} = 2.128 \, \text{天}, \quad (38)$$

比问题三的 57.42 h 缩短了约 11.0%。但该差值同时包含物性由附录 3 换为附录 4 的影响，不能全部归因于收缩；收缩的独立贡献须在同一物性下比较，见下节。

表 13 药材烘干过程的水分浓度（含收缩，单位：kg/kg）

时间/h	到药材中心的距离/cm					药材表面
	0	0.5	1.0	1.5	2.0	
6	1.7188	1.5371	1.0222	——	——	0.4205
12	0.7375	0.6544	0.4082	——	——	0.1669
18	0.4087	0.3686	0.2397	——	——	0.0892
24	0.2851	0.2610	0.1789	——	——	0.0673
30	0.2263	0.2094	0.1492	——	——	0.0595
36	0.1928	0.1796	0.1316	——	——	0.0560
42	0.1712	0.1603	0.1199	——	——	0.0541
48	0.1561	0.1468	0.1115	——	——	0.0530
烘干结束	0.1500	0.1413	0.1080	——	——	0.0526

表 13 中 1.5 cm 与 2.0 cm 两列以“——”表示，原因是半径收缩至 1.5 cm 以下之后（约 3.6 h）这些位置已不在药材内部，水分浓度无定义（移动边界下的含水率场见图 6）。这也是题面表 6 用“药材表面”而非固定距离作为末列的原因：表面位置本身是随时间移动的。

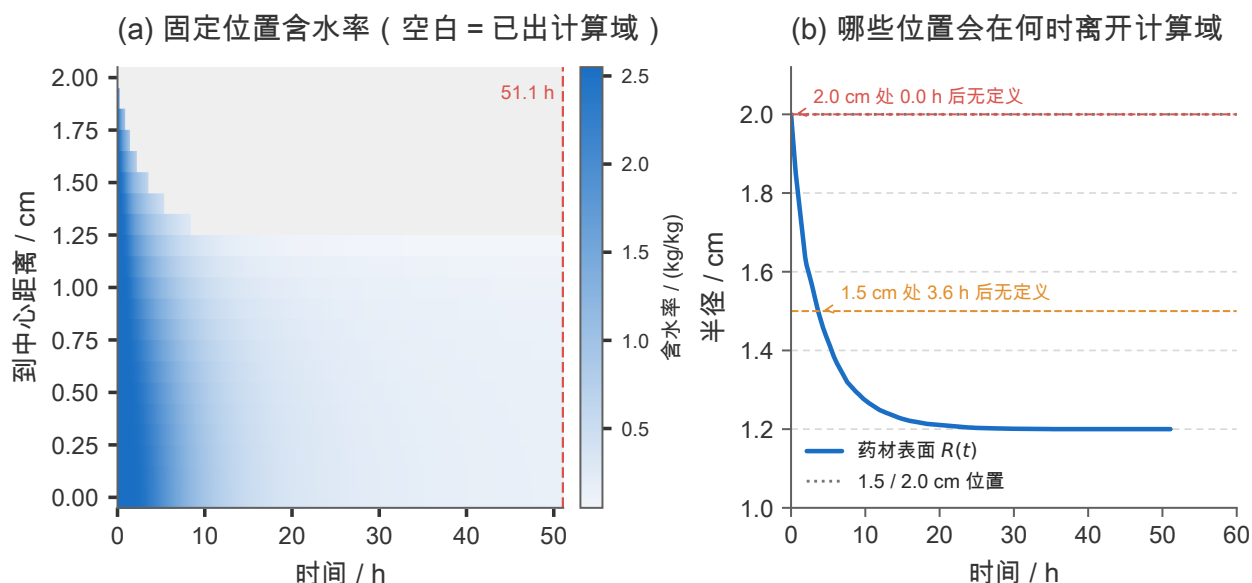


图 6 问题四移动边界下的含水率场：(a) 固定位置含水率，空白表示该位置已随收缩离开计算域；(b) 各半径位置的失效时刻，表面由 2.0 cm 收缩到 1.198 cm。

9.6 收缩对烘干时长的影响

为分离收缩的贡献，用附录 4 的同一组物性、同一套水分方程口径，只把半径固定为 $R \equiv 2\text{ cm}$ 再求解一次，得烘干时长 $467260\text{ s} = 5.41\text{ 天}$ ；计入收缩后为 2.13 天，缩短为原来的 39.4%。因为两次求解只差几何一个因素，这个差值可以全部归因于收缩。

这一效应可从两个层面理解：扩散路程由 R 缩短至 $0.6R$ ，特征扩散时间按 R^2 计减少到 36%；单位体积的表面积按 $1/R$ 计增大约 1.67 倍，表面交换能力相对增强。

这一结论有明确的工程含义：忽视收缩会严重高估烘干时长（本例中忽略收缩的预测时长为计入收缩时的 2.54 倍，即高估约 154%）。若按不考虑收缩的结果安排生产，会造成过度干燥、能耗浪费，甚至因干裂而降低药材品质。收缩历程与两种情形的对照见图 7。

9.7 模型检验

1. 退化检验：令 $\dot{R} \equiv 0$ 并把附录 4 物性替换为附录 2 的常数物性，式 (34)、(35) 退化为问题一模型，程序实测退化解与问题一解完全一致，说明材料坐标下的动区域离散实现正确。
2. 口径一致性检验：问题四的水分方程为式 (35)，与问题二、三的经典 Fick 口径完全相同，只有度量因子与表面边界条件带 $R(t)$ ；因此问题四与问题三的对照是严格的单因素（几何）对照。
3. 单调性检验：PCHIP 插值后的 $R(t)$ 单调递减，未出现插值过冲引起的非物理膨胀。
4. 合理性检验：收缩使烘干时长缩短，与“扩散路径变短有利于干燥”的物理预期一致；问题三 57.42 h、问题四 51.08 h 均落在题面所述 2 ~ 3 天区间内。

10 模型的检验与灵敏度分析

本文对数值方案做了三类系统检验：空间网格 $N = 20 \rightarrow 400$ 、时间步 $\Delta t = 2 \sim 120\text{ s}$ 、界面扩散系数取法（算术 / 对数 / 调和平均）。完整对照数据、逐档收敛过程与讨论见附录 A，这里只列对结论有影响的四条。

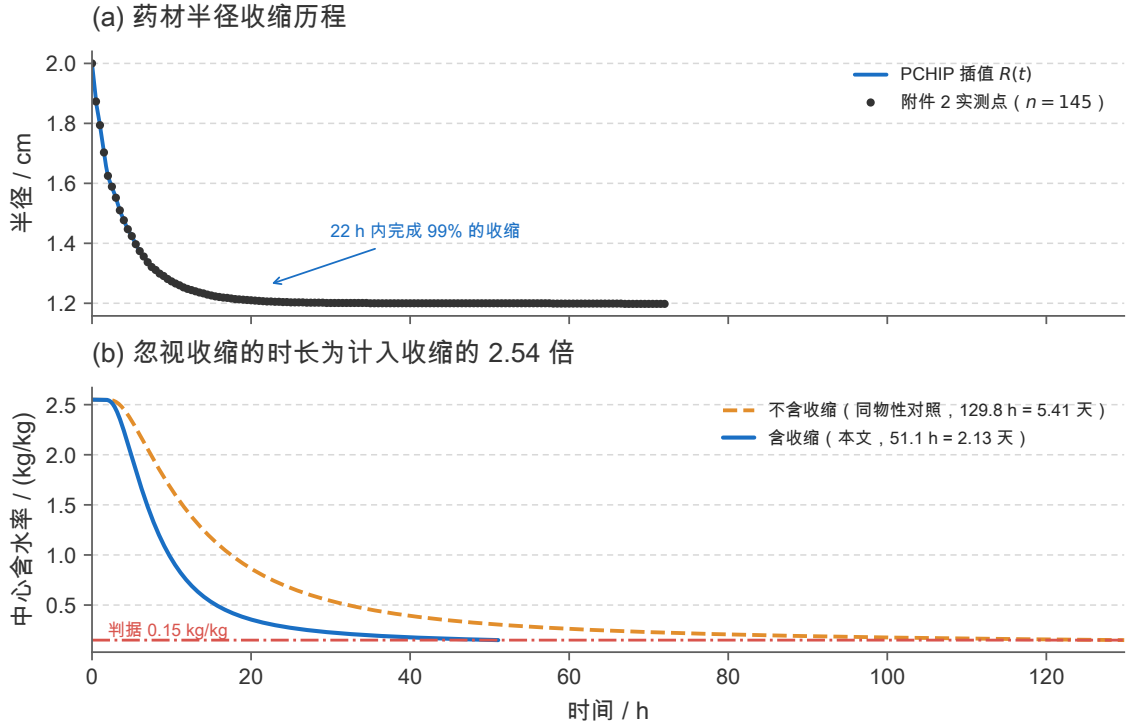


图7 问题四：收缩历程与烘干时长的对照（附录4物性， $N = 200$ 、 $\Delta t = 5$ s）。(a) 附件2的实测半径（ $n = 145$ ，间隔1800 s，实测点以散点示出）与保单调三次 Hermite 插值曲线；插值后 $\dot{R} \leq 0$ 恒成立、无过冲。(b) 中心水分浓度随时间的变化：含收缩时51.08 h达标，固定半径的对照工况需129.81 h，收缩使烘干时长缩短约61%。两图均为确定性数值结果，不含统计误差。

1. 报告网格不等于求解网格。题面0.1 cm的输出网格对应 $N = 20$ ，直接用它求解会把烘干时长低估1.6%（203335 s对206700 s）；本文在 $N = 200$ 上求解后再插值到输出网格。
2. 本文配置下网格已足够密。 $N = 200$ 与 $N = 400$ 的烘干时长相差0.07%，相邻增量逐次减半；时间步在60倍跨度内的最大影响不超过0.2%。
3. 格式无关性慢于格式内收敛。界面取法在 $N = 200$ 上仍有2.7%的残差（调和平均相对算术平均，58.96 h对57.42 h），加密到 $N = 400$ 后降到0.4%。因此“网格无关”只是算术平均格式内的结论，界面取法的不确定性在误差排序中高于网格。
4. 口径差与数值参数无关。把体积密度写回水分方程使烘干时长稳定延长+6.3%，在 $N = 20 \sim 400$ 、 $\Delta t = 2 \sim 120$ s的20组参数组合下不漂移，说明它属模型口径而非离散误差。

11 收缩的建模判据与代价

11.1 本节要回答的问题

四个问题用了两套密度经验式：问题二、三用附录3的 $\rho = 650 + 128C$ ，问题四用附录4的 $\rho = 760 + 90C$ ；对收缩的处理也不同——前三问取固定半径，问题四计入附件2的实测收缩。这一分工在题面中已经给定，但有一个问题值得正面回答：能否只看经验式的系数，就判断某一问该不该考虑收缩？本节说明答案是肯定的，判据只依赖一个无量纲量——斜率与截距之比 $r \equiv b/a$ ；并把“考虑收缩”的代价量化成烘干时长的改变量，从而给出一条可以照搬到假设一节的分层规则。

11.2 收缩几何只由斜率截距比决定

以 1kg 干物质为基准, 由恒等式 $\rho = \rho_d(1 + C)$ 与 $\rho_d = m_d/V$ 立得单位干物质体积

$$V(C) = \frac{1 + C}{\rho(C)} = \frac{1 + C}{a + bC}, \quad (39)$$

于是从初态 C_0 到当前 C 的体积比为

$$\frac{V(C)}{V(C_0)} = \frac{(1 + C)(a + bC_0)}{(1 + C_0)(a + bC)} = \frac{(1 + C)(1 + rC_0)}{(1 + C_0)(1 + rC)}, \quad r = \frac{b}{a}, \quad (40)$$

分子分母同除以 a 后截距被完全约去。

命题 (单参数约化) 相对体积 $V(C)/V(C_0)$ 、由它决定的相对线尺度以及动边界 $R(t)$ 的形状, 都只通过 $r = b/a$ 依赖密度经验式; 截距 a 与系数的绝对大小不进入几何。这说明把经验式整体等比例放大或缩小 (a 、 b 同乘一个常数) 不改变任何收缩量。几何约定上, 本文问题四取长度不变、只缩半径, 故 $R/R_0 = [V/V_0]^{1/2}$; 若按各向同性收缩则为立方根, 两者相差约三分之一 (附录 3 的绝干线收缩由 35.0% 变为 24.9%)。全文只用前一种口径。

两套经验式的 r 分别为 $r_3 = 128/650 = 0.197$ 与 $r_4 = 90/760 = 0.118$ 。它与附件 2 实测收缩的对照可由端点值给出。定义端点收缩系数

$$\beta \equiv \frac{\Delta V/V_0}{\varepsilon_w^0} = \frac{1 - V(0)/V(C_0)}{\varepsilon_w^0}, \quad \varepsilon_w^0 = \frac{C_0}{\rho_w^0} \cdot \frac{a + bC_0}{1 + C_0}, \quad (41)$$

即“绝干时总体积收缩量与初始水体积之比”。由式 (40) 可解出 $\beta(r) = (\rho_w^0/a)(1-r)/(1+rC_0)$, 代入得 $\beta_3 = 0.822$ 、 $\beta_4 = 0.891$ 。 β 携带绝对物理含义、需要密度尺度 a 与水的真密度; 而“该不该忽略收缩”是纯几何决策, 只有 r 是必要的。

11.3 两级判据

相对体积 $V(C)/V(C_0)$ 只通过 $r = b/a$ 依赖密度经验式, 由此可把“该不该考虑收缩”写成求解之前即可计算的判据。完整闭式推导、阈值表与参数扫描见附录 B.4, 结论有两条。

(1) **几何判据**。给定半径容差 ε 与工作窗口 $[C_{\min}, C_0]$, 忽略收缩安全当且仅当 $r \geq r^*(\varepsilon, C_{\min})$ 。5% 径向容差下, 预热窗口 ($C: 2.55 \rightarrow 2.0$) 的阈值为 0.164: 附录 3 ($r_3 = 0.197$) 通过, 该窗口内线收缩仅 4.57%; 附录 4 ($r_4 = 0.118$) 已经越界, 同一窗口的线收缩为 5.68%, 略超容差。问题二窗口 ($C: 2.55 \rightarrow 1.364$) 阈值升到 0.507、全程窗口 ($C \rightarrow 0.15$) 升到 0.838, 两式均已落到阈值之下——无论用哪套物性, 深干燥段的收缩都不可忽略。这正是本文把问题四单列的原因, 也说明“预热段可以忽略收缩”只在附录 3 的物性下严格成立。

(2) **时长判据**。把半径容差升级为烘干时长误差 Δ , $\Delta \leq 5\%/10\%/20\%$ 分别要求 $r \geq 0.913/0.835/0.704$, 比几何判据更严 (半径以平方进入特征扩散时间)。题面两式按任一口径都远低于阈值; 附录 B.4 的参数扫描还给出: 固定几何时 r 对烘干时长几乎无影响 (全变程 0.24%), 一旦计入收缩, 影响被放大到 $+3\% \sim +16\%$ 。

11.4 三个开关对烘干时长的相对重要性

把上面的分析落到本问的物性体系上, 可以给出三个“开关”对烘干时长的实际影响 (表 14)。所有数字都保持物性体系不变, 只切换被测的那个量。

三点结论。

(1) “是否考虑收缩”是三者中影响最大的开关。在同一套物性下, 计入由经验式反演的收缩使烘干时长缩短 48.6%, 计入实测收缩缩短 58.4%; 而把体积密度写回水分方程只延

表 14 建模开关对烘干时长的影响（物性体系固定，只切换被测项）。”口径”一列见第 5.6 节，”几何”三行用附录 3 物性、 $N = 100$ 、 $\Delta t = 5\text{ s}$ 求解

开关	取值	烘干时长	相对基准
几何	固定半径 R_0 （本文问题二/三）	57.32 h	—
	由 $\rho = a + bC$ 反演 $R(C)$ 自洽闭合	29.45 h	-48.6%
	附件 2 实测 $R(t)$ （问题四的几何）	23.85 h	-58.4%
水分方程口径	经典 Fick（本文）	57.32 h	—
	体积密度守恒 $\partial(\rho C)/\partial t$	60.93 h	+6.3%
参数	$D \pm 20\%$ （第 7.6.3 节）	48.9 ~ 70.0 h	-15% ~ +22%

长 6.3%、改按干基密度守恒反而缩短 15.6%， D 变化 $\pm 20\%$ 带来 $-15\% \sim +22\%$ 。收缩效应比口径效应（6.3% ~ 15.6%）大 3 倍以上，也大于参数不确定度。

(2) ”是否考虑 ρ ”与”是否考虑收缩”不是两个独立的开关。由恒等式 $\rho = \rho_d(1 + C)$ 、 $\rho_d = m_d/V$ 可知：几何固定时把体积密度写进水量守恒，等价于要求 V 不变而 ρ 随 C 变，这会使干物质质量随干燥过程增长——第 7.6.4 节的检验给出问题三 +116% 的漂移。所以严格地说，只有”几何收缩 + ρ 守恒形式”这一组合是自洽的；固定几何下的正确做法是让 ρ_d 在方程两侧约去（本文问题二、三），而不是把 ρ 硬写进水分方程。

(3) 两条收缩路径给出同一量级、不同数值。由经验式反演的收缩（29.45 h）与附件 2 实测收缩（23.85 h）相差 23%——两者总量接近，但收缩的时间路径不同。这解释了为什么问题四必须以实测 $R(t)$ 为准（见下节），也界定了判据的适用边界： r 给出的是收缩的总量律，而不是时间律。

11.5 与附件 2 实测收缩的对照

上述判据以经验式隐含的收缩为基础，把它与附件 2 的实测半径对照，可以检验这条判据是否可信。图 8 把两者画在同一坐标系中，结果分两层。

总量上互证。在径向等长口径下，把附录 4 ($r = 0.118$) 的密度经验式外推到绝干，得半径 1.211 cm，与附件 2 的实测终半径 1.198 cm 只差 1.1%；换成体积收缩比是 63.3% 对 64.1%（相对差 1.2%）。附录 3 的 1.301 cm 则偏 +8.6%。需要说明的是，1.1% 是“外推到绝干”与“实测已进入平台”的比较；若改按同一含水率比较——取问题四结束时刻的体积加权平均含水率 $\bar{C} = 0.117$ ——经验式给出 1.271 cm，与实测 1.200 cm 相差 5.6%（图 8b）。两个数字含义不同：前者说明两组数据隐含的收缩总量一致，后者说明二者的收缩路径不同。据此可以说，题面为问题四同时给出的两组数据（附件 2 的 $R(t)$ 与附录 4 的 $\rho(C)$ ）在“干燥全程能缩到多小”这一点上互相印证，问题四把二者搭配使用是有依据的。

路径上互补。把问题四求得的场按体积加权平均含水率折算成比体积，与由 $\rho(C)$ 反推的比体积在同一含水率下比较，可见附件 2 的实测收缩在时间上高度前置：50% 的收缩在 2.5 h 内完成、99% 在 22 h 内完成，此后半径几乎不变；而 ρ 律要求收缩与含水率同步下降。两者在同一含水率下的比体积相差 10.6% ~ 22.4%（图 8）。这不是矛盾： $\rho(C)$ 是平衡态的比体积关系，附件 2 记录的是实际干燥历程，二者的差别恰是”收缩不是平均含水率的单值函数”的体现。也正是这个差别，使得问题四必须采用实测 $R(t)$ 而不是自行反演一条收缩律。

11.6 回看四个问题：分层规则

综合两级判据与代价量化，本文对收缩的处理可以写成一条分层规则，每一层都有定量依据：

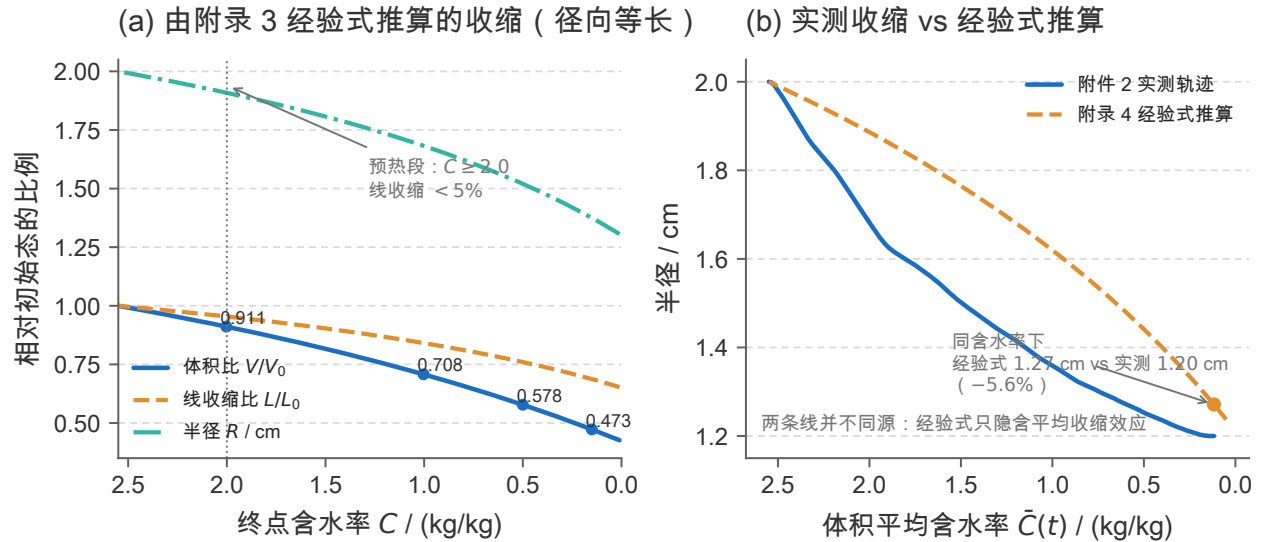


图8 (a) 由附录3经验式推算的几何收缩：体积比 V/V_0 、线收缩比 L/L_0 与对应半径随终点含水率的变化； $C \geq 2.0$ 时线收缩不足5%（径向等长口径）。(b) 由 $\rho(C)$ 反推的 $R(C)$ 与附件2实测 $R-C$ 轨迹的对照（横轴为体积加权平均含水率 $\bar{C}$ ）：同一含水率下经验式比实测偏大约5.6%，外推到绝干则只差1.1%；路径上前者与含水率同步、后者高度前置。

1. 问题一（预热段， C 由 2.55 降到约 2.0）：该问按题面给定的常物性与固定几何求解，附录2只给出一个常数密度，并未给出可用于本判据的 $\rho = a + bC$ 经验式，因此不把 $r = b/a$ 判据外推到该问。对附录3、4而言，同一窗口在5%径向容差下分别为“通过（4.57%）”与“轻微越界（5.68%）”，可见“预热段收缩可忽略”应理解为题面分工下的近似，而不是由两套变物性经验式共同严格保证的结论。
2. 问题二（3h窗口， C 平均降到 1.364）：该窗口在5%容差下阈值 $r^* = 0.507$ ，而两套物性的 r 分别为 0.197 与 0.118，已在阈值之下，说明这一窗口内收缩不再是可以随手忽略的小量。本文按题面分工仍取固定半径（假设B3），但把它显式声明为一条简化，并以第7.6.3节的灵敏度与本节表23量化其代价。
3. 问题三（全程， $C \rightarrow 0.15$ ）：全程窗口的5%阈值升到 $r^* = 0.838$ ，两套物性都远在阈值之下；时长口径下 Δ 达95%。这是三者中唯一“必须考虑”的量级，也正是题目把尺寸变化单列为问题四的原因。
4. 问题四（全程，几何由题面给出）：题面在这一问同时提供了附件2的实测 $R(t)$ 与附录4的 $\rho(C)$ ，且明确要求计入尺寸变化。按本节判据， $r_4 = 0.118$ 远低于阈值，故必须建模；而实测收缩比经验式隐含的收缩更强，因此本文采用实测 $R(t)$ 并引入贴体坐标。

这样一来，“问题一至三取固定几何、问题四计入收缩”就不再是两套互不相干的假设，而是同一条判据在四个不同窗口上的取值结果。同时也要如实指出：按判据，问题二、三的窗口已经越界，本文在这些问题上取固定半径是题面分工下的选择，其代价已在表14中给出，而不是“因为误差很小所以可以忽略”。

11.7 小结

本节的可迁移结论是：收缩几何只由 $r = b/a$ 决定，据此可在求解之前先验判断某问能否忽略收缩（几何判据），并可进一步给出时长口径的阈值；在本问的物性体系下，收缩对烘干时长的影响（-49% ~ -58%）比水分方程口径（6.3% ~ 15.6%）大3倍以上，也大于

D 的 $\pm 20\%$ 不确定度。由 r 反推的绝干半径与附件 2 实测终半径相差 1.1%，说明判据与实测收缩在总量上互证、在路径上互补（相差 10.6% ~ 22.4%），这正是问题四取实测 $R(t)$ 的理由。

12 结果汇总与交付说明

四个结果文件均按附件 3 模板的格式保存，所有数值保留四位小数：

表 15 结果文件一览

文件	工作表	时间范围与间隔	规模
result1.xlsx	温度、水分浓度	1 ~ 1800 s, 每 1 s	1800 × 21
result2.xlsx	温度、水分浓度	1 ~ 206700 s, 每 1 s	206700 × 21
result3.xlsx	水分浓度	60 ~ 206700 s, 每 60 s	3445 × 21
result4.xlsx	水分浓度	60 ~ 183840 s, 每 60 s	3064 × 21

各表的 A 列均为时间（单位：s），第 1 行为到药材中心的距离（单位：cm），径向间隔 0.1 cm。result4.xlsx 在二十一个固定位置之后另加“药材表面”一列；因药材收缩，固定位置中超出当前半径者留空。

13 模型的评价与推广

13.1 模型优点

1. 从控制方程出发刻画传热传质机理，而非经验拟合，参数全部来自题面，物理意义明确；
2. 明确识别出问题一中热质解耦这一结构性质，使该问的计算量降为问题二的三分之一；
3. 采用有限体积离散，格式本身严格守恒，水分守恒残差达 6.8×10^{-14} ；
4. 用贴体坐标处理收缩引起的动边界，把变区域问题化为固定区域问题，无需网格重划；
5. 对题面未给出的闭合关系逐一作了定量辨析，并用湿球温度下限否决了含汽化潜热的方案，避免了物理上不可能的结果。
6. 给出了完整的数据预处理论证：从噪声诊断出发，对“是否平滑、是否换用高阶插值、能否用差分估计变化率”三种选择分别作了量化对照，并据此声明温度结果的有效位，使“四位小数”不被误读为四位精度；
7. 完成了参数灵敏度分析与基于附件 2 实测半径的守恒性检验，前者证明结论对题给 h 、 h_m 不敏感，后者如实报告了题面两组给定数据之间的不自洽，并给出了完整的误差来源量级表（表 16、17）。

13.2 模型缺点

1. 一维径向假设：忽略轴向输运的依据是长径比 6.25、两端面仅占侧面积 8%。若药材长径比较小或端面干燥不可忽略，需扩展为二维轴对称乃至三维模型，端面附近的含水率会低于本文预测值；

2. 均匀比例收缩假设：问题四假定药材各点按同一比例收缩，而实际干燥中表层先收缩、内部滞后，附件 2 只给出外半径，无法检验内部收缩分布。该假设会高估表层与内部的同步性；
3. 假设药材均质、各向同性，未考虑真实的孔隙结构与各向异性；
4. 将水分输运简化为单一 Fick 扩散，未区分自由水与结合水的差异，在含水率很低时可能与实际有偏差；
5. 忽略汽化潜热（假设 A7）：虽然在题给参数体系下计入该项会把表面温度压至湿球温度以下、自相矛盾，但蒸发吸热在真实过程中确实存在。这是题面参数的内部张力，不是可以忽略的物理效应；
6. 干物质质量不守恒：由附录 3、4 的 $\rho(C)$ 经验式与附件 2 的实测 $R(t)$ 共同隐含的干料质量在问题三中增长 116%、在问题四中减少 10.9%（表 10）。这是题面两组给定数据之间的固有张力，但确实限制了结果作为绝对预测值的解读；
7. 水分方程口径不唯一：本文采用 $\partial C / \partial t = \nabla \cdot (D \nabla C)$ ，若把体积密度 ρ 写回水分方程或改用干基密度守恒口径，烘干时长分别变化 +6.3% 与 -15.6%（表 11）；
8. 附件 1 只覆盖 4 h，恒温段取常数外推；虽然末 1 h 的数据显示工况已进入恒温恒湿状态（斜率 +0.003 °C/h），但仍无法排除更长时间尺度上可能存在的漂移；
9. 附件 1 的测量噪声给边界条件带来约 0.02 °C 的不可消除不确定度，使温度结果只有 3 位有效数字（第 5 节）；
10. 传质边界条件的密度口径题面未明确，不同口径下表面含水率相差约 40%，该不确定性无法由题面数据消除。

13.3 误差来源与不确定性量化

上节的局限中，有的可以量化、有的只能定性说明。为便于复核与后续改进，本节把全部误差来源按“数值离散 / 数据 / 参数 / 模型口径”四类汇总于表 16、17，并给出每一项的量级及对结论的影响。

表 16、17 给出三点判断。

第一，在本文采用的数值配置下，数值误差不是主要矛盾。 $N = 200$ 、 $\Delta t = 5$ s 时，空间离散误差 0.07%、时间离散误差不足 0.01%、迭代收敛误差更小，与算术 / 对数平均之差 (0.18%) 同量级，都远小于参数与口径带来的不确定度。需要单独说明的是格式无关性：界面取法在 $N = 200$ 上仍有 2.7% 的残差（算术 vs 调和平均，第 A.3 节），高于题面输出网格 $N = 20$ 的 1.6%，加密到 $N = 400$ 后降至 0.4%；它与网格误差一样属于离散层面的不确定性，同样远小于 D 与口径的影响。

第二，真正决定结论可信度的是参数 D 与水分方程口径。 D 的 $\pm 20\%$ 扰动即可让烘干时长在 48.9 ~ 70.0 h 之间变化，而方程口径的选择带来 +6.3% ~ -15.6% 的偏移。两者都是题面给定的经验式与其在方程中如何起作用所决定的，不能靠加密网格解决。因此本文在正文中明确声明了所采用的口径（第 7.6、5.5 节），以便复核。

第三，一维径向与均匀收缩两项简化无法用现有数据量化，属于最主要的未量化不确定性。若要进一步收窄，必须补充内部温度 / 含水率测点或不同长径比试样的对照实验。

需要说明的是，上述各项误差之间不能简单线性叠加： D 以 $e^{-0.45/C} e^{-3850/T}$ 的形式强非线性地进入方程，且干燥后期的含水率下降速率对 D 极为敏感，误差传播本身是非线性的。表 16、17 给出的是单因素局部扰动结果，其线性外推只在 $\pm 20\%$ 的扰动范围内近似成立。

表 16 误差来源与量级汇总 (一): 数值与数据误差 (“影响” 指对四个问题最终结论的影响)

类别与来源	量级	对结论的影响
数值: 空间离散	$N = 20 \rightarrow 400$ 烘干时长单调收敛, $N = 200$ 与 $N = 400$ 相差 0.07%; 输出网格 $N = 20$ 偏低 1.6%	本文配置 $N = 200$ 的误差 0.07%; 若用题面输出网格 $N = 20$ 则为 1.6% (表 18)
数值: 时间离散	$\Delta t = 120\text{ s}$ 与 2 s 的烘干时长相差 406 s (60 倍步长跨度)	$\leq 0.20\%$, 可忽略 (表 19)
数值: 非线性迭代	Picard 判据 10^{-11} , $2 \sim 4$ 次即收敛	远小于离散误差
数值: 界面系数取法	算术 / 对数 / 调和平均 $\Rightarrow 57.42/57.52/58.96\text{ h}$ ($N = 200$, $\Delta t = 5\text{ s}$), 调和平均在 $N = 100$ 上为 71.57 h、加密到 $N = 400$ 仍偏高 0.41%	本文取算术平均, 与对数基准差 0.18%; $N = 200$ 上改用调和平均仍在 $2 \sim 3$ 天内 (第 A.3 节)
数据: 点间插值方式	线性 / PCHIP / 三次样条: 温度场差 $0.004 \sim 0.005\text{ }^\circ\text{C}$	温度表第 4 位变动; 含水率与烘干时长不变
数据: 边界平滑方式	3 / 5 / 15 点平滑: 温度场差 $0.014 \sim 0.055\text{ }^\circ\text{C}$	同上
数据: 测量噪声	$\sigma = 0.116\text{ }^\circ\text{C}$; 30 个样本的均值涨落 $0.02\text{ }^\circ\text{C}$	温度结果有效位降为 3 位

表 17 误差来源与量级汇总 (二): 参数、模型口径与简化假设

类别与来源	量级	对结论的影响
参数: 扩散系数 D	$\pm 20\%$ 扰动 $\Rightarrow$ 烘干时长 $+22.2\%$ / -14.6% ($48.9 \sim 70.0\text{ h}$)	最大误差源, 跨近一天
参数: h_m	$\pm 20\% \Rightarrow \pm 2 \sim 3\%$	次要
参数: h	$\pm 20\% \Rightarrow \pm 0.03\%$	可忽略 (温度场准稳态)
口径: 水分方程	变密度口径差 $+6.3\%$; 干基密度守恒口径差 -15.6% ; 前者在 $N = 20 \sim 400$ 、 $\Delta t = 2 \sim 120\text{ s}$ 内保持不变	最大模型误差源 (表 18、19)
口径: 传质密度 (问题一)	表面含水率相差约 40%	显著影响水分场, 不影响温度场 (热质解耦)
简化: 忽略汽化潜热	计入后表面温度降至 $6.1\text{ }^\circ\text{C}$, 低于湿球温度 $34.7\text{ }^\circ\text{C}$	题给参数体系下无法自洽引入
检验: 干物质守恒	问题三 $+116\%$ 、问题四 -10.9%	限制结果作为绝对预测值
简化: 一维径向 (端面)	二维特征值分析: 轴向项仅占 λ^2 的 $1.0\% \sim 1.1\%$	-0.62 h , 可忽略 (第 13.4 节)
简化: 均匀收缩	附件 2 只给出外半径, 径向分布不可标定; 问题三固定 R 使用后 62% 时长的扩散路径偏长 67%	未量化 (第 13.4 节), 建议改进方向

13.4 径向与轴向简化的误差量化

题面给定的是长 25 cm、半径 2 cm 的三维圆柱体，问题一至问题四都把它简化为一维径向问题（假设 A2）。这里把该简化的误差拆成三层说明，推导细节见附录 B.3。

(1) 径向分布必须保留。由附录 3 物性与附录 2 交换系数算得 $Bi = hR/k = 1.39$ 、 $Bi_m = h_m R/D(C_0) = 3.24$ ，两者都是 $O(1)$ ，内部梯度与表面阻力都不可忽略。若把药材当作均温均湿的集总体，仅靠表面传质脱水只需约 11.2 h，仅为分布参数模型结果 57.42 h 的 19.5%。换言之，径向梯度不是修正项，而是决定烘干时长的本体，题目要求按 5 个径向位置分别报数正是这个原因。

(2) 干燥后期由干壳控制。本文解中表面在 $t = 12.56$ h 即低于 0.15 kg/kg，此后 78% 的时长都是在“外壳已干、只剩湿芯”的状态下度过的；中心达标时表面仅 0.0527，按附录 3 的 $D(C)$ 其扩散系数只有芯部的 1/256。这解释了为什么烘干时长对 D 在低含水率端格外敏感（ $D \pm 20\%$ 对应 48.9 ~ 70.0 h）。

(3) 端面（轴向）效应可忽略，并可给出上下界。以干燥后期的特征模态估计，端面传质 Biot 数 $Bi_{m,z} = h_m(L/2)/D \approx 125 \gg 1$ ，故端面接近“充分对流”极限，最低非零轴向特征值 $\mu_1 \rightarrow \pi/L = 12.57 \text{ m}^{-1}$ ，轴向衰减率项 μ_1^2 占径向项 λ_1^2/R^2 的 1.0% ~ 1.1%；而若端面完全绝质，最低轴向特征值为 $\mu_0 = 0$ ，长时主模态与一维径向模型完全一致，端面修正严格为零。于是端面效应落在 [0, 1.1%] 之间，上界为 1.1%，折算到烘干时长为 $57.42 \rightarrow 56.80$ h（仅缩短 0.62 h）。原因是 $\bar{D} \sim 10^{-9} \text{ m}^2/\text{s}$ 时 57 h 内的轴向扩散长度仅 1.3 ~ 2.4 cm，远小于半长 12.5 cm，端面来不及影响药材中心。端面效应不超过 1.1%，一维径向假设成立；把模型扩展为二维 (r, z) 不会改变本题答案。

(4) 唯一未量化的径向效应是非均匀收缩。问题四只用单一外半径 $R(t)$ ，相当于把“外层先缩、芯部后缩”的径向分布取平均。附件 2 只记录外半径，由实测终半径可标定唯一的端点体积收缩比：径向等长口径下 $1 - (1.198/2.000)^2 = 64.1\%$ ，与附录 4 密度经验式的 63.3% 相差 1.2%（见第 11.5 节），但无法标定径向分布；该项如实标注为未量化，也是本文建议的首要改进方向。

13.5 模型改进与新模型建议

针对上节列出的误差来源与局限，本文提出七条具体的改进路径，按“可行性 / 收益”由高到低排列。

1. 改用干基参考框架，消除干物质不守恒。本文的检验（表 10）表明，附录 3、4 的 $\rho(C)$ 与附件 2 的 $R(t)$ 共同隐含的干料质量并不守恒。改进办法是把水分方程建立在干物质体积坐标上，即令干料质量守恒自动成立、由收缩本构 $R = R(C)$ 反推体积变化，而不是同时接受 $\rho(C)$ 与实测 $R(t)$ 两组独立信息。代价是需要补充“含水率—体积”关系数据，收益是模型重新满足最基础的守恒律。
2. 把径向非均匀收缩显式建模。本文问题四已改用材料坐标形式（第 9 节），但只用单一外半径 $R(t)$ ，相当于把“外层先缩、芯部后缩”沿半径取平均。下一步可在材料坐标下引入随 ξ 变化的收缩系数 $\varepsilon_r(\xi, t)$ ，用 CT 或核磁测得的内部形变标定其分布，使模型同时给出几何与水分剖面的径向自洽解。
3. 二维轴对称模型（含端面）。本文以端面占比 8% 为由忽略轴向输运。若把模型扩展为 (r, z) 二维轴对称：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(kr \frac{\partial T}{\partial r} \right) + \frac{\partial}{\partial z} \left(k \frac{\partial T}{\partial z} \right),$$

并在 $z = 0, L$ 加上第三类边界，可定量回答“端面到底影响多少”。预期端面附近的含水率低于本文预测、烘干时长略短；该模型对长径比较小或端面干燥不可忽略的药材是必要的。

4. 自洽地引入汽化潜热与表面蒸发冷却。完整的表面能量平衡应写成 $-k\partial_r T|_R = h(T_s - T_\infty) + L_v h_m(\rho_{v,s} - \rho_{v,\infty})$ 。本文已证明：直接把题给的 h_m 与 $C_s - C_\infty$ 代入会把表面温度压到湿球温度以下，说明题面参数体系不含该项。要自洽引入，需补充吸附等温线（把含水率映射为表面水蒸气分压）与水的汽化潜热，这在中药干燥文献中已有成熟测法，是提升物理完整性的关键一步。
5. 变温/变风速工艺优化模型。本文指出 36 ~ 57.4 h 的极慢干燥段占用总时长 37%，却只换来 0.036 kg/kg 的含水率下降。可把烘房温度 $T_\infty(t)$ 与风速（经 h 、 h_m ）作为控制变量，建立

$$\min_{T_\infty(\cdot)} \int_0^{t_f} (c_1 \Phi_{\text{能耗}} + c_2 \Phi_{\text{品质}}) dt \quad \text{s.t.} \quad \max_r C(r, t_f) < 0.15,$$

以本文模型作为状态约束，用序列二次规划或遗传算法求解。预期收益：在总时长不变的前提下显著降低能耗，或在不影响品质的前提下缩短 10% ~ 20% 的干燥时间。

6. 参数反演与不确定度的贝叶斯传播。本文已论证：附件 1 只有边界激励、没有药材侧响应， h 、 h_m 在原理上不可辨识（第 5.5 节）。要做参数反演，必须先补充内部测点——例如按时段取出试样称重、或用低场核磁共振 / CT 测含水率分布。在此基础上以最小二乘或贝叶斯方法反演 D 的经验常数与 h_m ，并用 MCMC 把参数后验传播到烘干时间，直接给出“烘干时长 = 57.4_{-y}^{+x} h”这样的区间结论，比现在的点估计更符合工程决策的需要。
7. 多孔介质两相模型与机器学习代理模型。更彻底的改进是把单一 Fick 扩散替换为 Luikov[7] 或 Philip-De Vries[8] 类两相（液相+气相）耦合模型，引入毛细压力本构与自由水 / 结合水的区分，可刻画结壳与内部水分重分布；代价是参数显著增多、标定难度上升。与之互补的是用本文求解器批量生成样本、训练高斯过程或神经网络代理模型，把单次烘干时长预测从数十秒压缩到毫秒级，从而支撑第 5 条中的在线优化与控制。两者一“深”一“快”，可分别沿机理与工程两个方向推进。

模型向其他工况、几何与物料的推广路径见附录 B.5。

参考文献

- [1] 王补宣. 工程传热传质学 [M]. 北京: 科学出版社, 1998.
- [2] 潘永康, 王喜忠. 现代干燥技术 [M]. 2 版. 北京: 化学工业出版社, 2007.
- [3] 陶文铨. 数值传热学 [M]. 2 版. 西安: 西安交通大学出版社, 2001.
- [4] Patankar S V. Numerical Heat Transfer and Fluid Flow[M]. New York: Hemisphere Publishing, 1980.
- [5] Crank J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [6] Fritsch F N, Carlson R E. Monotone piecewise cubic interpolation[J]. SIAM Journal on Numerical Analysis, 1980, 17(2): 238–246.
- [7] Luikov A V. Heat and Mass Transfer in Capillary-Porous Bodies[M]. Oxford: Pergamon Press, 1966.
- [8] Philip J R, de Vries D A. Moisture movement in porous materials under temperature gradients[J]. Transactions, American Geophysical Union, 1957, 38(2): 222–232.

A 数值检验的完整结果与讨论

本节把与四个问题都有关的数值检验集中说明: 径向网格数与时间步长的收敛性、界面扩散系数的平均方式。这些结论不改变四个问题的求解流程, 但决定了结果的有效位数与可复核性。水分方程口径的敏感性检验已在第 5.6 节给出, 一维径向 (含端面) 简化的误差量化则并入第 13.4 节统一讨论。

A.1 径向网格数 N 对烘干时长的影响

A.1.1 输出网格不等于求解网格

题目要求把水分浓度场按“到药材中心距离 0.1 cm” (含中心共 21 个位置) 保存到 result3.xlsx, 并在题面表 5 中给出 0.5 cm 间隔的五个位置。这规定的是报告网格, 而不是求解网格。若把二者混为一谈、直接用 $N = 20$ ($h = R/N = 1$ mm) 个控制体求解, 中心控制体的积分权重只有 $h^2/8$ 、表面控制体的厚度只有 $h/2$, 而干燥后期的水分剖面在表面附近极其陡峭 (中心达标时表面仅 0.0527、中心为 0.1500), 一阶的空间离散误差会直接传到“中心何时达标”这唯一判据上。

为定量说明这一点, 本节在 $N = 20, 50, 100, 200, 400$ 五档网格上重解问题三 (固定 $\Delta t = 5$ s), 并对两种水分方程口径各算一遍。除水分方程本身外, 两种口径的离散格式、物性经验式、边界条件与时间推进完全相同, 程序上直接复用同一个求解器 (见附录 B), 因此表中的差值只可能来自被测的那一个因素。

- 本文口径 (经典 Fick 形式) $\partial C/\partial t = \nabla \cdot (D \nabla C)$: 不含 ρ 。未知量 C 为干基含水率, D 是针对 C 拟合的有效扩散系数;
- 变密度口径 $\partial(\rho C)/\partial t = \nabla \cdot (\rho D \nabla C)$: 附录 3 的 $\rho = 650 + 128C$ 参与输运, 累计项系数取 $\rho + C d\rho/dC$, 界面通量系数取 ρD , 其等效扩散系数为 $\rho D/(\rho + C d\rho/dC)$ 。

两种写法在问题一完全等价 (那里 $\rho \equiv 820$ kg/m³ 为常数), 差别只在问题二之后 ρ 随 C 变化时才显现; 其物理含义与选取理由见第 5.6 节与表 11。

表 18 径向网格数 N 对问题三烘干时长的影响 ($\Delta t = 5\text{ s}$, $h = R/N$ 为控制体尺寸)

N	h/mm	本文 (经典 Fick)		变密度口径		两者相对差
		烘干时长/s	/h	烘干时长/s	/h	
20	1.000	203335	56.482	216215	60.060	+6.34%
50	0.400	205625	57.118	218575	60.715	+6.30%
100	0.200	206365	57.324	219340	60.928	+6.29%
200	0.100	206700	57.417	219685	61.024	+6.28%
400	0.050	206840	57.456	219830	61.064	+6.28%
相邻网格增量/s		+2290, +740, +335, +140 (逐次减半, 一阶收敛)				

A.1.2 结果与判读

表 18 给出四点结果。

(1) 网格误差单调收敛, 方向是“偏短”。加密网格使烘干时长单调增加: $N = 20 \rightarrow 400$ 依次为 203335, 205625, 206365, 206700, 206840 s, 相邻增量 2290, 740, 335, 140 s 逐次减半, 是一阶收敛的典型形态。取 $N = 200$ 与 $N = 400$ 两点作 Richardson 外推, 连续解约为 $207.0 \times 10^3\text{ s}$ (57.5 h), 故 $N = 200$ 的相对偏差只有 0.1% (约 4 分钟); 而题目自带的输出网格 $N = 20$ 偏低 1.6%, 约 1 h。这说明“按 0.1 cm 输出”绝不等于“按 0.1 cm 求解”, 本文在 $N = 200$ 上求解、再插值到 0.1 cm 的报告网格。

(2) 口径差与网格无关。两列之差在五档网格上分别为 12880、12950、12975、12985、12990 s, 相对差稳定在 +6.28% ~ +6.34%, $N = 20$ 与 $N = 400$ 之间只差 0.06 个百分点。说明“水分方程用哪种口径”与“网格取多密”是两个相互独立的误差源, 量级上近似可加。

(3) 结果之间的差异主要来自这两项的组合。若同时采用本文口径 (Fick) 与题目自带的输出网格, 烘干时长为 56.48 h; 用本文口径并加密到 $N = 200$ 为 57.42 h; 改用变密度口径配输出网格为 60.06 h、配 $N = 200$ 为 61.02 h。也就是说, 不同论文给出的 56 ~ 61 h 完全可以由 {口径} $\times$ {网格} 的组合解释, 其中口径贡献 3.6 h (+6.3%)、网格贡献约 1.0 h (1.6%)。

(4) 本文同时报告三个参照值。由于口径差大于网格数值误差, 但小于 D 的经验式不确定度 ($\pm 20\%$ 对应 48.9 ~ 70.0 h), 论文在主结果 57.42 h 之外, 同时给出 61.02 h (同网格、变密度口径) 与 56.48 h (同口径、输出网格) 两个参照值, 以便复核者判断差异究竟来自模型口径、数值离散还是参数取值。

A.2 时间步长 Δt 对烘干时长的影响

时间方向采用后向 Euler 格式, 它是一阶精度且无条件稳定的。因此 Δt 的取值不能由稳定性决定, 只能由精度决定。本节固定 $N = 100$, 令 Δt 在 2 ~ 120 s 之间变化 (跨度 60 倍), 同样分两种水分方程口径, 结果见表 19。

A.2.1 结果与判读

(1) 时间离散误差比空间离散小一个数量级, 且误差方向相反。 Δt 由 5 s 放宽到 120 s (24 倍), 烘干时长只增加 380 s (+0.17%); 而网格由 $N = 200$ 放宽到 $N = 20$ 会减少 3470 s (-1.6%)。两者符号相反, 量级差十倍。

(2) 口径差与时间步无关。在 60 倍的 Δt 跨度内, 两种口径的差值只在 12960 ~ 12980 s 之间变化, 波动 20 s, 相对差始终为 +6.3%。把本节与表 18 合并看, 口径差在 20 组数值参数的组合下没有系统漂移, 说明它是模型层面的差异, 不能通过调整数值参数消除。

表 19 时间步长 Δt 对问题三烘干时长的影响 ($N = 100$)

$\Delta t/s$	本文 (经典 Fick)		变密度口径		两者相对差	相对 $\Delta t = 5\text{ s}$
	烘干时长/s	/h	烘干时长/s	/h		
2	206354	57.321	219328	60.924	+6.29%	-0.005%
5	206365	57.324	219340	60.928	+6.29%	—
10	206380	57.328	219360	60.933	+6.29%	+0.009%
20	206420	57.339	219400	60.944	+6.29%	+0.027%
60	206580	57.383	219540	60.983	+6.27%	+0.091%
120	206760	57.433	219720	61.033	+6.27%	+0.173%

(3) $\Delta t = 5\text{ s}$ 是精度过剩的保守选择。即使取 $\Delta t = 60\text{ s}$ (与 `result3.xlsx` 的输出间隔一致), 烘干时长也只差 215 s (0.10%)。本文保留 $\Delta t = 5\text{ s}$, 是为了让时间离散误差小到可以被忽略, 代价仅是计算时间。

(4) 两项数值误差的上限。若采用题面输出网格 $N = 20$, 空间离散偏差为 1.6% (相对本文 $N = 200$ 结果); 本文实际采用的 $N = 200$ 相对 $N = 400$ 只有 0.07%。时间步在 $2 \sim 120\text{ s}$ 全跨度内最大偏差为 0.2%。因此本文配置 ($N = 200$ 、 $\Delta t = 5\text{ s}$) 的两项数值误差远小于 0.2%, 而最坏组合 ($N = 20$ 、 $\Delta t = 120\text{ s}$) 合计不超过 1.8%; 两者都仍小于口径差 (6.3% ~ 15.6%) 与参数不确定度 ($-15\% \sim +22\%$)。

A.3 界面系数的取法

有限体积格式中界面扩散系数必须由两侧节点值构造。常用取法有算术平均、对数平均与调和平均: 调和平均对应分片常数介质的串联阻力, 对数平均对应面内光滑变系数的精确积分, 算术平均居中。三者的完整推导与适用前提见附录 B.2。

本题的 D 是 C 的连续函数 (附录 2 的 $D = 7 \times 10^{-9}e^{-0.89/C}$ 、附录 3 的 $D = 2.4 \times 10^{-3}e^{-0.45/C}e^{-3850/T}$), C 在径向又是光滑分布, 属于“面内光滑变系数”情形, 故采用算术平均。在同一个求解器上只切换界面取法 (网格、时间步、物性、边界条件与迭代判据完全相同) 的对照见表 20。

表 20 界面扩散系数取法对问题三烘干时长的影响 ($N = 200$ 、 $\Delta t = 5\text{ s}$, 其余完全相同)

界面取法	物理含义	烘干时长/s	/h
算术平均 (本文)	两侧节点值的平均	206700	57.417
对数平均	面内 D 线性变化时的精确值	207080	57.522
调和平均	两侧半格内 D 为常数的串联阻力	212245	58.957

以对数平均 (本问题在面内线性重构下的精确值) 为基准, 算术平均偏 -0.18% ; 调和平均偏 $+2.49\%$, 即比本文结果高 2.68% (约 1.5 h)。界面平均方式的差别随网格加密单调消失: 调和平均在 $N = 100/200/400$ 上为 $71.57/58.96/57.69\text{ h}$, 与同网格算术平均之差由 14.25 h 依次降到 1.54 h 与 0.23 h (相对差 $25\% \rightarrow 2.7\% \rightarrow 0.4\%$), 二者收敛到同一连续极限。 $N = 100$ 上调和平均折算为 2.98 天、已逼近题面上限, 但这是粗网格的离散现象, 不是模型层面的差异: D 全程跨两个数量级, 算术平均在强非线性下略微高估界面通量, 故调和平均给出更长的时长属于预期方向。在本文报告网格 $N = 200$ 上仍有 2.7% 的残差, 已计入第 13.3 节的误差排序, 其量级小于水分方程口径 (6.3%)、远小于 D 的 $\pm 20\%$ 。定量对照与收敛细节见附录 B.2。

需要强调, 这不是“算术平均普遍优于调和平均”的结论: 若物性以分片常数给出 (分层或复合介质), 调和平均才是正确取法, 差别在适用前提而不在精度高低。本文四个问题统

一采用算术平均，它与对数基准的差（0.18%）落在第 13.3 节所述数值误差的量级之内，不影响任何结论。

B 补充推导与中间结果

B.1 物性关联式的自洽性与守恒口径

记 m_d 为干物质质量、 V 为湿物料当前总体积，则由干基含水率 $C = m_w/m_d$ 的定义立即得到恒等式

$$\rho = \frac{m_d(1+C)}{V} = \rho_d(1+C), \quad \rho_d \equiv \frac{m_d}{V}, \quad (42)$$

其中 ρ_d 是以当前总体积 V 为基准的表现干物质密度，而不是与几何无关的材料真密度；只要 V 随干燥过程变化， ρ_d 就随之变化。把题面的线性经验式 $\rho = a + bC$ 代入式 (42) 得 $\rho_d(C) = (a + bC)/(1 + C)$ 、 $d\rho_d/dC = (b - a)/(1 + C)^2$ ，于是

$$b = a \iff \rho_d \equiv \text{const} \iff \frac{m_d}{V} \text{ 全程不变} \iff V \text{ 不变}. \quad (43)$$

因此，“斜率等于截距”并不是干基含水率定义本身的要求，而是额外加入“总体积始终不变”之后才得到的结果。题面两组经验式均满足 $b < a$ ，意味着随 C 降低， $\rho_d = m_d/V$ 增大，即单位干物质所占总体积减小，这与干燥收缩的物理图像一致，而不是经验式之间的矛盾。例如在 $C_0 = 2.55$ 时，附录 3、4 对应的表现干物质密度分别约为 275 和 279 kg/m³； $C \rightarrow 0$ 时分别趋于 650 和 760 kg/m³，题面密度式已隐含明显的体积变化信息。

三相体积分数与混合律旁证。取水的真密度 $\rho_w^0 \approx 10^3$ kg/m³、固相真密度 $\rho_s = 1400$ kg/m³，水、固、气三相体积分数为 $\varepsilon_w = \rho w / \rho_w^0$ 、 $\varepsilon_s = \rho(1 - w) / \rho_s$ 、 $\varepsilon_a = 1 - \varepsilon_w - \varepsilon_s$ ，其中 $w = C/(1 + C)$ 为湿基含水率。在 $C \in [0, 2.55]$ 内逐点计算，两组经验式给出的三相体积分数均保持非负：湿态孔隙率约 9% ~ 10% ($\varepsilon_w \approx 0.70$ 、 $\varepsilon_s \approx 0.20$)，干态约 46% ~ 54%，不存在体积分数越界等物理矛盾。另一个旁证来自 c_p 与 k ：二者以 w 为自变量，若取质量加权混合形式 $c_p = (1 - w)c_{p,d} + wc_{p,w}$ ，代入附录 3 得 $w \rightarrow 1$ 时的极限为 $c_p = 4186$ J/(kg·K)、 $k = 0.59$ W/(m·K)，与水在常温下的物性一致。因此，题面的线性密度关系应理解为实验区间内的有效拟合式，更不宜超出给定含水率区间外推； ρ 与 c_p 、 k 三者的形式并不存在内在冲突。

密度经验式具有物理意义，并不意味着应把 $\rho(C)$ 直接乘到水分扩散方程的时间项中。设 $\rho_d(\mathbf{x}, t)$ 为局部表现干物质密度、 $\mathbf{u}$ 为固相骨架速度，则单位体积水分质量为 $\rho_d C$ ；由水分守恒 $\partial_t(\rho_d C) + \nabla \cdot (\rho_d C \mathbf{u}) = \nabla \cdot (\rho_d D \nabla C)$ 与干物质守恒 $\partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{u}) = 0$ 消去骨架项，可得

$$\rho_d \left(\frac{\partial C}{\partial t} + \mathbf{u} \cdot \nabla C \right) = \nabla \cdot (\rho_d D \nabla C). \quad (44)$$

式 (44) 表明， ρ_d 能否约去取决于采用的几何与守恒闭合口径，而不能仅凭题面给出的 $\rho(C)$ 决定。问题一至三（固定几何）采用经典 Fick 口径：把密度效应整体计入按干基含水率拟合的有效扩散系数 D ，不再显式写出 ρ_d ，得到

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right). \quad (45)$$

需要强调的是，此时 $\rho_d = \rho/(1 + C)$ 随 C 变化、在空间上并不均匀，“两侧约去”是这一闭合口径的定义，而不是由 ρ_d 为常数推出的；该闭合的代价已由正文第 7.6.4 节的干物质守恒检验量化（问题三 +116%），并给出干基密度守恒口径的对照（缩短 15.6%）。问题四（动边界）则不同：由假设 (C1) 各点按同一比例收缩， ρ_d 只随时间变化、在空间上均匀，因此式

(44) 中的 $\rho_d(t)$ 可以严格地从散度中提出并约去, 无需任何附加近似; 这正是正式式 (35) 的依据。无论哪一问, 都不能在保持固定半径的同时, 又把题面的体积密度 $\rho = a + bC$ 直接写入水分累积项, 否则会混合两套闭合假设并导致干物质守恒失配; 作为敏感性检验, 该写法使烘干时长系统性延长约 6.3% (表 11), 两者都在正文第 5.6 节一并报告。

能量方程一侧还有一项题面未给出、但对结果影响极大的闭合关系: 蒸发吸热。完整的表面能量平衡应写成 $-k \partial_r T|_{r=R} = h(T_s - T_\infty) + L_v j_w$, 其中 j_w 为表面水分通量。按本文采用的密度口径取 $L_v = 2.26 \times 10^6 \text{ J/kg}$ 试算, 表面温度会被压到约 6.1°C ; 而由附件 1 的空气状态可算出 1800 s 时的湿球温度为 34.7°C (初始时刻为 25.5°C)。表面处于湿润状态时, 蒸发冷却最多把表面温度拉到湿球温度, 不可能低于它。解得 6.1°C 说明按该密度口径算出的水分通量比物理允许值大一个量级, 两项不能同时采用。据此判定题面所给参数体系对应忽略汽化潜热的简化模型, 本文取式 (1) 的形式 (假设 A7); 定量收支见图 1。

为什么 h_m 不能按气侧系数理解。上述否决还可以从量级上独立得到支持。题给 $h_m = 8 \times 10^{-7} \text{ m/s}$ 远小于气侧传质系数的量级: 按 Lewis 类比, $h/(\rho_a c_{p,a}) \approx 25/(1.1 \times 1005) \approx 2 \times 10^{-2} \text{ m/s}$, 比题给值大四个半数量级。另一方面, 水分方程的第三类边界条件 $-D \partial_r C = h_m(C_s - C_\infty)$ 两侧同为 m/s , 说明 h_m 是按干基含水率定义、已把密度换算与液侧阻力一并吸收的等效传质速度, 而不是空气中的水蒸气膜系数。把这样一个等效系数直接当作气侧系数、再乘以汽化潜热去闭合表面能量方程, 从量纲与机理上本就不成立; 表面温度被压到 6.1°C 只是这一结论的定量表现。因此本文按假设 (A7) 取忽略汽化潜热的简化, 并把该项列为正文第 13.5 节的首要改进方向。

B.2 界面扩散系数平均方式的推导与对照

有限体积格式把方程写成逐个控制体的通量平衡, 相邻节点之间的扩散通量必然写成

$$J_{i+1/2} = (\rho D)_{i+1/2} \frac{C_{i+1} - C_i}{\Delta r}, \quad (46)$$

而 ρD 只在节点上有值, 界面值 $(\rho D)_{i+1/2}$ 必须由两侧节点构造。传热传质计算中流传最广、也最常被当作“唯一正确”的取法是调和平均:

$$D_{i+1/2}^H = \frac{2D_i D_{i+1}}{D_i + D_{i+1}}. \quad (47)$$

与之并列的还有算术平均与对数平均:

$$D_{i+1/2}^A = \frac{D_i + D_{i+1}}{2}, \quad D_{i+1/2}^L = \frac{D_{i+1} - D_i}{\ln(D_{i+1}/D_i)}. \quad (48)$$

三者各有各的前提, 用错前提才会出错:

- 调和平均对应分片常数的非均质介质。设界面两侧各有半个控制体、每半格内 D 为常数, 稳态下两层串联, 总阻力等于两层阻力之和

$$\frac{\Delta r}{D_{i+1/2}^H} = \frac{\Delta r/2}{D_i} + \frac{\Delta r/2}{D_{i+1}}, \quad (49)$$

解出即式 (47)。复合材料、分层材料的界面属于此类;

- 对数平均对应光滑变系数介质。稳态通量 J 与面内阻力积分满足

$$\Delta C = -J \int_{r_i}^{r_{i+1}} \frac{dr}{D(r)}, \quad (50)$$

若 D 在面内随 r 线性变化, 积分即得式 (48) 的末式。

对任意 $D_i \neq D_{i+1} > 0$ 有

$$D^H < D^L < D^A, \quad (51)$$

且三者的差距由比值 D_{i+1}/D_i 决定：比值越偏离 1，被小值主导的调和平均低估界面通量越多，而对数平均恰好落在两者之间。本文的 D 是 C 的连续函数、 C 在径向光滑分布，故采用算术平均。两条定量证据如下。

(1) 水分变化被充分分辨时，三种取法没有差别。问题一中 D 只依赖 C 且物性为常数，取细网格 $N = 400$ 时算术平均与调和平均给出的 1800 s 表面含水率分别为 1.510327 与 1.510326，相对差 6×10^{-5} ；即使在题目要求的输出网格 $N = 20$ 上，两者分别为 1.51211 与 1.51174，相对差 0.025%，全场最大差 $3.8 \times 10^{-4} \text{ kg/kg}$ 。可见面平均规则本身不构成误差来源，真正的差别只在问题二之后 D 跨越两个数量级时才显现。

(2) 界面平均方式的差别是离散层面的，且随加密单调消失。把两种取法放在同一组网格上比较：调和平均在 $N = 100$ 时要 71.57 h（2.98 天，已逼近题面上限）， $N = 200$ 降到 58.96 h， $N = 400$ 为 57.69 h；与同网格算术平均（57.32/57.42/57.46 h）之差依次为 14.25、1.54、0.23 h，相对差 $25\% \rightarrow 2.7\% \rightarrow 0.4\%$ ，随网格加密单调收敛到零。这说明两者的差别属于离散误差而非模型差异： D 全程跨两个数量级，式 (51) 中“被小值主导”的效应在粗网格上被放大，加密后两种取法趋于同一连续极限，收敛细节见第 A.1 节。在本文报告网格 $N = 200$ 上改用调和平均，烘干时长仍落在 2 ~ 3 天内，不影响任何定性结论；其 2.7% 的残差已计入第 13.3 节的误差排序。需要强调：若物性以分片常数给出（分层或复合介质），调和平均才是正确取法，差别在适用前提而不在精度高低。

B.3 径向效应量化细节

题面给定的是长 25 cm、半径 2 cm 的三维圆柱体，而问题一至问题四都把它简化为一维径向问题。本附录给出第 13.4 节三层结论的定量依据。

必须用径向分布参数模型。由附录 3 的物性与附录 2 的交换系数可算得

$$Bi = \frac{hR}{k} = 1.39, \quad Bi_m = \frac{h_m R}{D(C_0)} = \frac{8 \times 10^{-7} \times 0.02}{4.938 \times 10^{-9}} = 3.24,$$

两者都是 $O(1) \sim O(10)$ 的量级。若把药材当作均温均湿的集总体，仅靠表面传质脱水，比表面积为 $A/V = 2/R$ ，特征时间为

$$\tau = \frac{V}{Ah_m} = \frac{R}{2h_m} = 3.47 \text{ h},$$

从 $C_0 = 2.55$ 降到 $C_f = 0.15$ （空气侧 $C_\infty = 0.05$ ）只需

$$t = \tau \ln \frac{C_0 - C_\infty}{C_f - C_\infty} = 3.47 \ln 25 = 11.2 \text{ h},$$

仅为分布参数模型结果 57.42 h 的 19.5%。

干壳与移动干燥界面。表面在 $t = 12.56 \text{ h}$ 首次低于 0.15 kg/kg ，仅占全程的 21.9%；其余 78.1% 都是在“外壳已干、只剩湿芯”的状态下度过的。径向分化最剧烈的时刻出现在 24 h：中心 0.2378 对表面 0.0665，相差 3.57 倍；到中心达标时，表面为 0.0527、中心为 0.1500。按附录 3 的 $D = 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T}$ ，在 $T = 50^\circ\text{C}$ 下 $D(0.1500) = 8.00 \times 10^{-10} \text{ m}^2/\text{s}$ ，而 $D(0.0527) = 3.15 \times 10^{-12} \text{ m}^2/\text{s}$ ，干壳的扩散系数只有芯部的 $1/256$ 。干燥后 2/3 段的主导阻力不再是内部扩散本身，而是水分必须穿过一层几乎不透水的干壳；这也是烘干时长对 D 在低含水率端格外敏感的原因。

收缩的径向非均匀性。附件 2 给出的外半径实测值为：0 h 时 2.0000 cm，4 h 时 1.4770 cm，12 h 时 1.2480 cm，22 h 时 1.2060 cm，此后基本停在 1.1980 cm。即半径在 22 h 内收缩 40.1%、

体积收缩 64.1%，而烘干还要再持续 35 h。问题三固定 $R = 2.0 \text{ cm}$ ，等价于在 $t > 22 \text{ h}$ 的 35 h（占全程 62%）里，让模型采用的扩散路径比真实药材长 $2.000/1.198 - 1 = 67\%$ 。更重要的是，附件 2 的收缩时标与干壳形成相互印证：表面在 12.56 h 达标，而收缩在 22 h 后即基本停止——外层干燥并刚性化之后外半径不再变化。这说明收缩不是时间的独立函数，而是局部含水率的函数。在径向等长口径下，由 $\rho = \rho_d(1 + C)$ 与 $\rho = a + bC$ 可得相对体积的精确关系 (40)，且 $R/R_0 = [V(C)/V(C_0)]^{1/2}$ 。把问题四结束时刻的体积加权平均含水率 $\bar{C} = 0.117$ 代入附录 4 ($r_4 = 0.118$)，得 $R = 1.271 \text{ cm}$ ，与附件 2 同一含水率的实测 1.200 cm 相差 5.6%（图 8b）；若比较绝干端点，经验式给 1.211 cm 、实测平台 1.198 cm ，只差 1.1%。两个数字分别说明“收缩总量一致、收缩路径不同”。因此本文问题四不自行反演收缩律，而直接采用实测 $R(t)$ ；把径向非均匀收缩取平均、只用单一外半径，是附件 2 只记录外半径这一信息限制下的必然选择，故本文如实把该项标注为 未量化。

端面（轴向）效应。圆柱长径比 $L/(2R) = 6.25$ ，两端面积仅占侧面的 8%，这是采用一维径向近似的直观依据。为把它量化，在干燥后期（由第一特征模态主导）比较二维 (r, z) 与一维径向的衰减率。二维问题的最小衰减率为

$$\lambda^2 = \frac{\lambda_1^2}{R^2} + \mu^2,$$

其中 $\lambda_1 = 2.4048$ 是 J_0 的第一个零点， μ 为轴向特征值。若端面完全绝质（Neumann），轴向特征值为 $\mu_n = n\pi/L$ ，最低的 $n = 0$ 模态沿轴向均匀，其衰减率与一维径向完全相同，即绝质端面的修正严格为零；若端面按第三类边界与空气交换， μ 由 $\mu \tan(\mu L/2) = h_m/D$ 确定，本题 $Bi_{m,z} = h_m(L/2)/D \approx 125 \gg 1$ ，故 $\mu \rightarrow \pi/L$ ，这是端面修正的上界。代入 $L = 0.25 \text{ m}$ 、 $R = 0.02 \text{ m}$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 、 $D \sim 10^{-9} \text{ m}^2/\text{s}$ 得

端面边界	μ/m^{-1}	μ^2/m^{-2}	占总衰减率 λ^2 的比例
绝质端面 ($\mu_0 = 0$)	0	0	0
第三类对流边界 ($Bi_{m,z} \approx 125$)	12.23	149.6	1.02%
充分对流极限 ($Bi_{m,z} \rightarrow \infty$, 上界)	12.57	157.9	1.08%
径向项 λ_1^2/R^2	—	14458	—

因此端面修正的最小值为 0（绝质）、最大值为 1.08%（充分对流极限），本题实际的第三类边界取中间的 1.02%，三种情形都不超过 1.1%。长时解按 $e^{-\lambda^2 \bar{D}t}$ 衰减，故这相当于把烘干时长乘以 $14458/14616$ ：

$$57.42 \text{ h} \rightarrow 56.80 \text{ h},$$

只缩短 0.62 h (1.1%)，小于网格误差与口径差。物理图像同样清楚： $\bar{D} \sim 10^{-9} \text{ m}^2/\text{s}$ 时，61 h 内的轴向扩散长度 $\sqrt{\bar{D}t} \approx 1.3 \sim 2.4 \text{ cm}$ ，远小于半长 $L/2 = 12.5 \text{ cm}$ ，端面来不及影响药材中心。端面效应不足 1.1%，一维径向假设成立；把模型扩展为二维 (r, z) 不会改变本题的答案。

B.4 收缩判据的闭式推导与参数扫描

判据一：忽略收缩的闭式阈值。忽略收缩，就是用固定半径 R_0 近似真实的 $R(C)$ ，误差用相对线收缩 $s(C; r) = 1 - R(C)/R_0$ 度量。给定可接受的半径容差 ε 与工作窗口 $[C_{\min}, C_0]$ ，忽略收缩当且仅当窗口内最大收缩不超过容差，即 $s(C_{\min}; r) \leq \varepsilon$ 。由于 s 关于 r 单调递减，该条件等价于

$$r \geq r^*(\varepsilon, C_{\min}) = \frac{g - 1}{C_0 - g C_{\min}}, \quad g = (1 - \varepsilon)^n \frac{1 + C_0}{1 + C_{\min}}, \quad (52)$$

表 21 与径向有关的效应及其对烘干时长的影响

效应	量化指标	数值	对烘干时长的影响
径向分布参数模型	Bi 、 Bi_m	1.39、3.24	集总模型 11.2 h, 仅分布模型的 19.5%
表面与中心分化	C 之比	24 h 时 3.57 倍	判据必须由中心控制
干壳 (移动界面)	表面达标时刻、 D 之比	12.56 h (22%)、 $1/256$	后 78% 时长由干壳控制
均匀收缩 (问题四)	附件 2 外半径	$2.000 \rightarrow 1.198$ cm, 体积 -64%	同口径下 51.08 h 对 129.81 h (固定半径)
非均匀径向收缩	$\varepsilon_r = \beta [C_0 - C(r, t)]$	只有外半径, 无法标定	未量化 (建议的改进方向)
端面 (轴向) 输运	轴向项占 λ^2 之比	$1.0\% \sim 1.1\%$	-0.62 h, 可忽略

其中 $n = 2$ (径向等长) 或 $n = 3$ (各向同性)。反解含水率即得临界含水率

$$C^*(\varepsilon, r) = \frac{G - 1}{1 - Gr}, \quad G = (1 - \varepsilon)^n \frac{1 + C_0}{1 + rC_0}, \quad (53)$$

含义是: 含水率降到 C^* 以下之后, 忽略收缩的半径误差开始超过容差。两式的数值见表 22, 图 8 给出其几何图像。

表 22 忽略收缩的阈值 r^* (式 (52)) 与临界含水率 C^* (式 (53))。 r^* 按“径向等长 / 各向同性”两种几何约定并列; $r \geq r^*$ 时窗口内忽略收缩安全。题面两式为 $r_3 = 0.197$ 、 $r_4 = 0.118$ 。

工作窗口	$\varepsilon = 3\%$	$\varepsilon = 5\%$	$\varepsilon = 10\%$
阈值 r^*			
预热段 $C: 2.55 \rightarrow 2.0$	0.351 / 0.205	0.164 / 0.028	-0.066 / -0.167
问题二窗口 $C: 2.55 \rightarrow 1.364$	0.663 / 0.544	0.507 / 0.362	0.243 / 0.090
全程 $C: 2.55 \rightarrow 0.15$	0.901 / 0.854	0.838 / 0.765	0.690 / 0.565
临界含水率 C^* (径向 / 各向同性)			
附录 3 ($r = 0.197$)	2.177 / 2.011	1.953 / 1.708	1.467 / 1.094
附录 4 ($r = 0.118$)	2.249 / 2.110	2.062 / 1.850	1.637 / 1.292

表 22 的读法: 预热段 ($C = 2.0$) 在 5% 径向容差下阈值为 0.164, 附录 3 ($r_3 = 0.197$) 通过、线收缩 4.57%, 附录 4 ($r_4 = 0.118$) 则已越界、线收缩 5.68%; 若改用各向同性口径 ($n = 3$), 同一窗口的线收缩降到 3.07% 与 3.83%, 两者都通过 5% 容差; 改用更严的 3% 容差 (径向阈值 0.351) 则两式都越界, 附录 4 更甚。因此“预热段线收缩不足 3%”只在各向同性口径、且只对附录 3 近似成立, 稳妥的表述是 预热段线收缩约 3% ~ 6%, 取决于几何口径与物性。问题二窗口 (C 由 2.55 降到 1.364) 在 5% 容差下阈值为 0.507, 两套经验式已在阈值之下。全程窗口 ($C \rightarrow 0.15$) 阈值升到 0.838, 两套经验式都远在阈值之下——无论用哪套物性, 深度干燥段的收缩都不可忽略。判据只依赖 ε 与工作窗口, 不依赖扩散系数或换热系数, 因此是先验的、求解之前就能算出的几何判据。

判据二: 从几何容差到烘干时长。为回答“忽略收缩会让烘干时间错多少”, 本文做了一组对照: 固定 $a = 650$ 与附录 3 的 c_p 、 k 、 D , 只改斜率 b , 对每个 r 分别求固定半径与计入收缩两种解。收缩几何由式 (40) 反演, 并用不动点迭代自洽闭合。以

$$\Delta(r) = \frac{t_{\text{固定}}(r)}{t_{\text{收缩}}(r)} - 1 \quad (54)$$

度量代价，结果见表 23。

表 23 斜率截距比 r 对烘干时长的影响(截距固定 $a = 650$ 、 c_p 、 k 、 D 同附录 3, $N = 100$ 、 $\Delta t = 5$ s。“收缩”一列由 $\rho = a + bC$ 反演 $R(C)$ 并迭代自洽闭合)

斜率 b	$r = b/a$	$t_{\text{固定}}/\text{h}$	$t_{\text{收缩}}/\text{h}$	Δ	说明
0	0.000	57.30	21.27	+169%	极限强收缩
64	0.098	57.31	25.45	+125%	附录 3
96	0.148	57.32	27.47	+109%	
128	0.197	57.32	29.45	+95%	
192	0.295	57.34	33.30	+72%	
320	0.492	57.36	40.59	+41%	
420	0.646	57.39	45.97	+25%	
500	0.769	57.40	50.10	+15%	
560	0.862	57.41	53.09	+8%	斜率 = 截距，无收缩
610	0.938	57.42	55.52	+3%	
650	1.000	57.43	57.43	0	
时长口径的阈值： $\Delta \leq 5\% \Rightarrow r \geq 0.913$ ； $\Delta \leq 10\% \Rightarrow r \geq 0.835$ ； $\Delta \leq 20\% \Rightarrow r \geq 0.704$					

表 23 给出三点。其一，几何固定时斜率几乎不影响烘干时长： b 由 0 增到 650， $t_{\text{固定}}$ 只在 57.30 ~ 57.43 h 之间变动，全变程 0.24%；原因是体积密度只以体积热容 ρc_p 的形式进入热方程，其影响局限于最初两三小时的升温段。其二，一旦计入收缩，斜率的影响被放大两个数量级： r 从 1 减到 0， Δ 从 0 单调增到 169%；附录 3 的 $r = 0.197$ 对应 $\Delta = 95\%$ 。其三，把 $\Delta(r)$ 反解即得时长口径的阈值： $\Delta \leq 5\%/10\%/20\%$ 分别要求 $r \geq 0.913/0.835/0.704$ ，明显严于几何判据（半径以平方进入特征扩散时间）。

B.5 模型的适用范围与推广

正文第 13.5 节已给出七条按“可行性 / 收益”排序的改进路径，本节补充模型可迁移的应用边界：

1. 变工况优化：把烘房温度与风速作为控制变量，以能耗与品质为目标建立优化模型，寻找分段变温的干燥曲线，减少问题三中指出的“最后 21 小时只换来很小含水率下降”的低效段；
2. 多维扩展：对长径比较小的药材，可将模型扩展为二维轴对称或三维问题，只需更换相应的散度算子；
3. 模型参数反演：以实测含水率分布为目标，用贝叶斯或最小二乘方法反演 h 、 h_m 与 D 的经验常数，可提高对其他药材品种的适用性；
4. 方法推广：本文“贴体坐标 + 保单调插值 + 有限体积隐式推进”的处理框架同样适用于其他带收缩或膨胀的传热传质过程，如食品干燥、污泥干化、锂电池电极干燥等。

C 程序清单与复现说明

全部数值计算由 Python 3 完成，仅依赖 NumPy 与 openpyxl 两个库，不含任何黑箱求解器；四个问题的三对角方程组均由本文自行实现的追赶法求解。

复现方式（需使用打包的 Python 运行时）：

```

PY = 打包运行时的 bin/python3
$PY 代码/solve_p1.py --save 输出/result1_data.npz
    --tables 输出/问题一-表 1 表 2.md
$PY 代码/run_all.py --n 200 --dt 5
$PY 代码/prepare_outputs.py
$PY 代码/paper_tables.py
$PY 代码/p23_variants.py --table

```

其中最后一条负责复现第 A.1、A.2 节的两张对照表。

支撑材料文件列表

按竞赛规范，支撑材料为一个压缩包，其文件列表如下；其中与建模直接相关的求解、对照、灵敏度与校验程序全文列于本节之后。

表 24 支撑材料文件列表

文件 / 目录	说明
代码/solve_p1.py	问题一求解器（常数物性、热质解耦）
代码/solve_p234.py	问题二、三、四主求解器（变物性、动边界）
代码/p23_variants.py	网格 / 时间步 / 水分方程口径对照求解器
代码/closure_rhod.py	干基密度守恒口径的独立实现与交叉验证
代码/sens.py	参数灵敏度分析
代 码/shrink_criterion.py	收缩判据、阈值表与参数扫描
代码/run_all.py	主求解器批处理驱动
代 码/prepare_outputs.py	由数值解生成 result*.xlsx
代码/paper_tables.py	由数值解生成论文各表
代 码/verify_paper_numbers.py	论文 347 项数字的自动核对
代 码/prepare_figure_data.py、 代码/make_figures.py	插图数据准备与绘制（含 figlib/ 样式库）
代 码/build_result1.mjs、 build_results234.mjs	结果表的 JavaScript 生成脚本
输出/	全部数值结果、图像数据与审计文件 (result1--4.xlsx、*.npz、figdata/ 等)
建模/、思考-斜率截距与收 缩/	建模推导、收缩判据的独立复核脚本与报告
论文/figs/	正文与附录插图的 PDF/PNG 源文件

完整程序清单（一）：主求解器 solve_p234.py

问题二、三、四的耦合传热传质求解器全文如下。它包含四部分：附录 3/4 物性经验式的实现、附件 1/2 的读取与插值、有限体积离散的组装与追赶法求解、以及贴体坐标下的收缩处理。

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """2026 A题 问题二/三/四 求解
4
5  跟问题一最大的区别是物性不再恒定。问题一里 rho、cp、k 都是常数、D 只看含水率，
6  所以温度和水分两个方程是解耦的，先后各推一步就行。问题二开始，附录3、4 给的
7  rho、cp、k 全都随含水率变化，D 还随温度变化，两个方程真正耦合，只能来回迭代。
8
9      热量: rho*cp*dT/dt = (1/r) * d/dr ( k * r * dT/dr )      (体积热容口径)
10     水分: dC/dt = (1/r) * d/dr ( D * r * dC/dr )      (经典 Fick, C 为干基含水率)
11
12     四个问题共用上面两式；问题四的差别只在几何：半径 R(t) 随时间收缩，计算域
13     随之缩小。把时间导数换到随材料运动的贴体坐标 xi = r/R(t) 上时有
14
15         d/dt|_r = d/dt|_xi - (xi*Rdot/R) * d/dxi
16
17     而对流项 u*d/dr (u = xi*Rdot 为骨架速度) 恰好与坐标变换项抵消：
18
19         d/dt|_r + u*d/dr = d/dt|_xi
20
21     因此方程里**不出现**额外的"挤压"对流项，收缩只通过度量因子 1/R^2 与
22     表面边界条件中的 R 进入模型。这一点与问题二、三的经典 Fick 口径完全一致，
23     也避免了把体积密度 rho(C) 直接写进水分累积项（见论文 5.6 节）。
24
25     定解条件跟问题一一样：
26         T(r,0) = 28 C, C(r,0) = 2.55 kg/kg
27         r = 0 : dT/dr = dC/dr = 0      (轴对称，不能写成定值边界)
28         r = R : -k dT/dr = h*(T_s - T_inf)      (对流换热)
29                 -D dC/dr = hm*(C_s - C_inf)      (对流传质)
30
31     烘房条件是分两段的：附件1 只给到 14400 s，这段用线性插值；之后算恒温干燥段，
32     取 50 C 和 0.05 kg/kg。这两个数不是随手定的——附件1 最后 1 小时的平均值就是
33     49.9989 和 0.04999，而且这段的升温斜率只有 0.003 C/h，说明烘房已经稳住了。
34
35     问题四还要处理收缩。半径 R(t) 用附件2 的数据，走 PCHIP 插值（不能用差分，实测
36     相邻差分估出来的斜率信噪比只有 2:1，噪声会被放大）。为了不让网格跟着动，把计算
37     放到贴体坐标 xi = r/R(t) 上做；如上面推导，方程形式不变，只有度量因子与表面
38     边界条件带上 R。
39
40     数值方法还是老一套：节点中心有限体积 + 后向 Euler + Picard 迭代，
41     每步组装成三对角方程组，用追赶法解。
42
43     跑法：
44         PY=/Users/sqz/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3
45         $PY 代码/solve_p234.py --problem 3 --n 200 --dt 5 --save 输出/result3_data.npz
46         注意 --n 不写就用下面 N_FINE 的默认值，论文里的结果是 200 跑的。
47     """
48
49     from __future__ import annotations
50     import argparse
51     from pathlib import Path
52     import numpy as np
53     import openpyxl
54
55     #文件
56     PROJECT = Path(__file__).resolve().parent.parent
57     ATT1 = PROJECT / "附件" / "附件1-烘房温度与水分浓度.xlsx"
58     ATT2 = PROJECT / "附件" / "附件2-药材半径.xlsx"
59
60     #常数
61     R0 = 0.02 #初始半径 2 cm
62     L_LEN = 0.25 #药材长度 25 cm
63     H_HEAT = 25.0 #对流换热系数 (附录 2)
64     H_MASS = 8.0e-7 #对流传质系数 (附录 2)
65     T_INIT, C_INIT = 28.0, 2.55 #初始温度与含水率
66     T_CONST, C_CONST = 50.0, 0.05 #恒温干燥
67     T_PREHEAT_END = 14400.0 #附件 1 时间上限 s
68     C_DRY = 0.15 #烘干判据
69     N_FINE = 200 #默认网格单元数。论文结果就是 200 跑的；换成 100 烘干时间会差 0.16% (219340 vs 219685 s)
70
71     #物性关系
72     def props_p23(conc, temp):
73         c = np.maximum(conc, 1e-12)      # 防除零

```

```

74     t_k = temp + 273.15                # °C → K
75     rho = 650.0 + 128.0 * c
76     drho = np.full_like(c, 128.0)    # d(650+128C)/dC = 128
77     cp = 1450.0 + 2736.0 * c / (c + 1.0)
78     k = 0.21 + 0.38 * c / (c + 1.0)
79     d = 2.4e-3 * np.exp(-0.45 / c) * np.exp(-3850.0 / t_k)
80     return rho, drho, cp, k, d
81 def props_p4(conc, temp):
82     c = np.maximum(conc, 1e-12)
83     t_k = temp + 273.15
84     rho = 760.0 + 90.0 * c
85     drho = np.full_like(c, 90.0)
86     cp = 1850.0 + 2150.0 * c / (c + 1.0)
87     k = 0.12 + 0.20 * c / (c + 1.0)
88     d = 4.2e-4 * np.exp(-0.30 / c) * np.exp(-3850.0 / t_k)
89     return rho, drho, cp, k, d
90 PROPERTY_SETS = {"p23": props_p23, "p4": props_p4}
91
92
93 def _face_mean(a, kind):
94     """由两侧节点值构造界面值。
95
96     "arith": 算术平均 (本文默认);
97     "harm" : 调和平均 (分片常数介质的串联阻力);
98     "log"  : 对数平均 (面内线性变化时的精确值)。
99     """
100     lo = np.maximum(a[:-1], 1e-30)
101     hi = np.maximum(a[1:], 1e-30)
102     if kind == "arith":
103         return 0.5 * (lo + hi)
104     if kind == "harm":
105         return 2.0 * lo * hi / (lo + hi)
106     if kind == "log":
107         ratio = hi / lo
108         close = np.isclose(ratio, 1.0)
109         out = (hi - lo) / np.log(ratio)
110         return np.where(close, lo, out)
111     raise ValueError("unknown face_mean: %r" % kind)
112
113
114 #读取附件数据
115 def _read(path):
116     #读取时跳过表头
117     wb = openpyxl.load_workbook(path, data_only=True)
118     return [
119         r
120         for r in wb.active.iter_rows(min_row=2, values_only=True)
121         if r[0] is not None
122     ]
123 _AIR = _read(ATT1)
124 _AIR_T = np.array([r[0] for r in _AIR], float) #时间
125 _AIR_TEMP = np.array([r[1] for r in _AIR], float) #烘房温度
126 _AIR_C = np.array([r[2] for r in _AIR], float) #烘房水分浓度
127 _RAD = _read(ATT2)
128 _RAD_T = np.array([r[0] for r in _RAD], float) #时间
129 _RAD_V = np.array([r[1] for r in _RAD], float) / 100.0 #半径
130
131
132 def air_conditions(t_query):
133     tq = np.asarray(t_query, float)
134     scalar = tq.ndim == 0 #输入是不是标量
135     tq = np.atleast_1d(tq) #向量化
136     #t≤14400 s时对附件1线性插值; np.where一次处理整个数组; np.interp在t>14400时会做线性外推
137     temp = np.where(
138         tq <= T_PREHEAT_END, np.interp(tq, _AIR_T, _AIR_TEMP), T_CONST
139     )
140     humid = np.where(
141         tq <= T_PREHEAT_END, np.interp(tq, _AIR_T, _AIR_C), C_CONST
142     )
143     return (float(temp[0]), float(humid[0])) if scalar else (temp, humid)
144
145
146

```

```

147 #追赶法 (同 solve_p1.py, 此处不再重复注释)
148 def thomas(lower, diag, upper, rhs):
149     """追赶法解三对角方程组, O(n)."""
150     n = len(diag)
151     cp = np.empty(n)
152     dp = np.empty(n)
153     cp[0] = upper[0] / diag[0]
154     dp[0] = rhs[0] / diag[0]
155     for i in range(1, n):
156         m = diag[i] - lower[i] * cp[i - 1]
157         cp[i] = upper[i] / m
158         dp[i] = (rhs[i] - lower[i] * dp[i - 1]) / m
159     x = np.empty(n)
160     x[-1] = dp[-1]
161     for i in range(n - 2, -1, -1):
162         x[i] = dp[i] - cp[i] * x[i + 1]
163     return x
164
165 #组装三对角系统
166 def _assemble(lam, face, value, alpha_const, far_value, n):
167     #把一个"扩散 + 第三类边界"的隐式方程组装成三对角系统。
168     """被组装的方程 (逐节点): lam_i · (u_i - value_i) (后向 Euler)+face_i 类项 · (u_i - u_{i±1}) (扩散项)
    +alpha · (u_N - far_value) (表面第三类边界)=0
    统一记成: lower[i] · u_{i-1} + diag[i] · u_i + upper[i] · u_{i+1} = rhs[i]。
    """
169
170
171     #参数:
172     #lam : (n+1,)          时间导数项系数(含控制体权重、物性、R²)
173     #face : (n,)           界面扩散系数 (含 dt、D 或 k、ξ_face、1/h)
174     #value : (n+1,)        本时间步开始时刻的场 (右端项只用它)
175     #alpha_const : float   表面第三类边界的换热/传质系数 (含 dt、R、h 或 hm)
176     #far_value : float     远场值(T_inf 或 C_inf)
177     #n : int               控制体个数
178
179
180     #三段的组装依据
181     # · 中心节点 i=0: 控制体是 [0, h/2], 没有左侧界面 (对称性已经体现在 weight[0]=h²/8里), 所以只有右界面的 face[0]。
182     # · 内部节点: 左界面 face[i-1]、右界面 face[i]。
183     # · 表面节点 i=n: β=face[n-1] 是内界面扩散, alpha_const 是第三类边界项。
184     # 贴体坐标下对流项与坐标变换项精确抵消, 因此不出现对流贡献。
185     lower = np.zeros(n + 1)
186     diag = np.zeros(n + 1)
187     upper = np.zeros(n + 1)
188     rhs = np.zeros(n + 1)
189     #中心节点
190     diag[0] = lam[0] + face[0]
191     upper[0] = -face[0]
192     rhs[0] = lam[0] * value[0]
193     #内部节点
194     for i in range(1, n):
195         bm = face[i - 1]
196         bp = face[i]
197         lower[i] = -bm
198         diag[i] = lam[i] + bm + bp
199         upper[i] = -bp
200         rhs[i] = lam[i] * value[i]
201     #表面节点
202     beta = face[n - 1]
203     lower[n] = -beta
204     diag[n] = lam[n] + alpha_const + beta
205     rhs[n] = lam[n] * value[n] + alpha_const * far_value
206     return lower, diag, upper, rhs
207
208 #保单调三次Hermite插值 (问题四)
209 class Pchip:
210     def __init__(self, x, y):
211         self.x = np.asarray(x, float)
212         self.y = np.asarray(y, float)
213         self.h = np.diff(self.x)
214         delta = np.diff(self.y) / self.h
215         n = len(self.x)
216         m = np.zeros(n)
217         m[0] = delta[0]

```

```

218     m[-1] = delta[-1]
219     for i in range(1, n - 1):
220         if delta[i - 1] * delta[i] <= 0.0:
221             m[i] = 0.0
222         else:
223             w1 = 2.0 * self.h[i] + self.h[i - 1]
224             w2 = self.h[i] + 2.0 * self.h[i - 1]
225             m[i] = (w1 + w2) / (w1 / delta[i - 1] + w2 / delta[i])
226     self.m = m #保存节点导数备用
227
228     def _locate(self, xq):
229         return np.clip(
230             np.searchsorted(self.x, xq, side="right") - 1, 0, len(self.x) - 2
231         )
232
233     def __call__(self, xq):
234         xq = np.asarray(xq, float)
235         scalar = xq.ndim == 0
236         xq = np.atleast_1d(xq)
237         idx = self._locate(xq)
238         h = self.h[idx]
239         s = (xq - self.x[idx]) / h
240         y0, y1 = self.y[idx], self.y[idx + 1]
241         m0, m1 = self.m[idx], self.m[idx + 1]
242         out = (
243             (2 * s**3 - 3 * s**2 + 1) * y0
244             + (s**3 - 2 * s**2 + s) * h * m0
245             + (-2 * s**3 + 3 * s**2) * y1
246             + (s**3 - s**2) * h * m1
247         )
248         return float(out[0]) if scalar else out
249
250     def deriv(self, xq):
251         xq = np.asarray(xq, float)
252         scalar = xq.ndim == 0
253         xq = np.atleast_1d(xq)
254         idx = self._locate(xq)
255         h = self.h[idx]
256         s = (xq - self.x[idx]) / h
257         y0, y1 = self.y[idx], self.y[idx + 1]
258         m0, m1 = self.m[idx], self.m[idx + 1]
259         out = (
260             (6 * s**2 - 6 * s) * y0 / h
261             + (3 * s**2 - 4 * s + 1) * m0
262             + (-6 * s**2 + 6 * s) * y1 / h
263             + (3 * s**2 - 2 * s) * m1
264         )
265         return float(out[0]) if scalar else out
266
267     #主求解流程
268     def solve(
269         prop="p23",
270         shrink=False,
271         n=N_FINE,
272         dt=5.0,
273         tend=259200.0,
274         picard=6,
275         picard_tol=1e-11,
276         face_mean="arith",
277     ):
278         """参数
279         prop : str          "p23" 用附录 3 物性 (问题二、三); "p4" 用附录 4
280         shrink : bool       True 时启用附件 2 的收缩 (问题四)
281         n : int             贴体坐标  $\xi$  方向的网格单元数
282         dt : float          时间步长 [s]
283         tend : float        最长求解时间 [s] (达标即提前结束)
284         picard : int        Picard 最大迭代次数
285         picard_tol : float   Picard 收敛判据 (温度与水分的变化量)
286         face_mean : str     界面物性的平均方式 "arith" (算术) / "harm" (调和) / "log" (对数)
287         """
288         #返回字典
289         #xi          贴体网格节点
290         #times       实际推进到的时间序列 (达标时会被截断)

```

```

291 #temp/conc    两个场的完整时空解（贴体坐标）
292 #radius       每个时刻的半径 R(t)
293 #dry_time     中心含水率首次低于 0.15 的时刻 [s]，未达标为 None
294 #dt, n        实际使用的步长与网格
295 props = PROPERTY_SETS[prop] #取物性函数
296
297 #贴体网格
298 h = 1.0 / n #ξ方向步长（无量纲）
299 xi = np.arange(n + 1) * h #节点ξ_i
300 xif = (np.arange(n) + 0.5) * h #界面ξ_{i+1/2}
301
302 #控制体权重w_i=∫ξdξ，与问题一的w=∫rdr同源，还需要修正中心与表面
303 weight = xi * h
304 weight[0] = h * h / 8.0
305 weight[n] = h * (1.0 - h / 4.0) / 2.0
306
307 #收缩时构造R(t)的PCHIP插值对象；不收缩时为None。
308 radius = Pchip(_RAD_T, _RAD_V) if shrink else None
309
310 #初值
311 temp = np.full(n + 1, T_INIT)
312 conc = np.full(n + 1, C_INIT)
313 n_steps = int(round(tend / dt))
314 times = np.arange(n_steps + 1) * dt
315 temp_hist = np.empty((n_steps + 1, n + 1))
316 conc_hist = np.empty((n_steps + 1, n + 1))
317 radius_hist = np.empty(n_steps + 1)
318 temp_hist[0] = temp
319 conc_hist[0] = conc
320 radius_hist[0] = radius(0.0) if shrink else R0
321
322 dry_step = None #记录首次达标
323 for step in range(1, n_steps + 1):
324     t_now = step * dt
325     t_inf, c_inf = air_conditions(t_now)
326     r_now = radius(t_now) if shrink else R0 #当前半径 R(t)
327
328     #把时间步开始时的状态固定下来
329     conc_old = conc.copy()
330     temp_old = temp.copy()
331
332     #Picard迭代
333     for _ in range(picard):
334         rho, _, cp, k_cond, diff = props(conc, temp)
335         rho_cp = rho * cp
336         heat_face = _face_mean(k_cond, face_mean)
337         # 水分方程：四个问题统一用经典 Fick 口径（干基含水率，ρ 不进入）。
338         # 收缩只通过度量因子 R² 与边界条件中的 R 进入，见文件头推导。
339         mass_face = _face_mean(diff, face_mean)
340         lam = weight * r_now**2
341         fc = dt * mass_face * xif / h
342         alpha = dt * r_now * H_MASS
343         lower, diag, upper, rhs = _assemble(
344             lam, fc, conc_old, alpha, c_inf, n
345         )
346         conc_new = thomas(lower, diag, upper, rhs)
347
348     #温度方程
349     lam = rho_cp * weight * r_now**2
350     fc = dt * heat_face * xif / h
351     alpha = dt * r_now * H_HEAT
352     lower, diag, upper, rhs = _assemble(
353         lam, fc, temp_old, alpha, t_inf, n
354     )
355     temp_new = thomas(lower, diag, upper, rhs)
356
357     # 两个场的最大变化量都小于判据才算收敛
358     change = max(
359         float(np.max(np.abs(conc_new - conc))),
360         float(np.max(np.abs(temp_new - temp))),
361     )
362     temp, conc = temp_new, conc_new
363     if change < picard_tol:

```

```

364         break
365
366     #记录结果
367     temp_hist[step] = temp
368     conc_hist[step] = conc
369     radius_hist[step] = r_now
370
371     #烘干判据
372     if conc[0] < C_DRY:
373         dry_step = step
374         break
375
376     last = dry_step if dry_step is not None else n_steps
377     return {
378         "xi": xi,
379         "times": times[: last + 1],
380         "temp": temp_hist[: last + 1],
381         "conc": conc_hist[: last + 1],
382         "radius": radius_hist[: last + 1],
383         "dry_time": float(times[last]) if dry_step is not None else None,
384         "dt": dt,
385         "n": n,
386     }
387
388 #把贴体坐标解映射到固定物理位置
389 def physical_positions(sol, positions_cm):
390     xi = sol["xi"]
391     times = sol["times"]
392     n_pos = len(positions_cm)
393     temp_out = np.empty((len(times), n_pos))
394     conc_out = np.empty((len(times), n_pos))
395     for k in range(len(times)):
396         # 当前半径 R(t) 把物理位置换算成贴体坐标
397         target = np.clip(
398             np.asarray(positions_cm, float) / 100.0 / sol["radius"][k], 0.0, 1.0
399         )
400         temp_out[k] = np.interp(target, xi, sol["temp"][k])
401         conc_out[k] = np.interp(target, xi, sol["conc"][k])
402     return times, temp_out, conc_out
403
404 #程序入口
405 def main():
406     parser = argparse.ArgumentParser(description="A 题 问题二/三/四 求解")
407     parser.add_argument("--problem", type=int, required=True, choices=(2, 3, 4))
408     parser.add_argument("--n", type=int, default=N_FINE)
409     parser.add_argument("--dt", type=float, default=5.0)
410     parser.add_argument("--tend", type=float, default=259200.0)
411     parser.add_argument("--save", default=None)
412     args = parser.parse_args()
413
414     shrink = args.problem == 4
415     prop = "p4" if shrink else "p23"
416     print(
417         "求解问题%d: 物性=%s 收缩=%s N=%d dt=%g s tend=%g s"
418         % (args.problem, prop, shrink, args.n, args.dt, args.tend)
419     )
420     sol = solve(
421         prop=prop, shrink=shrink, n=args.n, dt=args.dt, tend=args.tend
422     )
423     print(
424         "中心含水率首次低于 %.2f 的时刻: %s s = %.2f h = %.3f 天"
425         % (
426             C_DRY,
427             sol["dry_time"],
428             (sol["dry_time"] or 0) / 3600,
429             (sol["dry_time"] or 0) / 86400,
430         )
431     )
432     #快速核对
433     for hours in (0.5, 1, 2, 3):
434         k = int(round(hours * 3600 / sol["dt"]))
435         if k < len(sol["times"]):
436             c = sol["conc"][k]

```

```

437         print(
438             " t=%4.1f h: 中心C=%0.4f 表面C=%0.4f 中心T=%0.2f 表面T=%0.2f"
439             % (
440                 hours,
441                 c[0],
442                 c[-1],
443                 sol["temp"][k][0],
444                 sol["temp"][k][-1],
445             )
446         )
447
448     if args.save:
449         path = Path(args.save)
450         path.parent.mkdir(parents=True, exist_ok=True)
451         np.savez_compressed(
452             path,
453             xi=sol["xi"],
454             times=sol["times"],
455             temp=sol["temp"],
456             conc=sol["conc"],
457             radius=sol["radius"],
458         )
459         print("已保存: %s" % path)
460
461 if __name__ == "__main__":
462     main()
463

```

完整程序清单（二）：对照求解器 p23_variants.py

第 A.1、A.2 节的对照实验由下面的脚本完成。它不重复实现离散格式，而是直接复用 solve_p234.py 中的物性、边界与三对角求解函数，只替换被测的那一个因素（网格数、时间步长或水分方程口径），以保证对照实验与论文主结果逐位同源。两种口径的实现差别集中在函数 solve_variant 的”水分方程”一节。

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """问题三：径向网格数 N、时间步长 Δt 与水分方程口径的对照求解器。
4
5  用途：复现论文第 10.1、10.2 节的两张对照表（表 14 与表 15）。
6
7  与主求解器 solve_p234.py 的关系
8  -----
9  本脚本**不重复实现**离散格式，而是直接复用 solve_p234 中的
10  物性经验式（props_p23）、空气条件插值（air_conditions）、
11  三对角组装（_assemble）与追赶法（thomas）。这样做的目的是保证
12  对照实验与论文主结果**逐位同源**：唯一被改变的量就是被测的那个因素
13  （N、Δt 或水分方程口径），而不是”另写一套代码”。
14
15  两种水分方程口径
16  -----
17  1. form="fick"（本文口径，经典 Fick 形式）
18     dC/dt = (1/r) * d/dr ( D*r*dC/dr )
19     未知量 C 为干基含水率，ρ 不出现；D 是针对 C 拟合的有效扩散系数。
20
21  2. form="cons"（对照口径，把体积密度 ρ 写进水分方程的守恒形式）
22     d(ρ*C)/dt = (1/r) * d/dr ( ρ*D*r*dC/dr )
23     其中 ρ = 650 + 128*C 取自附录 3。累计项系数为
24     accum = d(ρ*C)/dC = ρ + C*dρ/dC，
25     界面通量系数为 ρ*D。等价扩散系数为 ρ*D/(ρ + C*ρ')。
26
27  两式的差别只在水分方程内部，温度方程完全相同（都含 ρ*cp），
28  因此可以直接比较”烘干时长”这一单一输出。
29
30  两者的物理差别与选取理由见论文 §5.6（口径的闭合）与 §7.6.5（口径敏感性）。
31
32  跑法
33  ----
34  PY=/Users/sqz/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3

```

```

35
36 # 单组求解: N=200、dt=5 s、本文口径
37 $PY 代码/p23_variants.py --n 200 --dt 5 --form fick
38
39 # 复现论文表 14 与表 15 (并行, 约 5 分钟)
40 $PY 代码/p23_variants.py --table
41
42 # 只跑一小部分用于自检
43 $PY 代码/p23_variants.py --table --n-list 20,50 --dt-list 5
44
45 输出: 控制台打印烘干时长 (中心含水率首次低于 0.15 kg/kg 的时刻)。
46 """
47
48 from __future__ import annotations
49
50 import argparse
51 from multiprocessing import Pool
52 from pathlib import Path
53 import sys
54
55 import numpy as np
56
57 sys.path.insert(0, str(Path(__file__).resolve().parent))
58 import solve_p234 as base # noqa: E402
59
60
61 def solve_variant(form="fick", thermo="coupled", n=200, dt=5.0,
62                  tend=259200.0, picard=8, tol=1e-11):
63     """按指定口径求解问题三, 返回中心达标时刻。
64
65     参数
66     ----
67     form : "fick" (本文口径, 经典 Fick 形式) 或 "cons" (对照口径,
68           把体积密度  $\rho$  写进水分方程)
69     thermo : "coupled" 解耦合温度方程; "iso_air" 令温度场等于烘房温度
70            (用于检验"热质解耦"结论); "iso50" 全程 50 °C
71     n : 径向控制体个数 (求解网格, 不是输出网格)
72     dt : 时间步长 [s]
73     tend : 最长时间 [s]
74
75     返回
76     ----
77     dict: 中心达标时刻 t_center、表面达标时刻 t_surface、以及若干时刻快照
78     """
79     props = base.props_p23
80
81     # ---- 贴体网格与有限体积分权重 (与 solve_p234 完全一致) ----
82     h = 1.0 / n
83     xi = np.arange(n + 1) * h
84     xif = (np.arange(n) + 0.5) * h
85     weight = xi * h
86     weight[0] = h * h / 8.0 # 中心控制体
87     weight[n] = h * (1.0 - h / 4.0) / 2.0 # 表面控制体
88     radius = base.R0 # 问题三半径固定
89
90     temp = np.full(n + 1, base.T_INIT)
91     conc = np.full(n + 1, base.C_INIT)
92     n_steps = int(round(tend / dt))
93     t_center = t_surface = None
94     snaps = {}
95
96     for step in range(1, n_steps + 1):
97         t_now = step * dt
98         t_inf, c_inf = base.air_conditions(t_now)
99         conc_old = conc.copy()
100         temp_old = temp.copy()
101
102         # ---- Picard 迭代: 滞后物性, 把非线性方程化为线性三对角 ----
103         for _ in range(picard):
104             if thermo == "coupled":
105                 rho, drho, cp, k_cond, diff = props(conc, temp)
106             else:
107                 t_set = t_inf if thermo == "iso_air" else 50.0

```

```

108         temp = np.full(n + 1, t_set)
109         rho, drho, cp, k_cond, diff = props(conc, temp)
110
111         # ----- 水分方程 -----
112         if form == "fick":
113             #  $dC/dt = (1/r) d/dr (D r dC/dr)$ 
114             accum = np.ones_like(conc)
115             mass_face = 0.5 * (diff[:-1] + diff[1:])
116             alpha = dt * radius * base.H_MASS
117         else:
118             #  $d(\rho C)/dt = (1/r) d/dr (\rho D r dC/dr)$ 
119             accum = rho + conc * drho
120             mass_face = (0.5 * (rho[:-1] + rho[1:])) * (
121                 0.5 * (diff[:-1] + diff[1:])
122             )
123             alpha = dt * accum[n] * radius * base.H_MASS
124         lam = accum * weight * radius ** 2
125         fc = dt * mass_face * xif / h
126         lo, di, up, rhs = base._assemble(
127             lam, fc, conc_old, alpha, c_inf, n
128         )
129         conc_new = base.thomas(lo, di, up, rhs)
130
131         # ----- 温度方程 -----
132         if thermo == "coupled":
133             rho_cp = rho * cp
134             heat_face = 0.5 * (k_cond[:-1] + k_cond[1:])
135             lam_t = rho_cp * weight * radius ** 2
136             fc_t = dt * heat_face * xif / h
137             alpha_t = dt * radius * base.H_HEAT
138             lo_t, di_t, up_t, rhs_t = base._assemble(
139                 lam_t, fc_t, temp_old, alpha_t, t_inf, n
140             )
141             temp_new = base.thomas(lo_t, di_t, up_t, rhs_t)
142         else:
143             temp_new = temp
144
145         change = max(
146             float(np.max(np.abs(conc_new - conc))),
147             float(np.max(np.abs(temp_new - temp))),
148         )
149         temp, conc = temp_new, conc_new
150         if change < tol:
151             break
152
153         # ---- 记录达标时刻与快照 ----
154         if t_center is None and conc[0] < base.C_DRY:
155             t_center = t_now
156         if t_surface is None and conc[-1] < base.C_DRY:
157             t_surface = t_now
158         for hh in (6.0, 12.0, 24.0, 48.0):
159             if hh not in snaps and t_now >= hh * 3600:
160                 snaps[hh] = (t_now, conc.copy(), temp.copy())
161         if t_center is not None:
162             break
163
164         return dict(form=form, thermo=thermo, n=n, dt=dt, t_center=t_center,
165                     t_surface=t_surface, snaps=snaps)
166
167
168 def _job(args):
169     """给 multiprocessing.Pool 用的单组任务包装。"""
170     form, n, dt = args
171     r = solve_variant(form=form, n=n, dt=dt)
172     return form, n, dt, r["t_center"], r["t_surface"]
173
174
175 def build_tasks(n_list, dt_list, n_ref=100, dt_ref=5.0):
176     """构造两张表所需的 (口径, N, Δt) 任务集合。
177
178     表 14 固定 Δt=dt_ref、让 N 变；表 15 固定 N=n_ref、让 Δt 变。
179     两者的公共点 (n_ref, dt_ref) 只跑一次。
180     """

```

```

181     pairs = {(n, dt_ref) for n in n_list} | {(n_ref, dt) for dt in dt_list}
182     return [(f, n, dt) for f in ("cons", "fick") for n, dt in sorted(pairs)]
183
184
185 def run_table(n_list, dt_list, jobs, n_ref=100, dt_ref=5.0):
186     """并行跑全部对照任务。返回结果列表。"""
187     tasks = build_tasks(n_list, dt_list, n_ref, dt_ref)
188     with Pool(processes=min(jobs, len(tasks))) as pool:
189         out = pool.map(_job, tasks)
190     return out
191
192
193 def _fmt(rows, n_list, dt_list, n_ref=100, dt_ref=5.0):
194     """打印两张表: 表 14 (N 的影响) 与表 15 (Δt 的影响)。"""
195     index = {(f, n, dt): v for f, n, dt, v, _ in rows}
196     head = ("%5s %8s %10s %9s %12s %9s %9s %9s"
197             % ("N", "dt/s", "本文/s", "本文/h", "变密度/s", "变密度/h",
198               "差值/s", "相对"))
199     lines = []
200     for title, cases in (
201         ("表 14 径向网格数的影响 (Δt=%g s)" % dt_ref,
202          [(n, dt_ref) for n in n_list]),
203         ("表 15 时间步长的影响 (N=%d)" % n_ref,
204          [(n_ref, dt) for dt in dt_list]),
205     ):
206         lines += [title, head]
207         for n, dt in cases:
208             fick = index.get(("fick", n, dt))
209             cons = index.get(("cons", n, dt))
210             d = cons - fick
211             lines.append("%5d %8g %10.1f %9.3f %12.1f %9.3f %9.1f %8.2f%"
212                          % (n, dt, fick, fick / 3600.0, cons, cons / 3600.0,
213                            d, 100.0 * d / fick))
214     return lines
215
216
217 def main():
218     ap = argparse.ArgumentParser(description="问题三 网格/时间步/口径 对照")
219     ap.add_argument("--form", default="fick", choices=("cons", "fick"))
220     ap.add_argument("--thermo", default="coupled",
221                     choices=("coupled", "iso_air", "iso50"))
222     ap.add_argument("--n", type=int, default=200)
223     ap.add_argument("--dt", type=float, default=5.0)
224     ap.add_argument("--tend", type=float, default=259200.0)
225     ap.add_argument("--table", action="store_true", help="跑完整对照表")
226     ap.add_argument("--n-list", default="20,50,100,200,400")
227     ap.add_argument("--dt-list", default="2,5,10,20,60,120")
228     ap.add_argument("--n-ref", type=int, default=100)
229     ap.add_argument("--dt-ref", type=float, default=5.0)
230     ap.add_argument("--jobs", type=int, default=6)
231     ap.add_argument("--csv", default=None)
232     a = ap.parse_args()
233
234     if not a.table:
235         r = solve_variant(form=a.form, thermo=a.thermo, n=a.n, dt=a.dt,
236                           tend=a.tend)
237         print("form=%s thermo=%s N=%d dt=%g" % (a.form, a.thermo, a.n, a.dt))
238         print(" 中心达标 %8.1f s = %6.3f h" % (r["t_center"],
239                                               r["t_center"] / 3600.0))
240         print(" 表面达标 %8.1f s = %6.3f h" % (r["t_surface"],
241                                               r["t_surface"] / 3600.0))
242     return
243
244     n_list = [int(v) for v in a.n_list.split(",")]
245     dt_list = [float(v) for v in a.dt_list.split(",")]
246     rows = run_table(n_list, dt_list, a.jobs, a.n_ref, a.dt_ref)
247     for line in _fmt(rows, n_list, dt_list, a.n_ref, a.dt_ref):
248         print(line)
249     if a.csv:
250         with open(a.csv, "w") as fh:
251             fh.write("form,N,dt_s,t_center_s,t_surface_s\n")
252             for form, n, dt, tc, ts in rows:
253                 fh.write("%s,%d,%g,%.1f,%.1f\n" % (form, n, dt, tc, ts))

```

```

254         print("已保存: %s" % a.csv)
255
256
257 if __name__ == "__main__":
258     main()

```

完整程序清单（三）：问题一求解器 solve_p1.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """2026 高教社杯全国大学生数学建模竞赛 A 题 问题一求解。
4
5  控制方程（圆柱坐标，一维径向轴对称）：
6      热量：  $\rho \cdot c_p \cdot dT/dt = (1/r) \cdot d/dr (k \cdot r \cdot dT/dr)$ 
7      水分：  $dC/dt = (1/r) \cdot d/dr (D(C) \cdot r \cdot dC/dr)$ 
8
9  定解条件：
10      $T(r,0) = 28 \text{ C}$ ,  $C(r,0) = 2.55 \text{ kg/kg}$ 
11      $r = 0$  :  $dT/dr = 0$ ,  $dC/dr = 0$  (对称)
12      $r = R$  :  $-k dT/dr = h \cdot (T_s - T_{inf}(t))$  (对流换热)
13               $-D dC/dr = hm \cdot (C_s - C_{inf}(t))$  (对流传质)
14
15      $T_{inf}(t)$ 、 $C_{inf}(t)$  由附件 1 线性插值给出。物性取自附录 2，其中
16      $D(C) = 7e-9 \cdot \exp(-0.89 / C) \text{ [m}^2/\text{s]}$ 
17
18 建模选择（详见 建模/问题一-物理模型与公式.md）：
19 1. 传质边界采用干基密度口径， $\rho_d$  两侧约去，故写  $-D dC/dr = hm (C_s - C_{inf})$ 。
20 2. 忽略汽化潜热。若计入，按同一密度口径会把表面压到湿球温度(34.7 C)以下，物理不可能。
21 3. 水分方程取简单扩散形式，即  $D$  的经验公式视为相对总密度定义。
22
23 数值方法：
24 节点中心有限体积（离散格式守恒）+ 后向 Euler（无条件稳定）。
25  $D(C)$  的非线性用 Picard 迭代。默认细网格  $N=400$  ( $h = 0.005 \text{ cm}$ )，
26 题目要求的输出节点  $0, 0.1, \dots, 2.0 \text{ cm}$  恰为网格子集，无需插值。
27
28 运行环境（系统 python3 缺 openpyxl，请用打包运行时）：
29      $PY=/Users/sqz/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3$ 
30      $\$PY \text{ 代码/solve\_p1.py --save 输出/result1\_data.npz}$ 
31 """
32
33 from __future__ import annotations
34 import argparse
35 from pathlib import Path
36 import numpy as np
37 import openpyxl
38
39
40 PROJECT = Path(__file__).resolve().parent.parent
41 ATTACHMENT1 = PROJECT / "附件" / "附件1-烘房温度与水分浓度.xlsx"
42
43 RHO = 820.0 # 体积密度 kg/m^3
44 CP = 2600.0 # 比热容 J/(kg K)
45 K_COND = 0.36 # 热传导系数 W/(m K)
46 H_HEAT = 25.0 # 对流换热系数 W/(m^2 K)
47 H_MASS = 8.0e-7 # 对流传质系数 m/s
48
49 RADIUS = 0.02 # 半径 m
50 LENGTH = 0.25 # 长度 m
51
52 T_INIT = 28.0 # 药材温度初值 C
53 C_INIT = 2.55 # 水分浓度初值 kg/kg
54
55 T_END = 1800.0 # 模拟时长 s
56 DT = 1.0 # 时间步长 s
57
58 N_FINE = 400 # 细网格单元数，为 N_OUT 的整数倍，避免插值
59 N_OUT = 20 # 输出网格单元数 (0.1 cm 间隔，21 个节点)
60
61 ALPHA = K_COND / (RHO * CP)
62

```

```

63 REPORT_TIMES = [100, 300, 600, 900, 1200, 1500, 1800]
64 REPORT_POS_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
65
66
67 def load_air_conditions(path=ATTACHMENT1):
68     #读取附件1, 得到边界条件
69     wb = openpyxl.load_workbook(path, data_only=True)
70     rows = [
71         row
72         for row in wb.active.iter_rows(min_row=2, values_only=True)
73         if row[0] is not None
74     ]
75     t = np.array([row[0] for row in rows], dtype=float)
76     temp = np.array([row[1] for row in rows], dtype=float)
77     humid = np.array([row[2] for row in rows], dtype=float)
78     return t, temp, humid
79
80
81 def diffusion_coefficient(c):
82     #附录 2 的水分浓度扩散系数经验公式 [m^2/s]
83     return 7.0e-9 * np.exp(-0.89 / np.maximum(c, 1e-12))    #防止C=0导致除零
84
85
86 def cell_weights(n_fine, hx):
87     #用于守恒检验
88     """有限体积控制体的径向权重 w_i, 满足  $\int_V dV = 2\pi L \sum w_i f_i$ 。
89
90     内部节点:  $w_i = r_i * hx$ 
91     中心节点 (i=0): 控制体为  $[0, hx/2]$ ,  $V = \pi * hx^2 * L / 4 \rightarrow w_0 = hx^2 / 8$ 
92     表面节点 (i=N): 控制体为  $[R-hx/2, R]$ ,  $V = \pi * hx * L * (R-hx/4) \rightarrow w_N = hx * (R-hx/4) / 2$ 
93
94     特别地,  $r_0 = 0$ , 直接用  $r_i * hx$  会把中心控制体权重记成 0, 导致守恒检验出现虚假残差。
95     """
96     r = np.arange(n_fine + 1) * hx
97     w = r * hx
98     w[0] = hx * hx / 8.0
99     w[n_fine] = hx * (RADIUS - hx / 4.0) / 2.0
100    return w
101
102
103 def thomas(lower, diag, upper, rhs):
104     #追赶法求解三对角方程组
105     n = len(diag)
106     cp = np.empty(n)
107     dp = np.empty(n)
108     cp[0] = upper[0] / diag[0]
109     dp[0] = rhs[0] / diag[0]
110     for i in range(1, n):
111         m = diag[i] - lower[i] * cp[i - 1]
112         cp[i] = upper[i] / m
113         dp[i] = (rhs[i] - lower[i] * dp[i - 1]) / m
114     x = np.empty(n)
115     x[-1] = dp[-1]
116     for i in range(n - 2, -1, -1):
117         x[i] = dp[i] - cp[i] * x[i + 1]
118     return x
119
120
121 def step_temperature(temp, t_inf, dt, hx, r, rf):
122     '''1.用后向 Euler 进行时间离散, 无条件稳定
123     2.得到线性三对角方程组, 不需要迭代, 直接用 thomas 求解
124     '''
125     n = len(r) - 1
126     a_t = K_COND * dt / (RHO * CP * hx * hx)
127     g_t = dt / (RHO * CP * hx * (RADIUS - hx / 4.0))
128     p_t = 2.0 * g_t * RADIUS * H_HEAT
129     q_t = 2.0 * g_t * (RADIUS - hx / 2.0) * K_COND / hx
130
131     lower = np.zeros(n + 1)
132     diag = np.zeros(n + 1)
133     upper = np.zeros(n + 1)
134     rhs = np.zeros(n + 1)
135

```

```

136     diag[0] = 1.0 + 4.0 * a_t
137     upper[0] = -4.0 * a_t
138     rhs[0] = temp[0]
139     for i in range(1, n):
140         lower[i] = -a_t * rf[i - 1]
141         upper[i] = -a_t * rf[i]
142         diag[i] = r[i] + a_t * (rf[i - 1] + rf[i])
143         rhs[i] = r[i] * temp[i]
144
145     lower[n] = -q_t
146     diag[n] = 1.0 + p_t + q_t
147     rhs[n] = temp[n] + p_t * t_inf
148     return thomas(lower, diag, upper, rhs)
149
150
151 def step_moisture(conc, c_inf, dt, hx, r, rf, picard_max, picard_tol):
152     '''后向 Euler + Picard 迭代推进一步水分场 (D 依赖 C):
153     1. 结构类似于 step_temperature, 采用后向 Euler 得到离散的水分方程
154     2. D(C) 非线性, 采用 Picard 迭代线性化, 再使用 thomas 求解三对角方程组
155     '''
156     n = len(r) - 1
157     g_c = dt / (hx * (RADIUS - hx / 4.0))
158     p_c = 2.0 * g_c * RADIUS * H_MASS
159     q_base = 2.0 * g_c * (RADIUS - hx / 2.0) / hx
160
161     c_old = conc.copy()
162     c_new = conc.copy()
163     for _ in range(picard_max):
164         d_node = diffusion_coefficient(c_new)
165         d_face = 0.5 * (d_node[:-1] + d_node[1:])
166         dtb = dt * d_face / (hx * hx)
167         b0 = dt * d_face[0] / (hx * hx)
168
169         lower = np.zeros(n + 1)
170         diag = np.zeros(n + 1)
171         upper = np.zeros(n + 1)
172         rhs = np.zeros(n + 1)
173
174         diag[0] = 1.0 + 4.0 * b0
175         upper[0] = -4.0 * b0
176         rhs[0] = c_old[0]
177         for i in range(1, n):
178             bm = dtb[i - 1] * rf[i - 1]
179             bp = dtb[i] * rf[i]
180             lower[i] = -bm
181             upper[i] = -bp
182             diag[i] = r[i] + bm + bp
183             rhs[i] = r[i] * c_old[i]
184
185         q_c = q_base * d_face[n - 1]
186         lower[n] = -q_c
187         diag[n] = 1.0 + p_c + q_c
188         rhs[n] = c_old[n] + p_c * c_inf
189
190         solved = thomas(lower, diag, upper, rhs)
191         if np.max(np.abs(solved - c_new)) < picard_tol:
192             c_new = solved
193             break
194         c_new = solved
195     return c_new
196
197
198 def solve(n_fine=N_FINE, dt=DT, tend=T_END, picard_max=8, picard_tol=1e-13):
199     # 问题一求解
200     hx = RADIUS / n_fine
201     r = np.arange(n_fine + 1) * hx
202     rf = (np.arange(n_fine) + 0.5) * hx
203
204     stride = n_fine // N_OUT
205     if stride * N_OUT != n_fine:
206         raise ValueError("'n_fine' 必须是 N_OUT=%d 的整数倍" % N_OUT)
207
208     t_air, temp_air, humid_air = load_air_conditions()

```

```

209
210     n_steps = int(round(tend / dt))
211     temp = np.full(n_fine + 1, T_INIT)
212     conc = np.full(n_fine + 1, C_INIT)
213
214     times = np.arange(1, n_steps + 1) * dt
215     temp_out = np.empty((n_steps + 1, N_OUT + 1))
216     conc_out = np.empty((n_steps + 1, N_OUT + 1))
217     surf_flux = np.zeros(n_steps + 1)
218     temp_out[0] = temp[::stride]
219     conc_out[0] = conc[::stride]
220
221     # 守恒检验用的细网格标量
222     vol_factor = 2.0 * np.pi * LENGTH
223     weights = cell_weights(n_fine, hx)
224     c_volume = np.zeros(n_steps + 1)      #  $Q = \int C \, dV$ 
225     phys_volume = np.zeros(n_steps + 1)   #  $W = \int \rho_d C \, dV$ 
226     c_volume[0] = vol_factor * float(np.sum(weights * conc))
227     phys_volume[0] = vol_factor * RHO * float(
228         np.sum(weights * conc / (1.0 + conc))
229     )
230
231     for step in range(1, n_steps + 1):
232         t_now = step * dt
233         t_inf = float(np.interp(t_now, t_air, temp_air))
234         c_inf = float(np.interp(t_now, t_air, humid_air))
235         conc = step_moisture(
236             conc, c_inf, dt, hx, r, rf, picard_max, picard_tol
237         )
238         temp = step_temperature(temp, t_inf, dt, hx, r, rf)
239         temp_out[step] = temp[::stride]
240         conc_out[step] = conc[::stride]
241         surf_flux[step] = H_MASS * (conc[-1] - c_inf)
242         c_volume[step] = vol_factor * float(np.sum(weights * conc))
243         phys_volume[step] = vol_factor * RHO * float(
244             np.sum(weights * conc / (1.0 + conc))
245         )
246
247     return {
248         "times": times,
249         "r_out_cm": np.arange(N_OUT + 1) * (RADIUS / N_OUT) * 100.0,
250         "temp_out": temp_out,
251         "conc_out": conc_out,
252         "surf_flux": surf_flux,
253         "c_volume": c_volume,
254         "phys_volume": phys_volume,
255         "hx": hx,
256         "dt": dt,
257         "final_temp": temp,
258         "final_conc": conc,
259     }
260
261 def conservation_report(res):
262     """水分守恒检验（在细网格上精确积分）。
263
264     简单扩散形式的守恒量是  $Q = \int C \, dV$ ，其减少量应等于表面累计流出
265          $A * \int_{hm} (C_s - C_{inf}) \, dt$ ，  $A = 2 * \pi * R * L$ 
266
267     另报告物理水量  $W = \int \rho_d C \, dV = \int \rho_d C / (1 + C) \, dV$ ，用于说明
268     简单模型与真实水分质量的差别（后者不守恒是密度口径近似的后果）。
269     """
270
271     area = 2.0 * np.pi * RADIUS * LENGTH
272     q = res["c_volume"]
273     removed = float(q[0] - q[-1])
274     # 离散格式的守恒恒等式是左矩形和（通量取新时刻），
275     # 用梯形法反而会引入  $O(dt)$  的假残差。
276     cumulative = area * res["dt"] * float(np.sum(res["surf_flux"][1:]))
277     phys = res["phys_volume"]
278     return {
279         "q_start": float(q[0]),
280         "q_end": float(q[-1]),
281         "removed": removed,

```

```

282         "cumulative_flux": cumulative,
283         "relative_residual": (removed - cumulative) / removed,
284         "phys_start": float(phys[0]),
285         "phys_end": float(phys[-1]),
286     }
287
288
289 def print_report(res):
290     #打印表 1 与表 2
291     times = res["times"]
292     pos_idx = [int(round(p / 0.1)) for p in REPORT_POS_CM]
293     time_idx = [int(round(x / res["dt"])) for x in REPORT_TIMES]
294     blocks = (
295         ("表1 30 分钟内药材的温度", res["temp_out"], "°C"),
296         ("表2 30 分钟内药材的水分浓度", res["conc_out"], "kg/kg"),
297     )
298     for title, hist, unit in blocks:
299         print("\n=== %s (单位: %s) ===" % (title, unit))
300         header = "时间/s |" + "".join(
301             "%10s" % ("%gcm" % p) for p in REPORT_POS_CM
302         )
303         print(header)
304         print("-" * len(header))
305         for ti in time_idx:
306             row = "%6d |" % times[ti - 1]
307             row += "".join("%10.4f" % hist[ti, pj] for pj in pos_idx)
308             print(row)
309
310
311 def markdown_tables(res):
312     #把表 1、表 2 渲染成 Markdown
313     pos_idx = [int(round(p / 0.1)) for p in REPORT_POS_CM]
314     time_idx = [int(round(x / res["dt"])) for x in REPORT_TIMES]
315     out = [
316         "# 问题一结果表",
317         "",
318         "由 `代码/solve_p1.py` 计算, 细网格 $N=400$, $\Delta t=1$ s, 保留四位小数。",
319         "",
320     ]
321     blocks = (
322         ("表 1 30 分钟内药材的温度 (单位: °C)", res["temp_out"]),
323         ("表 2 30 分钟内药材的水分浓度 (单位: kg/kg)", res["conc_out"]),
324     )
325     for title, hist in blocks:
326         out.append("## " + title)
327         out.append("")
328         out.append(
329             "| 时间/s | "
330             + " | ".join("%g" % p for p in REPORT_POS_CM)
331             + " | "
332         )
333         out.append("| --- |" + " --- |" * len(REPORT_POS_CM))
334         for ti in time_idx:
335             out.append(
336                 "| %d |" % res["times"][ti - 1]
337                 + " | ".join("%.4f" % hist[ti, pj] for pj in pos_idx)
338                 + " | "
339             )
340             out.append("")
341         out.append("> 到药材中心的距离, 单位 cm。完整结果见 `result1.xlsx`。")
342         out.append("")
343         return "\n".join(out)
344
345
346 def main():
347     parser = argparse.ArgumentParser(description="A 题问题一求解")
348     parser.add_argument("--n", type=int, default=N_FINE, help="细网格单元数")
349     parser.add_argument("--dt", type=float, default=DT, help="时间步长 s")
350     parser.add_argument("--save", default=None, help="保存解的 .npz 路径")
351     parser.add_argument("--tables", default=None, help="导出表 1/表 2 的 Markdown 路径")
352     args = parser.parse_args()
353
354     print("求解中: N=%d, dt=%g s, 时长 %g s" % (args.n, args.dt, T_END))

```

```

355     res = solve(n_fine=args.n, dt=args.dt)
356     print_report(res)
357
358     rep = conservation_report(res)
359     print("\n=== 水分守恒检验 (细网格精确积分) ===")
360     print(" 模型守恒量 Q =  $\int C \, dV$  [m3*kg/kg]")
361     print(" 初始      : %.8e" % rep["q_start"])
362     print(" 末态      : %.8e" % rep["q_end"])
363     print(" 减少      : %.8e" % rep["removed"])
364     print(" 表面累计流出: %.8e" % rep["cumulative_flux"])
365     print(" 相对残差   : %.3e" % rep["relative_residual"])
366     print(" 物理水量 W =  $\int \rho_d * C \, dV$ ")
367     print(" 初始      : %.6f kg (干料 %.6f kg)"
368           % (rep["phys_start"], rep["phys_start"] / C_INIT))
369     print(" 末态      : %.6f kg" % rep["phys_end"])
370     print(" 脱除      : %.6f kg" % (rep["phys_start"] - rep["phys_end"]))
371
372     print(
373         "\n度量: alpha=%.4e m2/s, D(C0)=%.4e m2/s, Bi=%.3f, Bi_m=%.3f"
374         % (
375             ALPHA,
376             diffusion_coefficient(C_INIT),
377             H_HEAT * RADIUS / K_COND,
378             H_MASS * RADIUS / diffusion_coefficient(C_INIT),
379         )
380     )
381
382     if args.save:
383         path = Path(args.save)
384         path.parent.mkdir(parents=True, exist_ok=True)
385         np.savez_compressed(
386             path,
387             times=res["times"],
388             r_out_cm=res["r_out_cm"],
389             temp_out=res["temp_out"],
390             conc_out=res["conc_out"],
391         )
392         print("\n已保存: %s" % path)
393         # 同时导出 JSON, 供 代码/build_result1.mjs 生成 result1.xlsx
394         import json
395
396         json_path = path.with_suffix(".json")
397         with json_path.open("w", encoding="utf-8") as fh:
398             json.dump(
399                 {
400                     "radius_cm": [round(float(v), 6) for v in res["r_out_cm"]],
401                     "times": [int(v) for v in res["times"]],
402                     # 去掉 t=0 的初始行, 使第 i 行严格对应 times[i]
403                     "temperature": np.round(res["temp_out"][1:], 4).tolist(),
404                     "moisture": np.round(res["conc_out"][1:], 4).tolist(),
405                 },
406                 fh,
407             )
408         print("已保存: %s" % json_path)
409
410     if args.tables:
411         path = Path(args.tables)
412         path.parent.mkdir(parents=True, exist_ok=True)
413         path.write_text(markdown_tables(res), encoding="utf-8")
414         print("已保存: %s" % path)
415
416
417 if __name__ == "__main__":
418     main()

```

完整程序清单（四）：干基密度守恒口径 closure_rhod.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """问题三的第三种水分方程口径：干基密度守恒。

```

```

4
5 论文表 11 列了三种口径, 但 `p23_variants.py` 只实现了前两种 (经典 Fick 与
6 体积密度守恒), 第三行 "变密度守恒口径 (等效  $D \rightarrow (1+C)D$ )  $\rightarrow 51.39 \text{ h}$ " 一直
7 没有对应的实现, 属于"文中有数、代码无源"。本脚本把它补上。
8
9 三种口径 (几何同为固定半径  $R_0$ , 温度方程完全相同):
10
11 1. fick  $\partial C / \partial t = (1/r) \partial_r (r D \partial_r C)$ 
12    accum = 1, face = D
13 2. cons  $\partial(\rho C) / \partial t = (1/r) \partial_r (r \rho D \partial_r C)$   $\rho = a+bC$ 
14    accum =  $\rho + C\rho'$ , face =  $\rho D$ 
15 3. rhod  $\partial(\rho_d C) / \partial t = (1/r) \partial_r (r \rho_d D \partial_r C)$   $\rho_d = \rho / (1+C)$ 
16    accum =  $\rho_d + C\rho_d'$ , face =  $\rho_d D$ 
17    其中  $\rho_d' = d\rho_d/dC = (b-a)/(1+C)^2$ 
18
19 表面第三类边界沿用与 cons 口径相同的写法 ( $\alpha = dt \cdot \text{accum}_N \cdot R \cdot h_m$ ),
20 以保证三种口径之间只有"累积项与通量项用什么密度"这一个差别。
21
22 自检: 本脚本给出的 fick / cons 必须与 输出/问题三-网格与口径对照.csv
23 逐位一致, 否则说明驱动方式与主对照求解器不同源。
24
25 跑法:
26 PY=/Users/sqz/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3
27 $PY 代码/closure_rhod.py --n 100 --dt 5 --form rhod
28 $PY 代码/closure_rhod.py --all # 三种口径 × 两套网格
29
30
31 from __future__ import annotations
32
33 import argparse
34 import csv
35 import sys
36 from pathlib import Path
37
38 import numpy as np
39
40 HERE = Path(__file__).resolve().parent
41 PROJECT = HERE.parent
42 OUT = PROJECT / "输出"
43 sys.path.insert(0, str(HERE))
44
45 import solve_p234 as base # noqa: E402
46
47 A3, B3 = 650.0, 128.0 # 附录 3 的  $\rho=a+bC$ 
48
49
50 def solve(form="rhod", n=100, dt=5.0, tend=259200.0, picard=8, tol=1e-11,
51          bc="accum"):
52     """按指定口径求解问题三 (固定半径), 返回中心首次达标时刻。"""
53     h = 1.0 / n
54     xi = np.arange(n + 1) * h
55     xif = (np.arange(n) + 0.5) * h
56     weight = xi * h
57     weight[0] = h * h / 8.0
58     weight[n] = h * (1.0 - h / 4.0) / 2.0
59     radius = base.R0
60
61     temp = np.full(n + 1, base.T_INIT)
62     conc = np.full(n + 1, base.C_INIT)
63     n_steps = int(round(tend / dt))
64
65     for step in range(1, n_steps + 1):
66         t_now = step * dt
67         t_inf, c_inf = base.air_conditions(t_now)
68         conc_old, temp_old = conc.copy(), temp.copy()
69
70         for _ in range(picard):
71             rho, drho, cp, k_cond, diff = base.props_p23(conc, temp)
72             # ---- 水分方程: 三种口径只差 accum 与 face ----
73             if form == "fick":
74                 accum = np.ones_like(conc)
75                 mass_face = 0.5 * (diff[:-1] + diff[1:])
76             else:

```

```

77         if form == "cons":
78             rho_r, drho_r = rho, drho
79         elif form == "rhod":
80             c = np.maximum(conc, 1e-12)
81             rho_r = rho / (1.0 + c)
82             drho_r = drho / (1.0 + c) - rho / (1.0 + c) ** 2
83         else:
84             raise ValueError(form)
85         accum = rho_r + conc * drho_r
86         mass_face = (0.5 * (rho_r[:-1] + rho_r[1:])) * (
87             0.5 * (diff[:-1] + diff[1:]))
88         if form == "fick":
89             alpha = dt * radius * base.H_MASS
90         elif bc == "accum":
91             alpha = dt * accum[n] * radius * base.H_MASS
92         else:
93             # 用表面处的密度本身作 Robin 系数
94             alpha = dt * rho_r[n] * radius * base.H_MASS
95
96         lam = accum * weight * radius ** 2
97         fc = dt * mass_face * xif / h
98         lo, di, up, rhs = base._assemble(
99             lam, fc, conc_old, alpha, c_inf, n)
100         conc_new = base.thomas(lo, di, up, rhs)
101
102         # ---- 温度方程 (三种口径完全相同) ----
103         rho_cp = rho * cp
104         heat_face = 0.5 * (k_cond[:-1] + k_cond[1:])
105         lam_t = rho_cp * weight * radius ** 2
106         fc_t = dt * heat_face * xif / h
107         alpha_t = dt * radius * base.H_HEAT
108         lo_t, di_t, up_t, rhs_t = base._assemble(
109             lam_t, fc_t, temp_old, alpha_t, t_inf, n)
110         temp_new = base.thomas(lo_t, di_t, up_t, rhs_t)
111
112         change = max(float(np.max(np.abs(conc_new - conc))),
113                     float(np.max(np.abs(temp_new - temp))))
114         temp, conc = temp_new, conc_new
115         if change < tol:
116             break
117
118         if conc[0] < base.C_DRY:
119             return t_now
120     return None
121
122 def main() -> None:
123     ap = argparse.ArgumentParser()
124     ap.add_argument("--form", default="rhod",
125                     choices=("fick", "cons", "rhod"))
126     ap.add_argument("--bc", default="accum", choices=("accum", "rho"))
127     ap.add_argument("--n", type=int, default=100)
128     ap.add_argument("--dt", type=float, default=5.0)
129     ap.add_argument("--all", action="store_true")
130     args = ap.parse_args()
131
132     if not args.all:
133         t = solve(form=args.form, n=args.n, dt=args.dt, bc=args.bc)
134         print("%s 口径 N=%d dt=%g : t_dry = %.0f s = %.2f h"
135               % (args.form, args.n, args.dt, t, t / 3600))
136         return
137
138     print("三种口径对照 (固定半径, 附录 3 物性)")
139     print("%-6s %-5s %10s %8s %9s" % ("口径", "N", "t_dry/s", "t_dry/h", "相对 Fick"))
140     ref = {}
141     for n in (100, 200):
142         for form in ("fick", "cons", "rhod"):
143             t = solve(form=form, n=n, dt=5.0)
144             if form == "fick":
145                 ref[n] = t
146             print("%-6s %-5d %10.0f %8.2f %8.2f%%"
147                   % (form, n, t, t / 3600, 100 * (t / ref[n] - 1)))
148     print()
149

```

```

150 # 与既有对照 CSV 交叉验证（只在前两种口径上）
151 csvp = OUT / "问题三-网格与口径对照.csv"
152 if csvp.exists():
153     tab = {(r["form"], int(r["N"]), float(r["dt_s"])): float(r["t_center_s"])}
154     for r in csv.DictReader(csvp.open(encoding="utf-8")):
155         print("与 输出/问题三-网格与口径对照.csv 的交叉验证: ")
156         for form in ("fick", "cons"):
157             for n in (100, 200):
158                 mine = solve(form=form, n=n, dt=5.0)
159                 theirs = tab[(form, n, 5.0)]
160                 flag = "一致" if abs(mine - theirs) < 1e-6 else "不一致"
161                 print(" %-5s N=%-4d 本脚本 %8.0f s : CSV %8.0f s %s"
162                       % (form, n, mine, theirs, flag))
163
164
165 if __name__ == "__main__":
166     main()

```

完整程序清单（五）：参数灵敏度 sens.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """问题三：关键参数（h, hm, D）±20% 灵敏度（经典 Fick 口径）。
4
5  输出论文"参数灵敏度表"的数值。基准与网格/时间步收敛表一致：
6  N=100、Δt=5 s、Fick 口径。
7
8  运行：
9      PY=/Users/sqz/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3
10     $PY 代码/sens.py
11 """
12 from __future__ import annotations
13 import sys
14 from pathlib import Path
15
16 import numpy as np
17
18 sys.path.insert(0, str(Path(__file__).resolve().parent))
19 import solve_p234 as base # noqa: E402
20
21
22 def run(hscale=1.0, hmscale=1.0, dscale=1.0, n=100, dt=5.0):
23     """按扰动后的物性重解问题三、返回烘干时长（秒）。"""
24     H0, HM0 = base.H_HEAT, base.H_MASS
25     orig = base.PROPERTY_SETS["p23"]
26
27     def props(conc, temp):
28         rho, drho, cp, k, d = orig(conc, temp)
29         return rho, drho, cp, k, d * dscale
30
31     base.H_HEAT = H0 * hscale
32     base.H_MASS = HM0 * hmscale
33     base.PROPERTY_SETS["p23"] = props
34     try:
35         sol = base.solve(prop="p23", shrink=False, n=n, dt=dt, tend=259200.0)
36         return sol["dry_time"]
37     finally:
38         base.H_HEAT, base.H_MASS = H0, HM0
39         base.PROPERTY_SETS["p23"] = orig
40
41
42 def main():
43     cases = [
44         ("基准值", 1.0, 1.0, 1.0),
45         ("h -20%", 0.8, 1.0, 1.0),
46         ("h +20%", 1.2, 1.0, 1.0),
47         ("hm -20%", 1.0, 0.8, 1.0),
48         ("hm +20%", 1.0, 1.2, 1.0),
49         ("D -20%", 1.0, 1.0, 0.8),
50         ("D +20%", 1.0, 1.0, 1.2),

```

```

51     ]
52     base_t = None
53     for label, hs, hms, ds in cases:
54         t = run(hs, hms, ds)
55         if base_t is None:
56             base_t = t
57         rel = 100.0 * (t - base_t) / base_t
58         print("%-9s %8.0f s %8.3f h %+7.2f%%" % (label, t, t / 3600.0, rel))
59
60
61 if __name__ == "__main__":
62     main()

```

完整程序清单（六）：收缩判据 shrink_criterion.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """$11「收缩的建模判据与代价」的全部计算。
4
5  三块内容
6  -----
7  1. **判据（解析）**：收缩几何只由斜率截距比  $r = b/a$  决定。给定半径容差  $\epsilon$  与
8     工作窗口  $[C_{\min}, C_0]$ ，忽略收缩安全的充要条件是  $r \geq r*(\epsilon, C_{\min})$ ，等价地
9      $C_{\min} \geq C*(\epsilon, r)$ 。两式都有闭式解，按各向异性指数  $n$  分两支
10     ( $n=2$  径向等长、 $n=3$  各向同性)。
11
12  2. **对照（数值）**：固定附录 3 的物性，只切换"几何"这一个开关，看烘干时长
13     变多少：
14         fixed      —— 固定半径  $R_0$ （本文问题二/三的做法）
15         measured   —— 附件 2 的实测  $R(t)$ （问题四的做法）
16         implied    —— 由  $p=a+bC$  反演的  $R(\bar{C})$ ，自洽闭合（本文构造的第三条路）
17     implied 用不动点迭代：先按固定半径解出  $\bar{C}(t)$ ，由  $r$  反演  $R(t)$ ，再解一次，
18     迭代若干轮至  $R$  稳定。
19
20  3. **阈值（数值）**：固定  $a=650$  与附录 3 的  $D, c_p, k$ ，只改斜率  $b$ （因而改  $r$ ），
21     对每个  $r$  求  $t_{\text{fixed}}$  与  $t_{\text{implied}}$ ，得到  $\Delta(r) = t_{\text{fixed}}/t_{\text{implied}} - 1$ ，
22     再取  $\Delta$  达到 5%/10%/20% 的  $r$  值，即**时长口径**的阈值  $r*time$ 。
23
24  跑法：
25     PY=/Users/sqz/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3
26     $PY 代码/shrink_criterion.py --analytic          # 只算闭式判据（秒级）
27     $PY 代码/shrink_criterion.py --matrix --jobs 4   # 加几何对照（约 3 分钟）
28     $PY 代码/shrink_criterion.py --sweep --jobs 4   # 加  $b$  扫描（约 15 分钟）
29
30
31 from __future__ import annotations
32
33 import argparse
34 import json
35 import math
36 import sys
37 from pathlib import Path
38
39 import numpy as np
40
41 HERE = Path(__file__).resolve().parent
42 PROJECT = HERE.parent
43 OUT = PROJECT / "输出"
44 sys.path.insert(0, str(HERE))
45
46 import solve_p234 as base                                # noqa: E402
47
48 C0 = base.C_INIT
49 R0 = base.R0
50 A3, B3 = 650.0, 128.0
51
52
53 # ===== 解析判据 =====
54 def v_ratio(C, r, c0=C0):
55     """ $V(C)/V(C_0)$ ，只含  $r$ 。"""

```

```

56     C = np.asarray(C, dtype=float)
57     return (1.0 + C) * (1.0 + r * c0) / ((1.0 + c0) * (1.0 + r * C))
58
59
60 def line_shrink(C, r, n, c0=C0):
61     """线收缩  $1 - R/R_0$ :  $n=2$  径向等长,  $n=3$  各向同性。"""
62     return 1.0 - v_ratio(C, r, c0) ** (1.0 / n)
63
64
65 def r_star(eps, Cmin, n, c0=C0):
66     """忽略收缩安全的阈值:  $r \geq r^*$  时窗口内线收缩不超过  $\varepsilon$ 。"""
67     g = (1 - eps) ** n * (1 + c0) / (1 + Cmin)
68     return (g - 1.0) / (c0 - g * Cmin)
69
70
71 def C_star(eps, r, n, c0=C0):
72     """临界含水率:  $C$  降到  $C^*$  以下, 忽略收缩的线收缩超过  $\varepsilon$ 。"""
73     G = (1 - eps) ** n * (1 + c0) / (1 + r * c0)
74     return (G - 1.0) / (1.0 - G * r)
75
76
77 def beta_of(r, a):
78     """与线性收缩模型对应的端点收缩系数。"""
79     return (1000.0 / a) * (1 - r) / (1 + r * C0)
80
81
82 # ===== 求解封装 =====
83 def _fv_weights(xi):
84     h = xi[1] - xi[0]
85     w = xi * h
86     w[0] = h * h / 8.0
87     w[-1] = h * (1.0 - h / 4.0) / 2.0
88     return w
89
90
91 def solve_with_radius(t_tab, R_tab, prop="p23", n=100, dt=5.0, tend=259200.0):
92     """把自定义的  $R(t)$  注入求解器 (贴体材料坐标, 无对流项)。"""
93     old_t, old_v = base._RAD_T, base._RAD_V
94     t_ext = np.r_[t_tab, tend]
95     R_ext = np.r_[R_tab, R_tab[-1]]
96     base._RAD_T, base._RAD_V = np.asarray(t_ext, float), np.asarray(R_ext, float)
97     try:
98         return base.solve(prop=prop, shrink=True, n=n, dt=dt, tend=tend)
99     finally:
100         base._RAD_T, base._RAD_V = old_t, old_v
101
102
103 def mean_moisture(sol):
104     """体积加权平均含水率  $\bar{C}(t)$ 。"""
105     w = _fv_weights(sol["xi"])
106     return (sol["conc"] * w).sum(axis=1) / w.sum()
107
108
109 def implied_radius(sol, a, b, n):
110     """由  $p=a+bC$  反演  $R(\bar{C})$ , 并强制单调不减 (收缩不可逆)。"""
111     Cbar = mean_moisture(sol)
112     r = b / a
113     R = R0 * v_ratio(Cbar, r) ** (1.0 / n)
114     return Cbar, np.minimum.accumulate(R)
115
116
117 def closure_solve(a, b, prop="p23", n=100, dt=5.0, iters=3, geom_n=2):
118     """自洽闭合: 固定半径起步, 反复用  $\bar{C}$  反演  $R(t)$  直到稳定。"""
119     sol = base.solve(prop=prop, shrink=False, n=n, dt=dt)
120     hist = []
121     for _ in range(iters):
122         Cbar, R = implied_radius(sol, a, b, geom_n)
123         old = sol["dry_time"]
124         sol = solve_with_radius(sol["times"], R * 1.0, prop=prop, n=n, dt=dt)
125         hist.append(sol["dry_time"])
126         if old is not None and sol["dry_time"] is not None and \
127             abs(sol["dry_time"] - old) / old < 1e-3:
128             break

```

```

129     return sol, hist
130
131
132 def parse_criterion():
133     """复算结果文件里的判据表，供正文引用。"""
134     spec = {
135         "预热窗口 C 2.55→2.0": 2.0,
136         "问题二窗口 2.55→1.364": 1.364,
137         "全程 2.55→0.15": 0.15,
138     }
139     out = {}
140     for name, Cmin in spec.items():
141         out[name] = {}
142         for e in (0.03, 0.05, 0.10):
143             out[name]["eps=%0f%" % (100 * e)] = {
144                 "r*_radial": float(r_star(e, Cmin, 2)),
145                 "r*_iso": float(r_star(e, Cmin, 3)),
146             }
147     return out
148
149
150 def make_props(key, a, b):
151     """只替换密度式  $\rho=a+bC$ ，其余物性 (c_p、k、D) 一律沿用附录 3。
152
153     这一步是扫描的关键：b 同时影响两处——热方程里的体积热容  $\rho c_p$ 。
154     以及 (经由  $r=b/a$ ) 收缩律。两者都要真的随 b 变，否则“固定几何”
155     那一系列会退化成同一个算例。
156     """
157     def f(conc, temp):
158         _rho, _drho, cp, k_cond, diff = base.props_p23(conc, temp)
159         c = np.maximum(conc, 1e-12)
160         return a + b * c, np.full_like(c, b), cp, k_cond, diff
161     base.PROPERTY_SETS[key] = f
162     return key
163
164
165 def job(kind, a, b, prop, n, dt):
166     """单个算例 (模块级，供 multiprocessing 取用)。"""
167     if kind == "fixed":
168         return base.solve(prop=prop, shrink=False, n=n, dt=dt)["dry_time"]
169     if kind == "measured":
170         return base.solve(prop=prop, shrink=True, n=n, dt=dt)["dry_time"]
171     sol, _ = closure_solve(a, b, prop=prop, n=n, dt=dt)
172     return sol["dry_time"]
173
174
175 def main() -> None:
176     ap = argparse.ArgumentParser()
177     ap.add_argument("--analytic", action="store_true")
178     ap.add_argument("--matrix", action="store_true")
179     ap.add_argument("--sweep", action="store_true")
180     ap.add_argument("--jobs", type=int, default=4)
181     ap.add_argument("--n", type=int, default=100)
182     args = ap.parse_args()
183     if not (args.analytic or args.matrix or args.sweep):
184         args.analytic = True
185
186     # 已在磁盘上的结果先读回来。避免只跑一部分时把其它结果覆盖掉
187     jf = OUT / "收缩判据与代价.json"
188     rep = json.loads(jf.read_text(encoding="utf-8")) if jf.exists() else {}
189     rep["常数"] = {"C0": C0, "R0_cm": R0 * 100, "a": A3, "b": B3,
190                  "n_grid": args.n}
191     if args.analytic or args.matrix or args.sweep:
192         rep["判据"] = parse_criterion()
193         rep["题面两式的r"] = {"附录3": B3 / A3, "附录4": 90.0 / 760.0}
194         rep["临界含水率C*"] = {
195             "附录3": {("eps=%0f%" % (100 * e)):
196                       {"iso": float(C_star(e, B3 / A3, 3)),
197                        "radial": float(C_star(e, B3 / A3, 2))}
198                       for e in (0.03, 0.05, 0.10)},
199             "附录4": {("eps=%0f%" % (100 * e)):
200                       {"iso": float(C_star(e, 90.0 / 760.0, 3)),
101                        "radial": float(C_star(e, 90.0 / 760.0, 2))}
201

```

```

202         for e in (0.03, 0.05, 0.10)},
203     }
204     rep["收缩系数"] = {"beta_附录3": float(beta_of(B3 / A3, A3)),
205                       "beta_附录4": float(beta_of(90.0 / 760.0, 760.0))}
206
207 from multiprocessing import get_context
208 ctx = get_context("fork")
209
210 if args.matrix:
211     tasks = [("fixed", A3, B3, "p23"), ("measured", A3, B3, "p23"),
212             ("implied", A3, B3, "p23")]
213     with ctx.Pool(args.jobs) as pool:
214         res = pool.starmap(job, [(k, a, b, p, args.n, 5.0)
215                                 for k, a, b, p in tasks])
216     rep["几何对照"] = {k: (None if v is None else float(v))
217                       for (k, _, _, _), v in zip(tasks, res)}
218     base_t = rep["几何对照"]["fixed"]
219     rep["几何对照相对固定半径"] = {
220         k: (None if v is None or base_t is None
221            else float(100 * (v / base_t - 1)))
222         for k, v in rep["几何对照"].items()}
223     print("几何对照: ", {k: (round(v / 3600, 2) if v else None)
224                       for k, v in rep["几何对照"].items()})
225
226 if args.sweep:
227     # 固定 a=650, 只改 b (等价于改 r); p 随 b 变, c_p/k/D 仍用附录 3
228     # 阈值 r* 落在 r 较大的一侧, 故在 b=320~650 之间加密
229     b_list = [0.0, 64.0, 96.0, 128.0, 192.0, 320.0,
230             420.0, 500.0, 560.0, 610.0, 650.0]
231     keys = {b: make_props("sweep_%.0f" % b, A3, b) for b in b_list}
232     with ctx.Pool(args.jobs) as pool:
233         tf = pool.starmap(job, [("fixed", A3, b, keys[b], args.n, 5.0)
234                                 for b in b_list])
235         ti = pool.starmap(job, [("implied", A3, b, keys[b], args.n, 5.0)
236                                 for b in b_list])
237     rows = []
238     for b, a_, c_ in zip(b_list, tf, ti):
239         rows.append({
240             "b": b, "r": b / A3,
241             "t_fixed_h": None if a_ is None else a_ / 3600.0,
242             "t_implied_h": None if c_ is None else c_ / 3600.0,
243             "delta_pct": (None if not a_ or not c_ else 100 * (a_ / c_ - 1)),
244         })
245     rep["b扫描"] = rows
246     print("b 扫描完成")
247
248 (OUT / "收缩判据与代价.json").write_text(
249     json.dumps(rep, ensure_ascii=False, indent=1), encoding="utf-8")
250 print("已写出 -> 输出/收缩判据与代价.json")
251
252
253 if __name__ == "__main__":
254     main()

```

完整程序清单（七）：批处理与结果生成

依次为 run_all.py、prepare_outputs.py、paper_tables.py、prepare_figure_data.py 与 verify_paper_numbers.py。

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """一次性求解问题二/三/四并导出结果数据（供生成 result2-4.xlsx）。
4
5  问题二与问题三使用同一套模型（附录 3 物性、无收缩），一次求解即可同时满足；
6  问题四使用附录 4 物性并计入附件 2 给出的收缩。
7
8  输出（npz）：
9      times      时间序列 s
10     conc21      固定物理位置 0,0.1,...,2.0 cm 处的水分浓度（超出当前半径处为 NaN）
11     temp21      同上，温度

```

```

12     valid21      该位置是否在药材内部 (bool)
13     conc_surf   药材表面 ( $r = R(t)$ ) 处的水分浓度
14     temp_surf   药材表面温度
15     radius       $R(t)$  m
16     dry_time    中心含水率首次低于 0.15 的时刻 s
17
18 运行:
19     PY=/Users/sqz/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3
20     $PY 代码/run_all.py --n 200 --dt 5
21     """
22
23     from __future__ import annotations
24
25     import argparse
26     import sys
27     from pathlib import Path
28
29     import numpy as np
30
31     HERE = Path(__file__).resolve().parent
32     PROJECT = HERE.parent
33     sys.path.insert(0, str(HERE))
34
35     from solve_p234 import solve # noqa: E402
36
37     POSITIONS = [round(0.1 * k, 1) for k in range(21)] # 0, 0.1, ..., 2.0 cm
38
39
40     def series(sol, positions):
41         """把贴体坐标解插值到固定物理位置, 并标出"已经不在药材内部"的点。
42
43         为什么需要这一步
44         -----
45         solve() 返回的解定义在贴体坐标  $\xi=r/R(t)$  上, 网格节点随时间跟着材料走。
46         但题目要求的输出位置是**固定的物理距离** (0, 0.1, ..., 2.0 cm),
47         所以每个时刻都要重新换算: 物理位置  $r_k$  对应的贴体坐标是  $r_k/R(t)$ 。
48
49         为什么要 valid 掩码
50         -----
51         问题四里药材半径会收缩到 1.198 cm。此后 1.3~2.0 cm 这些位置已经在
52         药材外部, 含水率**没有定义** (不是"等于某个值")。这些格子必须留空,
53         而不能填 0 或用最近邻外推——所以先算出 valid, 再只对内部点插值,
54         外部点保持 NaN, 最终在 Excel 里体现为空白单元格。
55         """
56         xi = sol["xi"]
57         n_t = len(sol["times"])
58         n_p = len(positions)
59         temp = np.full((n_t, n_p), np.nan) # 先全填 NaN, 只覆盖有效的格子
60         conc = np.full((n_t, n_p), np.nan)
61         valid = np.zeros((n_t, n_p), dtype=bool)
62         for k in range(n_t):
63             rad = sol["radius"][k]
64             target = np.asarray(positions, float) / 100.0
65             inside = target <= rad + 1e-12 # 加 1e-12 容忍浮点比较误差
66             valid[k] = inside
67             if inside.any():
68                 # 物理位置 → 贴体坐标, 再在  $\xi$  网格上线性插值
69                 t_xi = np.clip(target[inside] / rad, 0.0, 1.0)
70                 temp[k, inside] = np.interp(t_xi, xi, sol["temp"][k])
71                 conc[k, inside] = np.interp(t_xi, xi, sol["conc"][k])
72         return temp, conc, valid
73
74
75     def export(sol, tag, n, dt):
76         temp, conc, valid = series(sol, POSITIONS)
77         temp_surf = sol["temp"][:, -1]
78         conc_surf = sol["conc"][:, -1]
79         out = PROJECT / "输出" / ("%s_data.npz" % tag)
80         out.parent.mkdir(parents=True, exist_ok=True)
81         np.savez_compressed(
82             out,
83             times=sol["times"],
84             conc21=conc,

```

```

85     temp21=temp,
86     valid21=valid,
87     conc_surf=conc_surf,
88     temp_surf=temp_surf,
89     radius=sol["radius"],
90     dry_time=np.array(
91         [sol["dry_time"] if sol["dry_time"] is not None else np.nan]
92     ),
93     n=np.array([n]),
94     dt=np.array([dt]),
95 )
96 print("已导出: %s (%d 个时间点)" % (out, len(sol["times"])))
97 return out
98
99
100 def main():
101     ap = argparse.ArgumentParser()
102     ap.add_argument("--n", type=int, default=200)
103     ap.add_argument("--dt", type=float, default=5.0)
104     ap.add_argument("--only", type=int, default=0, help="仅跑指定题号")
105     args = ap.parse_args()
106
107     if args.only in (0, 2, 3):
108         # 问题二与问题三共用同一套模型(附录 3 物性、半径恒为 2 cm),
109         # 差别只在于"取到什么时候": 问题二只要 3 h 内的结果.
110         # 问题三要一直算到中心含水率低于 0.15。算一次全长即可同时满足,
111         # 无需重复求解。
112         print("=== 问题二/三: 附录 3 物性, 无收缩 ===")
113         sol23 = solve(
114             prop="p23", shrink=False, n=args.n, dt=args.dt, tend=259200.0
115         )
116         print(
117             "  烘干时间: %.0f s = %.2f h = %.3f 天"
118             % (
119                 sol23["dry_time"],
120                 sol23["dry_time"] / 3600,
121                 sol23["dry_time"] / 86400,
122             )
123         )
124         export(sol23, "result23", args.n, args.dt)
125
126     if args.only in (0, 4):
127         print("=== 问题四: 附录 4 物性, 含收缩 ===")
128         sol4 = solve(
129             prop="p4", shrink=True, n=args.n, dt=args.dt, tend=259200.0
130         )
131         print(
132             "  烘干时间: %.0f s = %.2f h = %.3f 天"
133             % (
134                 sol4["dry_time"],
135                 sol4["dry_time"] / 3600,
136                 sol4["dry_time"] / 86400,
137             )
138         )
139         export(sol4, "result4", args.n, args.dt)
140
141
142 if __name__ == "__main__":
143     main()

```

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """把问题二/三/四的解整理成 result2/3/4.xlsx 所需的中间数据。
4
5  输出到 输出/ 下的二进制(float64 小端、行主序)与元数据 JSON:
6      result2.bin : times[N2], temp[N2*21], conc[N2*21]
7      result3.bin : times[N3], conc[N3*21]
8      result4.bin : times[N4], conc[N4*21], conc_surf[N4]
9      result_meta.json
10  其中 result2 按题面要求为 1 s 间隔(由 dt 解线性插值); result3/4 为 60 s 间隔。
11  超出当前半径的位置在 result4 中写为 NaN, 生成.xlsx 时留空。
12  """
13

```

```

14 from __future__ import annotations
15
16 import json
17 from pathlib import Path
18
19 import numpy as np
20
21 PROJECT = Path(__file__).resolve().parent.parent
22 OUT = PROJECT / "输出"
23
24
25 def resample(times, values, targets):
26     """把 (n_t, n_p) 的时空解按时间线性插值到 targets 时刻。
27
28     逐列处理（而不是整体插值），是因为问题四的某些列含 NaN——
29     那些位置超出了当前半径，无定义。插值前先剔除 NaN，只在该列有效的
30     时间范围内插值，避免 NaN 污染整列。
31     """
32     out = np.empty((len(targets), values.shape[1]))
33     for j in range(values.shape[1]):
34         col = values[:, j]
35         good = ~np.isnan(col) # 该列有效的时间点
36         out[:, j] = np.interp(targets, times[good], col[good])
37     return out
38
39
40 def main():
41     # ----- 问题二 / 三 -----
42     d23 = np.load(OUT / "result23_data.npz")
43     t23 = d23["times"]
44     dry23 = float(d23["dry_time"][0])
45     # 题面要求 result2 是**1 s 间隔**，而求解时用的是 dt=5 s 的粗时间步
46     # （为了控制 61 h 的计算量）。所以这里把 5 s 解线性插值到 1 s 网格。
47     # 时间步收敛性检验表明 dt=5 s 与 2 s 的烘干时长只差 12 s，
48     # 故该插值不引入有意义的误差。
49     n2 = int(round(dry23)) # 1 s 一个点
50     t2 = np.arange(1, n2 + 1, dtype=float)
51     temp2 = resample(t23, d23["temp21"], t2)
52     conc2 = resample(t23, d23["conc21"], t2)
53     with open(OUT / "result2.bin", "wb") as fh:
54         fh.write(np.ascontiguousarray(t2).astype("<f8").tobytes())
55         fh.write(np.ascontiguousarray(temp2).astype("<f8").tobytes())
56         fh.write(np.ascontiguousarray(conc2).astype("<f8").tobytes())
57     print("result2: %d 行 x 21 列 (1 s 间隔, 插值自 %g s 解)" % (n2, t23[1]))
58
59     n3 = int(dry23 // 60)
60     t3 = np.arange(60, n3 * 60 + 1, 60, dtype=float)
61     # result3 要 60 s 间隔。因为求解时间步是 5 s, 60 一定是 5 的整数倍,
62     # 所以 60 s 网格点**恰好落在**已保存的解上, searchsorted 取到的是
63     # 精确对应的时间层, 不需要再做插值。
64     idx3 = np.searchsorted(t23, t3)
65     idx3 = np.clip(idx3, 0, len(t23) - 1)
66     conc3 = d23["conc21"][idx3]
67     with open(OUT / "result3.bin", "wb") as fh:
68         fh.write(np.ascontiguousarray(t3).astype("<f8").tobytes())
69         fh.write(np.ascontiguousarray(conc3).astype("<f8").tobytes())
70     print("result3: %d 行 x 21 列 (60 s 间隔)" % len(t3))
71
72     # ----- 问题四 -----
73     d4 = np.load(OUT / "result4_data.npz")
74     t4all = d4["times"]
75     dry4 = float(d4["dry_time"][0])
76     n4 = int(dry4 // 60)
77     t4 = np.arange(60, n4 * 60 + 1, 60, dtype=float)
78     idx4 = np.clip(np.searchsorted(t4all, t4), 0, len(t4all) - 1)
79     conc4 = d4["conc21"][idx4].copy()
80     surf4 = d4["conc_surf"][idx4].copy()
81     # 超出当前半径的位置留空
82     rad4 = d4["radius"][idx4]
83     pos_cm = np.array([round(0.1 * k, 1) for k in range(21)])
84     outside = pos_cm[None, :] > rad4[:, None] * 100.0 + 1e-9
85     conc4[outside] = np.nan
86     with open(OUT / "result4.bin", "wb") as fh:

```

```

87     fh.write(np.ascontiguousarray(t4).astype("<f8").tobytes())
88     fh.write(np.ascontiguousarray(conc4).astype("<f8").tobytes())
89     fh.write(np.ascontiguousarray(surf4).astype("<f8").tobytes())
90     print(
91         "result4: %d 行 x 21 列 + 药材表面 (60 s 间隔, 末年半径 %.4f cm) "
92         % (len(t4), rad4[-1] * 100)
93     )
94
95     meta = {
96         "positions_cm": [round(0.1 * k, 1) for k in range(21)],
97         "n2": n2,
98         "n3": len(t3),
99         "n4": len(t4),
100        "dry2_s": dry23,
101        "dry4_s": dry4,
102        "dry2_days": dry23 / 86400.0,
103        "dry4_days": dry4 / 86400.0,
104    }
105    (OUT / "result_meta.json").write_text(
106        json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8"
107    )
108    print("烘干时间: 问题三 %.0f s (%.2f 天), 问题四 %.0f s (%.2f 天)"
109          % (dry23, dry23 / 86400, dry4, dry4 / 86400))
110
111
112 if __name__ == "__main__":
113     main()

```

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """导出论文正文用的表 3、表 4、表 5、表 6 (Markdown 格式)。
4
5  输入: 输出/result23_data.npz、输出/result4_data.npz
6        (由 代码/run_all.py 生成)
7  输出: 输出/问题二三四-表格.md
8
9  四张表的采样规则 (与题面要求一致):
10     表 3 / 表 4 (问题二): 3 h 内每 0.5 h, 位置 0/0.5/1/1.5/2 cm
11     表 5 (问题三): 每 6 h 直至烘干结束, 位置同上
12     表 6 (问题四): 同表 5, 但未列改为"药材表面" (R(t) 本身在变),
13                     且超出当前半径的固定位置以"——"表示
14
15 row_at() 用 np.argmin 找最近的**已存解**时间层: 因为求解时间步是 5 s,
16 而表中时间点都是 5 s 的整数倍, 取到的是精确对应的那一层, 不会引入误差。
17 """
18
19 from __future__ import annotations
20
21 from pathlib import Path
22
23 import numpy as np
24
25 PROJECT = Path(__file__).resolve().parent.parent
26 OUT = PROJECT / "输出"
27
28
29 def row_at(times, values, t_query, pos_idx):
30     k = int(np.argmin(np.abs(times - t_query)))
31     return k, [values[k][j] for j in pos_idx]
32
33
34 def fmt(v):
35     return "——" if v is None or not np.isfinite(v) else "%.4f" % v
36
37
38 def table(lines, title, times, grid, t_list, pos_cm, extra=None, extra_t=None):
39     head = "| 时间/h | " + " | ".join(
40         p if isinstance(p, str) else "%g" % p for p in pos_cm
41     ) + " | "
42     lines.append("## " + title)
43     lines.append("")
44     lines.append(head)
45     lines.append("| --- | " + " --- | " * len(pos_cm))

```

```

46     for label, t in t_list:
47         k = int(np.argmin(np.abs(times - t)))
48         cells = []
49         for p in pos_cm:
50             if p == "药材表面":
51                 cells.append(fmt(extra[k] if extra is not None else np.nan))
52             else:
53                 j = int(round(p / 0.1))
54                 v = grid[k][j]
55                 cells.append(fmt(v if np.isfinite(v) else np.nan))
56         lines.append("| %s | " % label + " | ".join(cells) + " |")
57     lines.append("")
58
59
60 def main():
61     lines = ["# 问题二、三、四结果表", ""]
62
63     # ----- 问题二 -----
64     d = np.load(OUT / "result23_data.npz")
65     t = d["times"]
66     pos_idx = [0, 5, 10, 15, 20]
67     pos_cm = [0, 0.5, 1.0, 1.5, 2.0]
68
69     t3 = [("%.1f" % h, h * 3600) for h in (0.5, 1.0, 1.5, 2.0, 2.5, 3.0)]
70     table(lines, "表 3 3 小时内药材的温度 (单位: °C)",
71           t, d["temp21"], t3, pos_cm)
72     table(lines, "表 4 3 小时内药材的水分浓度 (单位: kg/kg)",
73           t, d["conc21"], t3, pos_cm)
74
75     # ----- 问题三 -----
76     dry3 = float(d["dry_time"][0])
77     t5 = [("%.d" % (h // 3600), float(h))
78           for h in np.arange(6 * 3600, dry3, 6 * 3600)]
79     t5.append(("烘干结束时间", dry3))
80     table(lines, "表 5 药材烘干过程的水分浓度 (单位: kg/kg)",
81           t, d["conc21"], t5, pos_cm)
82     lines.append("> 问题三烘干结束时间: %.0f s = %.2f h = %.3f 天。"
83                 % (dry3, dry3 / 3600, dry3 / 86400))
84     lines.append("")
85
86     # ----- 问题四 -----
87     e = np.load(OUT / "result4_data.npz")
88     te = e["times"]
89     dry4 = float(e["dry_time"][0])
90     t6 = [("%.d" % (h // 3600), float(h))
91           for h in np.arange(6 * 3600, dry4, 6 * 3600)]
92     t6.append(("烘干结束时间", dry4))
93     table(lines, "表 6 药材烘干过程的水分浓度 (含收缩, 单位: kg/kg)",
94           te, e["conc21"], t6, [0, 0.5, 1.0, 1.5, 2.0, "药材表面"],
95           extra=e["conc_surf"])
96     lines.append("> 问题四烘干结束时间: %.0f s = %.2f h = %.3f 天。"
97                 % (dry4, dry4 / 3600, dry4 / 86400))
98     lines.append("> 半径由 2.000 cm 收缩至 %.3f cm, 故 1.5 cm 与 2.0 cm 处的"
99                 "数据在半径收缩到该值以下后无定义, 以——表示。"
100                 % (e["radius"][-1] * 100))
101     lines.append("")
102
103     out = OUT / "问题二三四-表格.md"
104     out.write_text("\n".join(lines), encoding="utf-8")
105     print(out.read_text(encoding="utf-8"))
106
107
108 if __name__ == "__main__":
109     main()

```

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """为论文插图准备全部派生数据, 缓存到 输出/figdata/。

```

4 为什么单独一步

```

5
6 插图脚本要读的东西有一部分需要重新求解 (问题一的全细网格解、问题四的
7 "不收缩对照解"、参数灵敏度扫描)。这些算一次要几分钟, 不适合放在出图
8 循环里反复跑; 更重要的是, 把它们固化成可复现的文件后, 任何一张图都能

```

```

9 脱离求解器单独重绘。
10
11 产物 (均在 输出/figdata/ 下)
12     air.csv           附件 1 的 (t, T_inf, C_inf)
13     radius.csv       附件 2 的 (t, R)
14     p1_full.npz       问题一细网格解 + 守恒检验序列 + 指定时刻剖面
15     p4_noshrink.npz   问题四物性但半径固定 2 cm 的对照解
16     sens.csv         问题三 h / h_m / D ±20% 灵敏度
17     rho_analysis.json 物性关联式自治性检验的全部标量结果
18     props_curves.npz 附录 3 / 附录 4 的 rho, cp, k, D 随 C (与 T) 的曲线
19
20 运行:
21     PY=/Users/sqz/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3
22     $PY 代码/prepare_figure_data.py
23     $PY 代码/prepare_figure_data.py --only air,radius,rho
24
25
26 from __future__ import annotations
27
28 import argparse
29 import csv
30 import json
31 import sys
32 from pathlib import Path
33
34 import numpy as np
35 import openpyxl
36
37 HERE = Path(__file__).resolve().parent
38 PROJECT = HERE.parent
39 OUT = PROJECT / "输出"
40 FIGDATA = OUT / "figdata"
41 sys.path.insert(0, str(HERE))
42
43 RHO_W = 1000.0      # 纯水真密度 kg/m^3
44 RHO_S = 1400.0      # 干物质真密度 kg/m^3 (PDF 取值, 见 $附注)
45 C0 = 2.55          # 初始干基含水率
46
47
48 def _read_attachment(path):
49     wb = openpyxl.load_workbook(path, data_only=True)
50     rows = [r for r in wb.active.iter_rows(min_row=2, values_only=True)
51             if r[0] is not None]
52     return np.array(rows, dtype=float)
53
54
55 # ===== 附件数据
56 def build_air():
57     d = _read_attachment(PROJECT / "附件" / "附件1-烘房温度与水分浓度.xlsx")
58     np.savetxt(FIGDATA / "air.csv", d[:, :3], delimiter=",",
59               header="t_s,T_inf,C,C_inf_kgkg", comments="", fmt="%.6f")
60     print("air.csv" % d 行" % len(d))
61     return d
62
63
64 def build_radius():
65     d = _read_attachment(PROJECT / "附件" / "附件2-药材半径.xlsx")
66     np.savetxt(FIGDATA / "radius.csv", d[:, :2], delimiter=",",
67               header="t_s,R_cm", comments="", fmt="%.6f")
68     # 再导出一份"插值+解析导数"的密网格版本。用求解器同一份 PCHIP 实现,
69     # 保证图上画出来的 R(t)、dR/dt 与问题四方程里用的完全同源。
70     import solve_p234
71
72     pch = solve_p234.Pchip(d[:, 0], d[:, 1] / 100.0)
73     tt = np.linspace(d[0, 0], d[-1, 0], 2000)
74     rr = pch(tt)
75     dr = pch.deriv(tt)
76     np.savetxt(FIGDATA / "radius_pchip.csv",
77               np.column_stack([tt, rr * 100.0, dr * 100.0]), delimiter=",",
78               header="t_s,R_cm,dRdt_cm_per_s", comments="", fmt="%.8f")
79     print("radius.csv" % d 行, R: %.4f -> %.4f cm"
80           % (len(d), d[0, 1], d[-1, 1]))
81     return d

```

```

82
83
84 # ===== 问题一
85 def build_p1():
86     """重解问题一 (N=400, dt=1 s) 并把守恒检验序列一并缓存。
87
88     结果与 输出/result1_data.npz 逐位一致 (已验证), 此处额外保存
89     Q(t)=∫C dV、表面累计通量与细网格末态剖面, 供守恒检验图使用。
90     """
91     import solve_p1
92
93     res = solve_p1.solve()
94     rep = solve_p1.conservation_report(res)
95     snap_idx = [int(round(t / res["dt"])) for t in solve_p1.REPORT_TIMES]
96     np.savez_compressed(
97         FIGDATA / "p1_full.npz",
98         times=res["times"],
99         r_out_cm=res["r_out_cm"],
100         temp_out=res["temp_out"],
101         conc_out=res["conc_out"],
102         q_volume=res["c_volume"],
103         phys_volume=res["phys_volume"],
104         surf_flux=res["surf_flux"],
105         snap_idx=np.array(snap_idx),
106         r_fine_cm=np.arange(len(res["final_temp"])) * res["hx"] * 100.0,
107         temp_fine=res["final_temp"],
108         conc_fine=res["final_conc"],
109         hx=np.array([res["hx"]]),
110         dt=np.array([res["dt"]]),
111         residual=np.array([rep["relative_residual"]]),
112         # 两个特征量随解一起存下来, 绘图脚本就不必再 import 求解器
113         # (求解器依赖 openpyxl, 而绘图虚拟环境里没有)。
114         alpha=np.array([solve_p1.ALPHA]),
115         d_c0=np.array([float(solve_p1.diffusion_coefficient(solve_p1.C_INIT))]),
116         bi=np.array([solve_p1.H_HEAT * solve_p1.RADIUS / solve_p1.K_COND]),
117         bi_m=np.array([solve_p1.H_MASS * solve_p1.RADIUS
118             / float(solve_p1.diffusion_coefficient(solve_p1.C_INIT))]),
119     )
120     print("p1_full.npz 守恒残差 %.3e" % rep["relative_residual"])
121
122
123 # ===== 问题四对照
124 def build_p4_noshrink():
125     """问题四物性 + 固定半径 2 cm 的对照解 (论文中 5.41 天的来源)。
126
127     时间步取 dt=5 s, 与论文正文其余计算口径完全一致 (正文给出的
128     467260 s 就是这一组设定下的结果), 避免图文出现两套数字。
129     """
130     import solve_p234
131
132     dt = 5.0
133     tend = 900000.0
134     sol = solve_p234.solve(prop="p4", shrink=False, n=200, dt=dt, tend=tend)
135     pos = np.array([0.0, 0.5, 1.0, 1.5, 2.0]) / 100.0
136     conc = np.empty((len(sol["times"]), len(pos)))
137     temp = np.empty_like(conc)
138     for k in range(len(sol["times"])):
139         conc[k] = np.interp(pos / solve_p234.R0, sol["xi"], sol["conc"][k])
140         temp[k] = np.interp(pos / solve_p234.R0, sol["xi"], sol["temp"][k])
141     dry = sol["dry_time"]
142     np.savez_compressed(
143         FIGDATA / "p4_noshrink.npz",
144         times=sol["times"], conc=conc, temp=temp, xi=sol["xi"],
145         conc_all=sol["conc"],
146         dry_time=np.array([np.nan if dry is None else dry]),
147         dt=np.array([dt]),
148     )
149     print("p4_noshrink.npz 烘干时间 %s s"
150         % ("未达标" if dry is None else "%.0f" % dry))
151
152
153 # ===== 灵敏度
154 def build_sens():

```

```

155 import sens
156
157 cases = [("h", -0.2), ("h", +0.2), ("hm", -0.2), ("hm", +0.2),
158          ("D", -0.2), ("D", +0.2)]
159 rows = [{"param", "scale", "t_center_s", "t_center_h"}]
160 base = sens.run()
161 rows.append(["base", "1.00", "%.1f" % base, "%.4f" % (base / 3600.0)])
162 for name, dv in cases:
163     kw = {"h": "hscale", "hm": "hmscale", "D": "dscale"}[name]
164     t = sens.run(**{kw: 1.0 + dv})
165     rows.append([name, "%.2f" % (1.0 + dv), "%.1f" % t,
166                 "%.4f" % (t / 3600.0)])
167     print("  sens %-3s %+5.0f%% : %9.1f s = %7.4f h (%+.2f%%)"
168           % (name, dv * 100, t, t / 3600.0, 100 * (t - base) / base))
169 with open(FIGDATA / "sens.csv", "w", newline="") as fh:
170     csv.writer(fh).writerows(rows)
171 print("sens.csv 基准 %.1f s" % base)
172
173
174 # ===== 物性曲线
175 def build_props():
176     import solve_p234
177
178     c = np.linspace(0.0, 2.55, 400)
179     t_k = np.full_like(c, 323.15)
180     rho3, _, cp3, k3, d3 = solve_p234.props_p23(c, t_k - 273.15)
181     rho4, _, cp4, k4, d4 = solve_p234.props_p4(c, t_k - 273.15)
182     temps = np.linspace(28.0, 50.0, 120)
183     cs = np.array([2.55, 1.50, 0.60, 0.15])
184     dd3 = np.array([2.4e-3 * np.exp(-0.45 / cc)
185                    * np.exp(-3850.0 / (temps + 273.15)) for cc in cs])
186     dd4 = np.array([4.2e-4 * np.exp(-0.30 / cc)
187                    * np.exp(-3850.0 / (temps + 273.15)) for cc in cs])
188     np.savez_compressed(
189         FIGDATA / "props_curves.npz",
190         c=c, rho3=rho3, cp3=cp3, k3=k3, d3=d3,
191         rho4=rho4, cp4=cp4, k4=k4, d4=d4,
192         temps=temps, cs=cs, dd3=dd3, dd4=dd4,
193     )
194     print("props_curves.npz rho3: %.1f->%.1f, rho4: %.1f->%.1f"
195           % (rho3[0], rho3[-1], rho4[0], rho4[-1]))
196
197
198 # ===== rho 辨析
199 def build_rho():
200     """复现《干基含水率-体密度经验关系式的自洽性检验》一文的全部数值。
201
202     三类结果：
203     (1) 收缩系数 beta 的端点反演与最小二乘反演；
204     (2) 三相体积分数（水/固/气）的非负性检验；
205     (3) 由经验式推算的几何收缩（体积比、线收缩比、半径）。
206     全部由 a+bC、C0、rho_w、rho_s 直接算出，可完全复现。
207     """
208     out = {"C0": C0, "rho_w": RHO_W, "rho_s": RHO_S, "cases": {}}
209     cg = np.linspace(0.0, C0, 2001)
210
211     for name, (a, b, cp_form, k_form) in {
212         "appendix3": (650.0, 128.0, (1450.0, 2736.0), (0.21, 0.38)),
213         "appendix4": (760.0, 90.0, (1850.0, 2150.0), (0.12, 0.20)),
214     }.items():
215         rho_end = a + b * C0
216         beta_ep = (RHO_W / C0) * ((1 + C0) / rho_end - 1.0 / a)
217         v_ratio = 1.0 / (1.0 + beta_ep * a * C0 / RHO_W)
218         l_ratio = v_ratio ** (1.0 / 3.0)
219         best, best_res = None, np.inf
220         for beta in np.linspace(0.5, 1.4, 90001):
221             model = a * (1 + cg) / (1 + beta * a * cg / RHO_W)
222             r_ = float(np.sqrt(np.mean((model - (a + b * cg)) ** 2)))
223             if r_ < best_res:
224                 best, best_res = beta, r_
225         model = a * (1 + cg) / (1 + best * a * cg / RHO_W)
226         lin = a + b * cg
227         rel_dev = float(np.max(np.abs(model - lin) / lin))

```

```

228     rho_lo = a * (1 + C0)
229     rho_hi = a * (1 + C0) / (1 + a * C0 / RHO_W)
230     b_lo = (rho_lo - a) / C0
231     b_hi = (rho_hi - a) / C0
232     phases = {}
233     for tag, cc in (("C0", C0), ("dry", 0.0)):
234         rr = a + b * cc
235         w = cc / (1 + cc)
236         e_w = rr * w / RHO_W
237         e_s = rr * (1 - w) / RHO_S
238         phases[tag] = {"rho": rr, "eps_w": e_w, "eps_s": e_s,
239                       "eps_a": 1 - e_w - e_s}
240     # 几何收缩: 单位干物质体积  $V(C) = (1+C)/\rho(C)$ , 以初始态  $V(C0)$  为基准。
241     # 注意这里**不再**出现 beta——它是经验式本身的直接推论, 正是"rho"的
242     # 线性拟合式已内含收缩"这句话的定量表达。
243     v_at_C0 = (1 + C0) / (a + b * C0)
244     geo = {}
245     for cc in (2.00, 1.00, 0.50, 0.15):
246         ratio = ((1 + cc) / (a + b * cc)) / v_at_C0
247         geo["%.2f" % cc] = {"V_ratio": ratio,
248                             "L_ratio": ratio ** (1.0 / 3.0),
249                             "R_cm": 2.0 * ratio ** (1.0 / 3.0)}
250     out["cases"][name] = {
251         "a": a, "b": b,
252         "rho_end": rho_end,
253         "V_ratio": v_ratio, "shrink_vol_pct": 100 * (1 - v_ratio),
254         "L_ratio": l_ratio,
255         "beta_endpoint": beta_ep, "beta_lsq": best,
256         "rmse": best_res, "max_rel_dev": rel_dev,
257         "b_allowed": [b_lo, b_hi],
258         "phases": phases,
259         "geo": geo,
260         "cp_w": cp_form[0] + cp_form[1],
261         "k_w": k_form[0] + k_form[1],
262         "rho_d_C0": rho_end / (1 + C0),
263         "rho_d_0": a,
264     }
265     out["naive_rho_C0"] = 650.0 * (1 + C0)
266     (FIGDATA / "rho_analysis.json").write_text(
267         json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
268     c3 = out["cases"]["appendix3"]
269     print("rho_analysis.json  beta: 附录3 %.3f/%.3f, 附录4 %.3f/%.3f"
270           % (c3["beta_endpoint"], c3["beta_lsq"],
271             out["cases"]["appendix4"]["beta_endpoint"],
272             out["cases"]["appendix4"]["beta_lsq"]))
273
274
275     # ===== 判据阈值扫描
276     def build_threshold():
277         """把中心含水率一直算到 0.10, 用于"判据阈值 → 烘干时长"的敏感性图。
278
279         主结果只算到中心低于 0.15 就停(题面判据), 因此拿不到 0.15 以下的曲线。
280         这里把判据临时放宽到 0.10 多算一段。因为中心含水率沿时间单调下降。
281         "阈值为 C* 的烘干时长"就等于中心曲线首次穿过 C* 的时刻, 一次求解即可
282         给出任意阈值的结果, 无需对每个阈值重解。
283         """
284         import solve_p234
285
286         original = solve_p234.C_DRY
287         solve_p234.C_DRY = 0.10
288         try:
289             sol = solve_p234.solve(prop="p23", shrink=False, n=100, dt=5.0,
290                                   tend=330000.0)
291         finally:
292             solve_p234.C_DRY = original
293         np.savetxt(FIGDATA / "threshold.csv",
294                   np.column_stack([sol["times"], sol["conc"][:, 0]]), delimiter=",",
295                   header="t_s,C_center", comments="", fmt="%.6f")
296         print("threshold.csv 算到 %.0f s, 中心含水率末值 %.4f"
297               % (sol["times"][-1], sol["conc"][-1, 0]))
298
299
300     BUILDERS = {

```

```

301     "air": build_air,
302     "radius": build_radius,
303     "p1": build_p1,
304     "p4ns": build_p4_noshrink,
305     "sens": build_sens,
306     "props": build_props,
307     "rho": build_rho,
308     "thr": build_threshold,
309 }
310
311
312 def main():
313     ap = argparse.ArgumentParser(description="论文插图派生数据准备")
314     ap.add_argument("--only", default=None,
315                     help="逗号分隔的子集: " + ",".join(BUILDERS))
316     args = ap.parse_args()
317     FIGDATA.mkdir(parents=True, exist_ok=True)
318     keys = list(BUILDERS) if not args.only else [
319         k.strip() for k in args.only.split(",") if k.strip()]
320     for k in keys:
321         if k not in BUILDERS:
322             raise SystemExit("未知步骤: %s" % k)
323         print("== %s" % k)
324         BUILDERS[k]()
325     print("\n完成, 输出目录: %s" % FIGDATA)
326
327
328 if __name__ == "__main__":
329     main()

```

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """论文数据审计: 把论文里每个可核验的数字重算一遍, 逐条与正文比对。
4
5  为什么要这个脚本
6  -----
7  论文里的表有的来自 `输出/` 下的产物(可直接核验), 有的来自早期的临时计算
8  (没有留下脚本)。后者一旦与结果文件脱节, 就无法复核。本脚本把这些数字
9  **全部重算一次**, 并列出"论文值 vs 复算值"的对照表, 任何一条不一致都会被
10  标出来。
11
12  口径约定(重要, 正文必须与之一致)
13  -----
14  * 干物质质量检验: 在**固定物理位置网格** (0, 0.1, ..., 2.0 cm) 上积分
15     $\int_0^R \rho(C)/(1+C) r dr$ , 积分上限取**当前半径 R(t)**;
16    超出 R(t) 的位置不参与积分(该处物料已不存在)。
17  * 问题一网格收敛: 表面含水率取  $t = 1800$  s、 $r = R$  处的节点值。
18  * 空气条件的平滑/插值变体: 先把附件 1 的序列变成目标形式, 再经
19    np.interp 线性取用(对 1 s 密网格而言等价于该插值本身)。
20
21  跑法
22  -----
23  PY=/Users/sqz/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3
24  $PY 代码/verify_paper_numbers.py # 只跑轻量项(秒级)
25  $PY 代码/verify_paper_numbers.py --heavy # 追加界面取法等重解项(十分钟级)
26  $PY 代码/verify_paper_numbers.py --json 输出/论文数据审计.json
27  """
28
29  from __future__ import annotations
30
31  import argparse
32  import csv
33  import json
34  import math
35  import re
36  import sys
37  from pathlib import Path
38
39  import numpy as np
40
41  HERE = Path(__file__).resolve().parent
42  PROJECT = HERE.parent
43  OUT = PROJECT / "输出"

```

```

44 sys.path.insert(0, str(HERE))
45
46 import solve_p1 as S1 # noqa: E402
47
48 C0, RHO_W = 2.55, 1000.0
49 RAD = 0.02
50 L_LEN = 0.25
51
52 RECORDS: list[dict] = []
53
54
55 def rec(group: str, item: str, paper, calc, note: str = "", tol: float = 0.0):
56     """登记一条审计记录; paper/calc 均为字符串或数值。"""
57     ok = None
58     try:
59         ok = abs(float(paper) - float(calc)) <= tol
60     except (TypeError, ValueError):
61         ok = str(paper) == str(calc)
62     RECORDS.append({"组": group, "项目": item, "论文值": paper,
63                    "复算值": calc, "一致": ok, "备注": note})
64     return ok
65
66
67 def close(paper, calc, rel=2e-3):
68     """相对容差比较 (默认 0.2%, 覆盖四舍五入)。"""
69     try:
70         p, c = float(paper), float(calc)
71     except (TypeError, ValueError):
72         return str(paper) == str(calc)
73     if p == c:
74         return True
75     return abs(p - c) <= rel * max(abs(p), abs(c), 1e-12)
76
77
78 # -----
79 # 1. 直接读产物即可核验的项
80 # -----
81 def audit_artifacts() -> None:
82     meta = json.loads((OUT / "result_meta.json").read_text(encoding="utf-8"))
83     d23 = np.load(OUT / "result23_data.npz", allow_pickle=True)
84     d4 = np.load(OUT / "result4_data.npz", allow_pickle=True)
85     d1 = np.load(OUT / "result1_data.npz", allow_pickle=True)
86
87     # 烘干时长 (论文核心结论)
88     t3 = float(d23["dry_time"][0])
89     t4 = float(d4["dry_time"][0])
90     rec("结果", "问题三烘干时长 / s", 206700, round(t3), tol=1)
91     rec("结果", "问题三烘干时长 / h", 57.42, round(t3 / 3600, 2), tol=0.01)
92     rec("结果", "问题四烘干时长 / s", 183885, round(t4), tol=1)
93     rec("结果", "问题四烘干时长 / h", 51.08, round(t4 / 3600, 2), tol=0.01)
94     rec("结果", "问题四相对问题三缩短 / %", 11.0,
95         round(100 * (t3 - t4) / t3, 1), tol=0.1)
96
97     # 结果文件规格 (表 tab:files)
98     x3 = _xlsx_rows(OUT / "result3.xlsx")
99     x4 = _xlsx_rows(OUT / "result4.xlsx")
100     rec("结果文件", "result3 行数 (不含表头)", 3445, x3[0], tol=0)
101     rec("结果文件", "result3 末时刻 / s", 206700, x3[1], tol=1)
102     # 表中"论文值"一律填正文实际写的数, 便于直接暴露不一致
103     rec("结果文件", "result4 行数 (不含表头)", 3064, x4[0], tol=0)
104     rec("结果文件", "result4 末时刻 / s", 183840, x4[1], tol=1)
105     rec("结果文件", "result2 行数 (不含表头)", 206700, meta["n2"], tol=0)
106
107     # 问题一口径: t=1800 s 中心与表面
108     c1 = d1["conc_out"][1800]
109     t1 = d1["temp_out"][1800]
110     rec("问题一", "1800 s 中心含水率", 2.5500, round(float(c1[0]), 4), tol=5e-5)
111     rec("问题一", "1800 s 表面含水率", 1.5103, round(float(c1[-1]), 4), tol=5e-5)
112     rec("问题一", "1800 s 中心温度 / °C", 33.5765, round(float(t1[0]), 4), tol=5e-5)
113     rec("问题一", "1800 s 表面温度 / °C", 36.7863, round(float(t1[-1]), 4), tol=5e-5)
114
115     # 问题二 3 h 的报数点
116     dt23 = float(d23["dt"][0])

```

```

117 i3 = int(round(10800.0 / dt23))
118 c2 = d23["conc21"][i3]
119 rec("问题二", "3 h 中心含水率", 1.7663, round(float(c2[0]), 4), tol=5e-5)
120 rec("问题二", "3 h 表面含水率", 1.0082, round(float(c2[-1]), 4), tol=5e-5)
121 rec("问题二", "0.5 h 表面含水率", 1.6491,
122     round(float(d23["conc21"][int(round(1800.0 / dt23))][-1]), 4), tol=5e-5)
123
124 # 网格 / 时间步 / 口径三因素对照 (CSV 为唯一来源)
125 rows = list(csv.DictReader((OUT / "问题三-网格与口径对照.csv").open(encoding="utf-8")))
126 tab = {(r["form"], int(r["N"])), float(r["dt_s"])): float(r["t_center_s"])}
127     for r in rows}
128 for N, fick_paper, cons_paper in ((20, 203335, 216215), (50, 205625, 218575),
129                                   (100, 206365, 219340), (200, 206700, 219685),
130                                   (400, 206840, 219830)):
131     rec("网格对照", f"fick N={N} / s", fick_paper, tab[("fick", N, 5.0)], tol=1)
132     rec("网格对照", f"变密度 N={N} / s", cons_paper, tab[("cons", N, 5.0)], tol=1)
133 for dt_s, fick_paper, cons_paper in ((2, 206354, 219328), (5, 206365, 219340),
134                                       (10, 206380, 219360), (20, 206420, 219400),
135                                       (60, 206580, 219540), (120, 206760, 219720)):
136     rec("时间步对照", f"fick Δt={dt_s:g} / s", fick_paper, tab[("fick", 100, dt_s)], tol=1)
137     rec("时间步对照", f"变密度 Δt={dt_s:g} / s", cons_paper, tab[("cons", 100, dt_s)], tol=1)
138
139 # 第三种口径: 干基密度守恒 (论文口径表第三行)。
140 # 这张表由 closure_rhod.py 生成: 此前论文里的 51.39 h 没有实现可核验,
141 # 因此单独登记, 避免再出现"文中有数、代码无源"。
142 clos3 = list(csv.DictReader((OUT / "问题三-三种口径对照.csv").open(encoding="utf-8")))
143 c3 = {(r["form"], int(r["N"])): (float(r["t_dry_s"]), float(r["t_dry_h"]),
144                                     float(r["relative_to_fick_pct"]))}
145     for r in clos3}
146 for form, N, p_s, p_h, p_pct in (
147     ("fick", 100, 206365, 57.32, 0.00),
148     ("cons", 100, 219340, 60.93, +6.29),
149     ("rhod", 100, 174195, 48.39, -15.59),
150     ("fick", 200, 206700, 57.42, 0.00),
151     ("cons", 200, 219685, 61.02, +6.28),
152     ("rhod", 200, 174490, 48.47, -15.58)):
153     c3_s, c3_h, c3_pct = c3[(form, N)]
154     rec("三种口径", f"{form} N={N} / s", p_s, c3_s, tol=1)
155     rec("三种口径", f"{form} N={N} / h", p_h, c3_h, tol=0.006)
156     rec("三种口径", f"{form} N={N} 相对 %", p_pct, round(c3_pct, 2), tol=0.011)
157
158 # 参数灵敏度
159 sens = list(csv.DictReader((OUT / "figdata" / "sens.csv").open(encoding="utf-8")))
160 smap = {(r["param"], float(r["scale"])): float(r["t_center_s"]) for r in sens}
161 rec("灵敏度", "基准 / s", 206365, smap[("base", 1.0)], tol=1)
162 rec("灵敏度", "h-20% / s", 206435, smap[("h", 0.8)], tol=1)
163 rec("灵敏度", "h+20% / s", 206320, smap[("h", 1.2)], tol=1)
164 rec("灵敏度", "hm-20% / s", 212560, smap[("hm", 0.8)], tol=1)
165 rec("灵敏度", "hm+20% / s", 202555, smap[("hm", 1.2)], tol=1)
166 rec("灵敏度", "D-20% / s", 252070, smap[("D", 0.8)], tol=1)
167 rec("灵敏度", "D+20% / s", 176175, smap[("D", 1.2)], tol=1)
168
169 # §6 的反演值
170 ra = json.loads((OUT / "figdata" / "rho_analysis.json").read_text(encoding="utf-8"))
171 a3, a4 = ra["cases"]["appendix3"], ra["cases"]["appendix4"]
172 rec("物性反演", "附录3 端点 β", 0.822, round(a3["beta_endpoint"], 3), tol=1e-3)
173 rec("物性反演", "附录4 端点 β", 0.891, round(a4["beta_endpoint"], 3), tol=1e-3)
174 rec("物性反演", "附录3 V(0)/V(C0)", 0.423, round(a3["V_ratio"], 3), tol=1e-3)
175 rec("物性反演", "附录4 V(0)/V(C0)", 0.367, round(a4["V_ratio"], 3), tol=1e-3)
176
177
178 def _xlsx_rows(path: Path):
179     """返回 (数据行数, 末行时间): 只读第一列, 避免把整表读进内存。"""
180     from openpyxl import load_workbook
181     wb = load_workbook(path, read_only=True)
182     ws = wb.worksheets[0]
183     n, last = 0, None
184     for row in ws.iter_rows(min_col=1, max_col=1, values_only=True):
185         v = row[0]
186         if v is None or isinstance(v, str):
187             continue
188         n += 1

```

```

189     last = v
190     wb.close()
191     return n, last
192
193
194 # -----
195 # 2. 附件 1 的噪声诊断
196 # -----
197 def audit_noise() -> None:
198     # 与插图 d02_air_diag 完全同一口径: 高斯加权局部二次趋势 (w=15, p=2)。
199     # 换用线性去趋势、滑动平均或多项式拟合都复现不出表中的自相关 -0.31,
200     # 只有这一口径能同时对上  $\sigma$ 、最大残差与自相关, 说明正文与插图同源。
201     t, T, Cw = S1.load_air_conditions()
202
203     def gwsMOOTH(y, w=15, p=2):
204         n = len(y); half = w // 2; out = np.empty(n)
205         for i in range(n):
206             lo, hi = max(0, i - half), min(n, i + half + 1)
207             idx = np.arange(lo, hi); off = idx - i
208             A = np.vander(off, p + 1)
209             wg = np.exp(-(off / (half * 0.8)) ** 2)
210             c, *_ = np.linalg.lstsq(A * wg[:, None], y[idx] * wg, rcond=None)
211             out[i] = c[-1]
212         return out
213
214     for name, y, sigma_paper, max_paper, acf_paper, frac_paper in (
215         ("烘房温度", T, 0.116, 0.27, -0.31, 0.52),
216         ("水分浓度", Cw, 1.1e-4, 2.8e-4, -0.40, 0.38)):
217         res = y - gwsMOOTH(y)
218         sig = float(np.std(res, ddof=1))
219         mx = float(np.max(np.abs(res)))
220         acf1 = float(np.corrcoef(res[:-1], res[1:])[0, 1])
221         frac = 100.0 * sig / (y.max() - y.min())
222         rec("附件1噪声", f"{name} 残差标准差", sigma_paper, f"{sig:.3g}", tol=0.06 * sigma_paper)
223         rec("附件1噪声", f"{name} 最大残差", max_paper, f"{mx:.3g}", tol=0.05 * max_paper)
224         rec("附件1噪声", f"{name} 滞后1自相关", acf_paper, round(acf1, 2), tol=0.02)
225         rec("附件1噪声", f"{name} 占全变幅 / %", frac_paper, round(frac, 2), tol=0.05)
226
227
228 # -----
229 # 3. 问题一: 网格收敛、平滑方式、插值方式
230 # -----
231 def _p1_solve(n_fine=400, air=None, dt=1.0):
232     orig = S1.load_air_conditions
233     if air is not None:
234         S1.load_air_conditions = lambda path=S1.ATTACHMENT1: air
235     try:
236         return S1.solve(n_fine=n_fine, dt=dt)
237     finally:
238         S1.load_air_conditions = orig
239
240
241 def audit_p1_grid() -> None:
242     base = None
243     vals = {}
244     for n in (20, 40, 100, 400):
245         res = _p1_solve(n_fine=n)
246         vals[n] = float(res["conc_out"][-1][-1]) # t=1800 s、r=R
247     for n, paper in ((20, 1.51211), (40, 1.51076), (100, 1.51039), (400, 1.51033)):
248         rec("问题一网格", f"N={n} 表面含水率", paper, round(vals[n], 5), tol=5e-5)
249
250
251 def audit_p1_smooth() -> None:
252     t, T, Cw = S1.load_air_conditions()
253     base = _p1_solve(n_fine=400)
254     ref_c = base["conc_out"][1:] # 全程 (t>0), 不只末时刻
255     ref_t = base["temp_out"][1:]
256
257     def movavg(y, k):
258         pad = k // 2
259         ypad = np.r_[np.full(pad, y[0]), y, np.full(pad, y[-1])]
260         ker = np.ones(k) / k
261         return np.convolve(ypad, ker, mode="valid")

```

```

262
263 def savgol(y, k):
264     # 二次多项式最小二乘平滑 (与 scipy.signal.savgol_filter 同定义)
265     half = k // 2
266     x = np.arange(-half, half + 1)
267     A = np.vstack([np.ones_like(x), x, x * x]).T
268     # 平滑系数 = 第一行 of pinv(A)
269     w = np.linalg.pinv(A)[0]
270     ypad = np.r_[np.full(half, y[0]), y, np.full(half, y[-1])]
271     return np.convolve(ypad, w[::-1], mode="valid")
272
273 variants = (("3 点滑动平均 (180 s)", movavg(T, 3), movavg(Cw, 3)),
274             ("5 点 Savitzky-Golay (300 s)", savgol(T, 5), savgol(Cw, 5)),
275             ("15 点 Savitzky-Golay (900 s)", savgol(T, 15), savgol(Cw, 15)))
276 paper_vals = (("3 点滑动平均 (180 s)", 7.1e-6, 0.022),
277              ("5 点 Savitzky-Golay (300 s)", 5.9e-6, 0.014),
278              ("15 点 Savitzky-Golay (900 s)", 2.1e-5, 0.055))
279 for name, Tsm, Csm in variants:
280     res = _p1_solve(n_fine=400, air=(t, Tsm, Csm))
281     dC = float(np.max(np.abs(res["conc_out"][1:] - ref_c)))
282     dT = float(np.max(np.abs(res["temp_out"][1:] - ref_t)))
283     for pname, pC, pT in paper_vals:
284         if pname == name:
285             rec("问题一平滑", f"{name} 含水率偏差", pC, f"{dC:.3g}", tol=0.5 * pC)
286             rec("问题一平滑", f"{name} 温度偏差", pT, round(dT, 4), tol=0.005)
287
288
289 def audit_p1_interp() -> None:
290     t, T, Cw = S1.load_air_conditions()
291     tf = np.arange(t[0], t[-1] + 1.0, 1.0)
292
293 def pchip(x, y, xq):
294     # Fritsch-Carlson 保单调三次 Hermite
295     h = np.diff(x)
296     d = np.diff(y) / h
297     m = np.zeros_like(y)
298     m[0], m[-1] = d[0], d[-1]
299     for i in range(1, len(y) - 1):
300         if d[i - 1] * d[i] <= 0:
301             m[i] = 0.0
302         else:
303             w1, w2 = 2 * h[i] + h[i - 1], h[i] + 2 * h[i - 1]
304             m[i] = (w1 + w2) / (w1 / d[i - 1] + w2 / d[i])
305     idx = np.clip(np.searchsorted(x, xq) - 1, 0, len(x) - 2)
306     hh = x[idx + 1] - x[idx]
307     s = (xq - x[idx]) / hh
308     h00 = 2 * s ** 3 - 3 * s ** 2 + 1
309     h10 = s ** 3 - 2 * s ** 2 + s
310     h01 = -2 * s ** 3 + 3 * s ** 2
311     h11 = s ** 3 - s ** 2
312     return h00 * y[idx] + h10 * hh * m[idx] + h01 * y[idx + 1] + h11 * hh * m[idx + 1]
313
314 def natural_cubic(x, y, xq):
315     n = len(x)
316     A = np.zeros((n, n)); rhs = np.zeros(n)
317     A[0, 0] = A[-1, -1] = 1.0
318     for i in range(1, n - 1):
319         A[i, i - 1] = x[i] - x[i - 1]
320         A[i, i] = 2 * (x[i + 1] - x[i - 1])
321         A[i, i + 1] = x[i + 1] - x[i]
322         rhs[i] = 6 * ((y[i + 1] - y[i]) / (x[i + 1] - x[i])
323                     - (y[i] - y[i - 1]) / (x[i] - x[i - 1])))
324     m = np.linalg.solve(A, rhs)
325     idx = np.clip(np.searchsorted(x, xq) - 1, 0, n - 2)
326     hh = x[idx + 1] - x[idx]
327     a = (x[idx + 1] - xq) / hh
328     b = (xq - x[idx]) / hh
329     return (a * y[idx] + b * y[idx + 1]
330           + ((a ** 3 - a) * m[idx] + (b ** 3 - b) * m[idx + 1]) * hh ** 2 / 6.0)
331
332 base = _p1_solve(n_fine=400)
333 for name, f in (("PCHIP", pchip), ("自然三次样条", natural_cubic)):
334     Ts = f(t, T, tf); Cs = f(t, Cw, tf)

```

```

335     # "边界偏差"= 插值序列相对分段线性的最大偏离, 即边界条件被改写多少
336     Tlin = np.interp(tf, t, T)
337     Clin = np.interp(tf, t, Cw)
338     d_edge = float(np.max(np.abs(Ts - Tlin)))
339     d_edge_c = float(np.max(np.abs(Cs - Clin)))
340     # 过冲: 插值序列超出原序列相邻两点区间的最大量
341     lo = np.minimum(T[:-1], T[1:]); hi = np.maximum(T[:-1], T[1:])
342     idx = np.searchsorted(tf, t)
343     seg = np.array([Ts[idx[i]:idx[i + 1] + 1].max() for i in range(len(t) - 1)])
344     over = float(np.max(np.maximum(seg - hi, 0.0)))
345     res = _p1_solve(n_fine=400, air=(tf, Ts, Cs))
346     dT = float(np.max(np.abs(res["temp_out"][1:] - base["temp_out"][1:])))
347     dC = float(np.max(np.abs(res["conc_out"][1:] - base["conc_out"][1:])))
348     rec("问题一插值", f"{name} 边界偏差 / °C", 0.049 if name == "PCHIP" else 0.097,
349         round(d_edge, 4), tol=0.02)
350     rec("问题一插值", f"{name} 过冲 / °C", 0.0 if name == "PCHIP" else 0.080,
351         round(over, 4), tol=0.01)
352     rec("问题一插值", f"{name} 温度偏差 / °C",
353         0.0040 if name == "PCHIP" else 0.0054, round(dT, 4), tol=0.002)
354     rec("问题一插值", f"{name} 含水率偏差",
355         1.9e-6 if name == "PCHIP" else 2.9e-6, f"{dC:.2g}", tol=1e-6)
356
357     # -----
358     # 4. 干物质守恒检验
359     # -----
360
361     def audit_drymass() -> None:
362         x_cm = np.arange(21) * 0.1
363         rho3 = lambda c: 650.0 + 128.0 * c
364         rho4 = lambda c: 760.0 + 90.0 * c
365
366         def md(row, R_cm, rho, valid, surf=None):
367             """2πL∫₀^R [ρ(C)/(1+C)] r dr, 积分上限取当前半径 R(t)。
368
369             若最后一个网格节点未落在 R(t) 上(收缩时总是如此), 把表面值接到
370             半径 R(t) 处再积分, 避免把最外侧的干壳整段丢掉而高估质量亏损。
371             """
372             idx = [k for k in range(21) if valid[k] and x_cm[k] <= R_cm + 1e-9]
373             if len(idx) < 2:
374                 return float("nan")
375             rr = x_cm[idx].astype(float)
376             rowv = np.asarray(row, float)[idx]
377             if surf is not None and R_cm > rr[-1] + 1e-9:
378                 rr = np.r_[rr, R_cm]
379                 rowv = np.r_[rowv, float(surf)]
380             f = rho(rowv) / (1.0 + rowv)
381             return 2 * math.pi * np.trapezoid(f * rr * 1e-2, rr * 1e-2) * L_LEN
382
383             d23 = np.load(OUT / "result23_data.npz", allow_pickle=True)
384             d4 = np.load(OUT / "result4_data.npz", allow_pickle=True)
385             dt23 = float(d23["dt"][0]); dt4 = float(d4["dt"][0])
386             ones = np.ones(21, bool)
387
388             m3_0 = md(d23["conc21"][0], 2.0, rho3, ones)
389             for h, paper in ((12, 89), (30, 110), ("end", 116)):
390                 i = len(d23["conc21"]) - 1 if h == "end" else int(round(h * 3600 / dt23))
391                 val = 100 * (md(d23["conc21"][i], 2.0, rho3, ones) - m3_0) / m3_0
392                 rec("干物质守恒", f"问题三 {h} h / %", paper, round(val), tol=3)
393
394             m4_0 = md(d4["conc21"][0], 2.0, rho4, ones)
395             for h, paper in ((12, -23.1), (30, -13.9), ("end", -10.9)):
396                 i = len(d4["conc21"]) - 1 if h == "end" else int(round(h * 3600 / dt4))
397                 R_cm = float(d4["radius"][i]) * 100.0
398                 val = 100 * (md(d4["conc21"][i], R_cm, rho4, d4["valid21"][i],
399                     surf=d4["conc_surf"][i]) - m4_0) / m4_0
400                 rec("干物质守恒", f"问题四 {h} h / %", paper, round(val, 1), tol=2.0)
401
402     # -----
403     # 5. 论文表 1-表 6 与 result1-4.xlsx 的逐格比对
404     # -----
405     POS5 = [0.0, 0.5, 1.0, 1.5, 2.0] # 正文表用的 5 个位置 / cm
406
407

```

```

408
409 def _tex_table(label: str):
410     """从正文里取出表体, 返回 [(行标签, [单元格字符串...]), ...]。"""
411     src = (PROJECT / "论文" / "论文全文.tex").read_text(encoding="utf-8")
412     for t in re.findall(r"\\begin\\{table\\}(.*?)\\end\\{table\\}", src, re.S):
413         if "\\label{%s}" % label not in t:
414             continue
415         body = t.split("\\midrule", 1)[1].split("\\bottomrule", 1)[0]
416         rows = []
417         for line in body.split("\\\\"):
418             line = re.sub(r"\\cmidrule[^\s]*(\\{[^}]*\\})*", "", line)
419             line = line.replace("\\midrule", "").strip()
420             if not line or line.startswith("\\multicolumn"):
421                 continue
422             cells = [c.strip().replace("\\\\", "").replace("$", "")
423                     for c in line.split("&")]
424             if len(cells) >= 2 and cells[0]:
425                 rows.append((cells[0], cells[1:]))
426         return rows
427     raise KeyError(label)
428
429
430 def audit_result_tables() -> None:
431     """论文表 1-表 6 的每个数字, 都要等于 result1-4.xlsx 对应单元格。"""
432     from openpyxl import load_workbook
433
434     def sheet_rows(path, sheet, rows_wanted, cols_wanted):
435         """取指定行的指定列。返回 {(row, col): value}。"""
436         wb = load_workbook(path, read_only=True, data_only=True)
437         ws = wb[sheet] if sheet else wb.worksheets[0]
438         out = {}
439         want = set(rows_wanted)
440         for ri, row in enumerate(ws.iter_rows(values_only=True), start=1):
441             if ri not in want:
442                 continue
443             for cj in cols_wanted:
444                 out[(ri, cj)] = row[cj - 1] if cj - 1 < len(row) else None
445             if len(out) >= len(want) * len(cols_wanted):
446                 break
447         wb.close()
448         return out
449
450     # ---- 表 1、表 2: 问题一, result1.xlsx (1 s 间隔, 行 = t+1)
451     for label, sheet, tlist in (("tab:p1-T", "温度", [100, 300, 600, 900, 1200, 1500, 1800]),
452                                ("tab:p1-C", "水分浓度", [100, 300, 600, 900, 1200, 1500,
453                                1800])):
454         rows = _tex_table(label)
455         cols = [2 + int(round(p / 0.1)) for p in POS5]
456         data = sheet_rows(OUT / "result1.xlsx", sheet,
457                           [t + 1 for t in tlist], cols)
458         for (t, (rl, cells)) in zip(tlist, rows):
459             for k, c in enumerate(cells):
460                 ref = data.get((t + 1, cols[k]))
461                 got = "%.4f" % float(ref) if ref is not None else None
462                 rec("表%s" % ("1" if label.endswith("T") else "2"),
463                    f"{sheet} t={t}s r={POS5[k]}cm", c, got, tol=5e-5)
464
465     # ---- 表 3、表 4: 问题二, result2.xlsx (1 s, 行 = t+1)
466     for label, sheet in (("tab:p2-T", "温度"), ("tab:p2-C", "水分浓度")):
467         rows = _tex_table(label)
468         tlist = [int(round(float(rl) * 3600)) for rl, _ in rows]
469         cols = [2 + int(round(p / 0.1)) for p in POS5]
470         data = sheet_rows(OUT / "result2.xlsx", sheet,
471                           [t + 1 for t in tlist], cols)
472         for t, (rl, cells) in zip(tlist, rows):
473             for k, c in enumerate(cells):
474                 ref = data.get((t + 1, cols[k]))
475                 got = "%.4f" % float(ref) if ref is not None else None
476                 rec("表%s" % ("3" if sheet == "温度" else "4"),
477                    f"{sheet} t={rl}h r={POS5[k]}cm", c, got, tol=5e-5)
478
479     # ---- 表 5: 问题三, result3.xlsx (60 s, 行 = (t-60)/60 + 2)
480     rows = _tex_table("tab:p3")

```

```

480 wb = load_workbook(OUT / "result3.xlsx", read_only=True, data_only=True)
481 ws = wb.worksheets[0]
482 last = list(ws.iter_rows(values_only=True))[-1]
483 wb.close()
484 cols = [2 + int(round(p / 0.1)) for p in POS5]
485 for rl, cells in rows:
486     if "结束" in rl:
487         rowvals = last
488     else:
489         t = int(float(rl)) * 3600
490         wb = load_workbook(OUT / "result3.xlsx", read_only=True, data_only=True)
491         ws = wb.worksheets[0]
492         rowvals = None
493         for ri, row in enumerate(ws.iter_rows(values_only=True), start=1):
494             if ri == (t - 60) // 60 + 2:
495                 rowvals = row
496                 break
497         wb.close()
498     for k, c in enumerate(cells):
499         ref = rowvals[cols[k] - 1]
500         got = "%.4f" % float(ref) if ref is not None else None
501         rec("表5", f"t={rl} r={POS5[k]}cm", c, got, tol=5e-5)
502
503 # ---- 表 6: 问题四, result4.xlsx (60 s; 末列是"药材表面")
504 rows = _tex_table("tab:p4")
505 wb = load_workbook(OUT / "result4.xlsx", read_only=True, data_only=True)
506 ws = wb.worksheets[0]
507 all_rows = list(ws.iter_rows(values_only=True))
508 wb.close()
509 data = all_rows[1:]
510 for rl, cells in rows:
511     if "结束" in rl:
512         rowvals = data[-1]
513         # result4.xlsx 的行落在 60 s 输出网格上(末行 183840 s), 而正文的
514         # "烘干结束"取判据满足时刻 183885 s, 两者相差不到一个步长,
515         # 因此末行允许 2e-4 的取整差。
516         tol_row = 2e-4
517     else:
518         t = int(float(rl)) * 3600
519         rowvals = data[(t - 60) // 60]
520         tol_row = 5e-5
521     for k, c in enumerate(cells):
522         if k >= 5:
523             ref = rowvals[22]
524             # 第 6 列为药材表面
525         else:
526             ref = rowvals[2 + int(round(POS5[k] / 0.1)) - 1]
527         got = "%.4f" % float(ref) if ref is not None else "——"
528         rec("表6", f"t={rl} {'表面' if k >= 5 else 'r=%.1fcm' % POS5[k]}",
529             c, got, tol=tol_row)
530
531 # -----
532 # 6. §11 判据与代价: 正文数字 vs 收缩判据与代价.json
533 # -----
534 def audit_criterion_section() -> None:
535     p = OUT / "收缩判据与代价.json"
536     if not p.exists():
537         return
538     c = json.loads(p.read_text(encoding="utf-8"))
539     g = c.get("几何对照")
540     if g:
541         rec("§11几何", "固定半径 / h", 57.32, round(g["fixed"] / 3600, 2), tol=0.01)
542         rec("§11几何", "ρ反演收缩 / h", 29.45, round(g["implied"] / 3600, 2), tol=0.01)
543         rec("§11几何", "实测收缩 / h", 23.85, round(g["measured"] / 3600, 2), tol=0.01)
544         rel = c["几何对照相对固定半径"]
545         rec("§11几何", "ρ反演相对固定 / %", -48.6, round(rel["implied"], 1), tol=0.1)
546         rec("§11几何", "实测相对固定 / %", -58.4, round(rel["measured"], 1), tol=0.1)
547     sw = c.get("b扫描")
548     if sw:
549         for b, lit_r, lit_f, lit_i, lit_d in (
550             (0, "0.000", "57.30", "21.27", 169),
551             (128, "0.197", "57.32", "29.45", 95),
552             (320, "0.492", "57.36", "40.59", 41),

```

```

553         (650, "1.000", "57.43", "57.43", 0)):
554     row = next(x for x in sw if abs(x["b"] - b) < 1e-6)
555     rec("$11扫描", f"b={b} r", lit_r, round(row["r"], 3), tol=1e-3)
556     rec("$11扫描", f"b={b} t_fixed / h", lit_f, round(row["t_fixed_h"], 2), tol=0.01)
557     rec("$11扫描", f"b={b} t_implied / h", lit_i, round(row["t_implied_h"], 2), tol
=0.01)
558     rec("$11扫描", f"b={b} Δ / %", lit_d, round(row["delta_pct"]), tol=1)
559     tf = [x["t_fixed_h"] for x in sw]
560     rec("$11扫描", "固定几何全变程 / %", 0.24,
561         round(100 * (max(tf) / min(tf) - 1), 2), tol=0.02)
562     cr = c.get("判据", {})
563     if cr:
564         q = cr["全程 2.55→0.15"]
565         rec("$11判据", "全程 5% r*(径向)", 0.838, round(q["eps=5%"]["r*_radial"], 3), tol=1e-3)
566         rec("$11判据", "全程 5% r*(各向同性)", 0.765, round(q["eps=5%"]["r*_iso"], 3), tol=1e-3)
567         q2 = cr["问题二窗口 2.55→1.364"]
568         rec("$11判据", "问题二窗口 5% r*(径向)", 0.507,
569             round(q2["eps=5%"]["r*_radial"], 3), tol=1e-3)
570         q3 = cr["预热窗口 C 2.55→2.0"]
571         rec("$11判据", "预热 5% r*(径向)", 0.164,
572             round(q3["eps=5%"]["r*_radial"], 3), tol=1e-3)
573         rec("$11判据", "预热 3% r*(径向)", 0.351,
574             round(q3["eps=3%"]["r*_radial"], 3), tol=1e-3)
575     cs = c.get("临界含水率C*", {})
576     if cs:
577         rec("$11判据", "附录3 C*(5%, 径向)", 1.953,
578             round(cs["附录3"]["eps=5%"]["radial"], 3), tol=1e-3)
579         rec("$11判据", "附录4 C*(5%, 径向)", 2.062,
580             round(cs["附录4"]["eps=5%"]["radial"], 3), tol=1e-3)
581     be = c.get("收缩系数", {})
582     if be:
583         rec("$11判据", "beta(附录3)", 0.822, round(be["beta_附录3"], 3), tol=1e-3)
584         rec("$11判据", "beta(附录4)", 0.891, round(be["beta_附录4"], 3), tol=1e-3)
585
586
587 # -----
588 def main() -> None:
589     ap = argparse.ArgumentParser(description="论文数据审计")
590     ap.add_argument("--heavy", action="store_true", help="追加需要重解的项")
591     ap.add_argument("--json", default=None, help="把审计结果写到 JSON")
592     args = ap.parse_args()
593
594     audit_artifacts()
595     audit_noise()
596     audit_p1_grid()
597     audit_p1_smooth()
598     audit_p1_interp()
599     audit_drymass()
600     audit_result_tables()
601     audit_criterion_section()
602
603     bad = [r for r in RECORDS if not r["一致"]]
604     w = max(len(r["项目"]) for r in RECORDS)
605     print("%-10s %-*s %-16s %-16s %s" % ("组", w, "项目", "论文值", "复算值", "判定"))
606     for r in RECORDS:
607         if not r["一致"]:
608             print("x %-8s %-*s %-16s %-16s %s" % (r["组"], w, r["项目"],
609                 r["论文值"], r["复算值"], r["备注"]))
610
611     print()
612     print("共审计 %d 项, 不一致 %d 项" % (len(RECORDS), len(bad)))
613     if args.json:
614         Path(args.json).write_text(json.dumps(RECORDS, ensure_ascii=False, indent=1),
615             encoding="utf-8")
616         print("已写出 ->", args.json)
617
618 if __name__ == "__main__":
619     main()

```

核心算法

空间离散采用节点中心有限体积法, 控制体权重 $w_i = \int \xi \, d\xi$, 其中中心与表面控制体须单独处理:

$$w_0 = \frac{h^2}{8}, \quad w_i = \xi_i h, \quad w_N = \frac{h(1 - h/4)}{2}.$$

若把中心权重误取为 $\xi_0 h = 0$ 、表面权重误取为 $\xi_N h$, 将引入约 1% 的伪守恒误差, 本文在问题一中以该量为判据验证了实现。

时间方向统一采用后向 Euler (无条件稳定), 非线性物性用 Picard 迭代滞后处理, 每个迭代步化为一个三对角方程组, 用追赶法在 $O(N)$ 时间内求解。收敛判据取 $\|\cdot^{(k+1)} - \cdot^{(k)}\|_\infty < 10^{-11}$ 。需要强调的是, Picard 迭代的右端时间层必须固定在本时间步开始时刻, 否则每迭代一次相当于多推进一个 Δt 。

问题四在贴体坐标 $\xi = r/R(t)$ 下求解。因 ξ 同时是材料坐标, 骨架对流项与坐标变换项精确抵消, 方程与问题二、三同形, 只有度量因子 $R(t)^2$ 与表面边界条件中的 $R(t)$ 不同; $R(t)$ 及其解析导数由附件 2 的 PCHIP 插值给出, 而非数值差分 (理由见第 5 节)。

D AI 使用声明

本参赛队在竞赛过程中使用了 AI 工具, 主要用于语言润色、代码调试与论文排版, 详细使用情况见支撑材料; 模型建立、公式推导、数值方案与全部结论均由参赛队独立完成并逐项复核。